

Pacman

Generated by Doxygen 1.16.1

1 Pacman	1
1.1 Assignment	1
1.1.1 Implement the classic game Pac-Man against the computer (you can also implement other variants).	1
1.1.1.1 The game must meet the following functionalities:	1
1.1.1.2 Where to use polymorphism? (recommended)	1
1.1.1.3 Additional information	1
1.1.1.4 Note	2
1.2 Results	2
1.2.0.1 Installing Dependencies	2
1.2.0.2 How to run:	2
1.2.0.3 How to play:	2
1.2.0.4 Documentation:	2
1.2.0.5 Source Code Structure:	2
2 Namespace Index	5
2.1 Namespace List	5
3 Hierarchical Index	7
3.1 Class Hierarchy	7
4 Class Index	9
4.1 Class List	9
5 File Index	13
5.1 File List	13
6 Namespace Documentation	15
6.1 TextureSizes Namespace Reference	15
6.1.1 Detailed Description	15
7 Class Documentation	17
7.1 AIComponent Class Reference	17
7.1.1 Detailed Description	18
7.1.2 Constructor & Destructor Documentation	18
7.1.2.1 AIComponent()	18
7.1.3 Member Function Documentation	18
7.1.3.1 FixedUpdate()	18
7.1.3.2 ResetControlled()	18
7.1.3.3 Update()	19
7.2 Bonus Class Reference	19
7.2.1 Detailed Description	20
7.2.2 Constructor & Destructor Documentation	20
7.2.2.1 Bonus()	20
7.2.3 Member Function Documentation	20
7.2.3.1 FixedUpdate()	20

7.2.3.2 Render()	20
7.2.3.3 Update()	20
7.3 Canvas Class Reference	21
7.3.1 Member Function Documentation	22
7.3.1.1 Render() [1/2]	22
7.3.1.2 Render() [2/2]	22
7.3.1.3 Update()	22
7.4 CatchPlayerCommand Class Reference	22
7.4.1 Detailed Description	23
7.4.2 Member Function Documentation	23
7.4.2.1 Execute()	23
7.5 ChasePlayerCommand Class Reference	23
7.5.1 Detailed Description	24
7.5.2 Member Function Documentation	24
7.5.2.1 CheckDirection()	24
7.5.2.2 DecideDirection()	25
7.5.2.3 Execute()	25
7.6 Command Class Reference	25
7.6.1 Detailed Description	26
7.6.2 Member Function Documentation	26
7.6.2.1 Execute()	26
7.7 Controller Class Reference	26
7.7.1 Detailed Description	27
7.7.2 Constructor & Destructor Documentation	28
7.7.2.1 Controller()	28
7.7.3 Member Function Documentation	28
7.7.3.1 FixedUpdate()	28
7.7.3.2 Update()	28
7.8 ControllerComponent Class Reference	28
7.8.1 Detailed Description	29
7.8.2 Constructor & Destructor Documentation	29
7.8.2.1 ControllerComponent()	29
7.8.3 Member Function Documentation	30
7.8.3.1 ResetControlled()	30
7.8.3.2 Update()	30
7.9 CrossTile Class Reference	30
7.9.1 Detailed Description	32
7.9.2 Constructor & Destructor Documentation	33
7.9.2.1 CrossTile()	33
7.9.3 Member Function Documentation	33
7.9.3.1 CanTurnAside()	33
7.9.3.2 GetType()	33
7.9.3.3 Render()	33

7.10 EmptyTile Class Reference	33
7.10.1 Detailed Description	36
7.10.2 Constructor & Destructor Documentation	36
7.10.2.1 EmptyTile()	36
7.10.3 Member Function Documentation	36
7.10.3.1 AddBonus()	36
7.10.3.2 CanTurnAside()	36
7.10.3.3 CollectBonus()	37
7.10.3.4 GetType()	37
7.10.3.5 Render()	37
7.11 Game Class Reference	37
7.11.1 Detailed Description	37
7.12 GameActor Class Reference	38
7.12.1 Detailed Description	39
7.12.2 Constructor & Destructor Documentation	40
7.12.2.1 GameActor()	40
7.12.3 Member Function Documentation	40
7.12.3.1 Move()	40
7.12.3.2 Render()	40
7.12.3.3 StopMoving()	40
7.12.3.4 TurnDown()	40
7.12.3.5 TurnLeft()	40
7.12.3.6 TurnRight()	40
7.12.3.7 TurnTo()	40
7.12.3.8 TurnUp()	41
7.13 GameAudio Class Reference	41
7.14 GameEvents Class Reference	41
7.14.1 Detailed Description	41
7.14.2 Member Function Documentation	41
7.14.2.1 isQuit()	41
7.14.2.2 isQuitEvent()	42
7.15 GameFilesystem Class Reference	42
7.15.1 Detailed Description	42
7.16 GameGraphics Class Reference	42
7.16.1 Detailed Description	43
7.17 GameLocator Class Reference	43
7.17.1 Detailed Description	43
7.17.2 Member Function Documentation	44
7.17.2.1 GetFilesystem()	44
7.17.2.2 GetMap()	44
7.17.2.3 GetTime()	44
7.17.2.4 ProvideAudio()	44
7.17.2.5 ProvideFilesystem()	44

7.17.2.6 ProvideGraphics()	44
7.17.2.7 ProvideManager()	45
7.17.2.8 ProvideMap()	45
7.17.2.9 ProvideTime()	45
7.18 GameManager Class Reference	45
7.18.1 Detailed Description	47
7.18.2 Constructor & Destructor Documentation	47
7.18.2.1 GameManager()	47
7.19 GameMap Class Reference	47
7.19.1 Detailed Description	47
7.19.2 Constructor & Destructor Documentation	48
7.19.2.1 GameMap()	48
7.19.3 Member Function Documentation	48
7.19.3.1 CentralizePosition()	48
7.19.3.2 GetPixelsPosition()	48
7.19.3.3 LoadMap()	48
7.20 GameObject Class Reference	49
7.20.1 Detailed Description	49
7.20.2 Member Function Documentation	49
7.20.2.1 Update()	49
7.21 GameState Class Reference	49
7.21.1 Detailed Description	51
7.21.2 Member Function Documentation	51
7.21.2.1 InitGameState()	51
7.21.2.2 PlayerCollision()	51
7.21.2.3 Update()	51
7.22 GameStateNormal Class Reference	51
7.22.1 Detailed Description	53
7.22.2 Member Function Documentation	53
7.22.2.1 FixedUpdate()	53
7.22.2.2 InitGameState()	53
7.22.2.3 PlayerCollision()	53
7.22.2.4 SetBonusAmount()	53
7.22.2.5 Update()	53
7.23 GameStateOver Class Reference	54
7.23.1 Detailed Description	55
7.23.2 Member Function Documentation	55
7.23.2.1 FixedUpdate()	55
7.23.2.2 InitGameState()	55
7.23.2.3 Update()	55
7.24 GameStateReset Class Reference	56
7.24.1 Detailed Description	57
7.24.2 Member Function Documentation	57

7.24.2.1 FixedUpdate()	57
7.24.2.2 InitGameState()	57
7.24.2.3 PlayerCollision()	57
7.24.2.4 Update()	58
7.25 GameTime Class Reference	58
7.25.1 Detailed Description	58
7.26 Ghost Class Reference	58
7.26.1 Detailed Description	61
7.26.2 Constructor & Destructor Documentation	61
7.26.2.1 Ghost()	61
7.26.3 Member Function Documentation	61
7.26.3.1 FixedUpdate()	61
7.26.3.2 Move()	61
7.26.3.3 Render()	61
7.26.3.4 StopMoving()	61
7.26.3.5 TurnDown()	61
7.26.3.6 TurnLeft()	61
7.26.3.7 TurnRight()	61
7.26.3.8 TurnTo()	62
7.26.3.9 TurnUp()	62
7.26.3.10 Update()	62
7.27 GraphicsComponent Class Reference	62
7.27.1 Constructor & Destructor Documentation	62
7.27.1.1 GraphicsComponent()	62
7.27.2 Member Function Documentation	63
7.27.2.1 GetTextureSize()	63
7.27.2.2 Render()	63
7.28 MapHeader Struct Reference	63
7.28.1 Detailed Description	63
7.28.2 Member Data Documentation	63
7.28.2.1 spawnPoints__	63
7.29 MapInfo Struct Reference	64
7.29.1 Detailed Description	64
7.30 MapParser Class Reference	64
7.30.1 Detailed Description	65
7.30.2 Constructor & Destructor Documentation	65
7.30.2.1 MapParser()	65
7.30.3 Member Function Documentation	65
7.30.3.1 ParseMap()	65
7.31 MapReader Class Reference	65
7.31.1 Detailed Description	66
7.31.2 Constructor & Destructor Documentation	66
7.31.2.1 MapReader()	66

7.31.3 Member Function Documentation	66
7.31.3.1 Read()	66
7.32 MoveCommand Class Reference	66
7.32.1 Member Function Documentation	67
7.32.1.1 Execute()	67
7.33 NullTexture Class Reference	67
7.33.1 Member Function Documentation	69
7.33.1.1 RenderTexture()	69
7.33.1.2 SetNativeTextureSize()	69
7.34 NullTile Class Reference	69
7.34.1 Detailed Description	71
7.34.2 Constructor & Destructor Documentation	72
7.34.2.1 NullTile()	72
7.34.3 Member Function Documentation	72
7.34.3.1 GetType()	72
7.34.3.2 isObstacle()	72
7.34.3.3 Render()	72
7.35 Pacman Class Reference	72
7.35.1 Detailed Description	75
7.35.2 Constructor & Destructor Documentation	75
7.35.2.1 Pacman()	75
7.35.3 Member Function Documentation	75
7.35.3.1 FixedUpdate()	75
7.35.3.2 Move()	75
7.35.3.3 Render()	75
7.35.3.4 StopMoving()	75
7.35.3.5 TryChangeDirection()	75
7.35.3.6 TurnDown()	75
7.35.3.7 TurnLeft()	76
7.35.3.8 TurnRight()	76
7.35.3.9 TurnTo()	76
7.35.3.10 TurnUp()	76
7.35.3.11 Update()	76
7.36 PhysicsComponent Class Reference	76
7.36.1 Detailed Description	77
7.36.2 Constructor & Destructor Documentation	77
7.36.2.1 PhysicsComponent()	77
7.36.3 Member Function Documentation	77
7.36.3.1 isOnNextTile()	77
7.36.3.2 Move()	78
7.36.3.3 TouchesObstacle()	78
7.37 PlayerController Class Reference	78
7.37.1 Detailed Description	80

7.37.2 Constructor & Destructor Documentation	80
7.37.2.1 PlayerController()	80
7.37.3 Member Function Documentation	80
7.37.3.1 FixedUpdate()	80
7.37.3.2 ResetControlled()	80
7.37.3.3 Update()	80
7.38 PortalTile Class Reference	81
7.38.1 Constructor & Destructor Documentation	84
7.38.1.1 PortalTile()	84
7.38.2 Member Function Documentation	84
7.38.2.1 GetType()	84
7.38.2.2 isPortal()	84
7.38.2.3 Teleport()	84
7.39 StopMoveCommand Class Reference	85
7.39.1 Member Function Documentation	85
7.39.1.1 Execute()	85
7.40 TargetCommand Class Reference	85
7.40.1 Detailed Description	86
7.40.2 Member Function Documentation	86
7.40.2.1 Execute()	86
7.41 Text Class Reference	86
7.41.1 Member Function Documentation	88
7.41.1.1 LoadTexture() [1/2]	88
7.41.1.2 LoadTexture() [2/2]	88
7.41.1.3 RenderTexture() [1/2]	89
7.41.1.4 RenderTexture() [2/2]	89
7.42 Texture Class Reference	89
7.42.1 Detailed Description	90
7.42.2 Member Function Documentation	90
7.42.2.1 CreateRenderRectCentered()	90
7.42.2.2 GetSize()	91
7.42.2.3 LoadTexture() [1/2]	91
7.42.2.4 LoadTexture() [2/2]	91
7.42.2.5 RenderTexture()	91
7.42.2.6 SetNativeTextureSize()	92
7.42.2.7 SetTextureSize()	92
7.43 TextureSize< T_ > Struct Template Reference	92
7.43.1 Detailed Description	93
7.43.2 Constructor & Destructor Documentation	93
7.43.2.1 TextureSize() [1/2]	93
7.43.2.2 TextureSize() [2/2]	93
7.43.3 Member Function Documentation	93
7.43.3.1 Zero()	93

7.44 Tile Class Reference	93
7.44.1 Detailed Description	95
7.44.2 Constructor & Destructor Documentation	95
7.44.2.1 Tile()	95
7.44.3 Member Function Documentation	96
7.44.3.1 AddBonus()	96
7.44.3.2 CanTurnAside()	96
7.44.3.3 CollectBonus()	96
7.44.3.4 FixedUpdate()	96
7.44.3.5 GetOffsetFromTileCenter()	96
7.44.3.6 GetType()	97
7.44.3.7 isObstacle()	97
7.44.3.8 isPortal()	97
7.44.3.9 isTunnel()	97
7.44.3.10 Render()	97
7.44.3.11 Teleport()	97
7.44.3.12 Update()	98
7.45 TileData Struct Reference	98
7.45.1 Detailed Description	98
7.46 TileInfo Struct Reference	98
7.46.1 Detailed Description	99
7.47 TileNode Class Reference	99
7.47.1 Detailed Description	100
7.47.2 Member Function Documentation	100
7.47.2.1 AddBonus()	100
7.47.2.2 CanTurnAside()	101
7.47.2.3 CollectBonus()	101
7.47.2.4 GetOffsetFromNodeCenter()	101
7.47.2.5 GetType()	101
7.47.2.6 isObstacle()	101
7.47.2.7 isPortal()	102
7.47.2.8 isTunnel()	102
7.47.2.9 NextNode()	102
7.47.2.10 Teleport()	102
7.48 TileNodeBuilder Class Reference	102
7.48.1 Detailed Description	103
7.48.2 Member Function Documentation	103
7.48.2.1 LoadTileNodes()	103
7.49 TransformComponent Class Reference	103
7.49.1 Detailed Description	104
7.49.2 Constructor & Destructor Documentation	104
7.49.2.1 TransformComponent() [1/2]	104
7.49.2.2 TransformComponent() [2/2]	104

7.50 TunnelTile Class Reference	105
7.50.1 Constructor & Destructor Documentation	107
7.50.1.1 TunnelTile()	107
7.50.2 Member Function Documentation	108
7.50.2.1 GetType()	108
7.50.2.2 isTunnel()	108
7.51 TurnDownCommand Class Reference	108
7.51.1 Member Function Documentation	109
7.51.1.1 Execute()	109
7.52 TurnLeftCommand Class Reference	109
7.52.1 Member Function Documentation	110
7.52.1.1 Execute()	110
7.53 TurnRightCommand Class Reference	110
7.53.1 Member Function Documentation	110
7.53.1.1 Execute()	110
7.54 TurnUpCommand Class Reference	111
7.54.1 Member Function Documentation	111
7.54.1.1 Execute()	111
7.55 UIElement Class Reference	112
7.55.1 Member Function Documentation	113
7.55.1.1 FixedUpdate()	113
7.55.1.2 Update()	113
7.56 Vector2< T_ > Struct Template Reference	113
7.56.1 Constructor & Destructor Documentation	115
7.56.1.1 Vector2() [1/3]	115
7.56.1.2 Vector2() [2/3]	115
7.56.1.3 Vector2() [3/3]	115
7.56.2 Member Function Documentation	116
7.56.2.1 Abs()	116
7.56.2.2 ConvertTo()	116
7.56.2.3 Distance()	116
7.56.2.4 Down()	116
7.56.2.5 Left()	116
7.56.2.6 One()	116
7.56.2.7 operator=()	117
7.56.2.8 Right()	117
7.56.2.9 ToBack()	117
7.56.2.10 ToLeft()	117
7.56.2.11 ToRight()	117
7.56.2.12 Up()	117
7.56.2.13 Zero()	118
7.57 WallTile Class Reference	118
7.57.1 Detailed Description	120

7.57.2 Constructor & Destructor Documentation	120
7.57.2.1 WallTile()	120
7.57.3 Member Function Documentation	120
7.57.3.1 GetType()	120
7.57.3.2 isObstacle()	121
7.57.3.3 Render()	121
8 File Documentation	123
8.1 AIComponent.hpp	123
8.2 Blinky.hpp	123
8.3 Clyde.hpp	123
8.4 Inky.hpp	123
8.5 Pinky.hpp	123
8.6 AICommands.hpp	124
8.7 AnimationComponent.hpp	124
8.8 Command.hpp	124
8.9 ControllerComponent.hpp	125
8.10 GraphicsComponent.hpp	125
8.11 PhysicsComponent.hpp	125
8.12 PlayerController.hpp	126
8.13 TransformComponent.hpp	126
8.14 Game.hpp	127
8.15 GameAudio.hpp	127
8.16 GameEvents.hpp	127
8.17 GameFilesystem.hpp	128
8.18 GameGraphics.hpp	129
8.19 GameLocator.hpp	129
8.20 GameManager.hpp	131
8.21 GameMap.hpp	131
8.22 GameTime.hpp	132
8.23 Bonus.hpp	132
8.24 Controller.hpp	133
8.25 CrossTile.hpp	133
8.26 EmptyTile.hpp	133
8.27 GameActor.hpp	134
8.28 GameObject.hpp	135
8.29 Ghost.hpp	135
8.30 NullTile.hpp	136
8.31 Pacman.hpp	136
8.32 PortalTile.hpp	137
8.33 Tile.hpp	137
8.34 TunnelTile.hpp	138
8.35 WallTile.hpp	138

8.36 GameState.hpp	138
8.37 GameStateNormal.hpp	139
8.38 GameStateOver.hpp	139
8.39 GameStateReset.hpp	140
8.40 MapParser.hpp	140
8.41 MapReader.hpp	141
8.42 NullTexture.hpp	142
8.43 Text.hpp	142
8.44 Texture.hpp	143
8.45 TextureSize.hpp	143
8.46 TextureSize.inl	144
8.47 TileNode.hpp	144
8.48 TileNodeBuilder.hpp	146
8.49 Vector2.hpp	146
8.50 Vector2.inl	148
8.51 Canvas.hpp	151
8.52 UIElement.hpp	151
Index	153

Chapter 1

Pacman

This software was developed with the support of the Faculty of Information Technology, Czech Technical University in Prague. For more information, visit fit.cvut.cz.

1.1 Assignment

1.1.1 Implement the classic game Pac-Man against the computer (you can also implement other variants).

1.1.1.1 The game must meet the following functionalities:

- Adherence to basic rules, collecting bonuses, coins (dots), cherries, teleporters, etc.
- Implementation of several types of ghost AI. The game must be able to display different ghosts with different AIs simultaneously. To improve clarity, ghosts with different AIs can be displayed using different colors or symbols.
- Ability to load maps from a file (maze + positions of bonuses and ghosts).
- Configurable game difficulty from a file (ghost speed, duration of individual game modes, probability of adding a new bonus to the map, etc.)
- Scalable ghost difficulty (various ghost strategies based on difficulty and behavior)

1.1.1.2 Where to use polymorphism? (recommended)

- Player types: player, ghost (various types of artificial intelligence)
- **Game** mode: classic (ghosts move according to their strategies), frantic (ghosts chase the player), invulnerable (ghosts are slower, can be killed)
- **Game** tile: ground, wall, dot, bonus (changes mode), cherry (more points), teleport
- User interface: console, ncurses, SDL, OpenGL (various variants), etc.

1.1.1.3 Additional information

[Pac-Man - Wikipedia](#)

[Ghost behavior in Pac-Man Part 1](#)

[Ghost behavior in Pac-Man Part 2](#)

1.1.1.4 Note

This task is graded by an automatic system and then by the instructor. First, submit and debug the task using the automatic system, then submit one of your chosen solutions for evaluation by the instructor. Submission for instructor evaluation is done by clicking the "Submit to Instructor" button on the selected solution. The instructor will evaluate only one of your solutions and will not begin evaluating until you submit the task. Until the task is evaluated by both the automatic system and the instructor, no points are awarded (only partial ratings in percentages are displayed).

1.2 Results

Application is created using SDL graphic library.

Also look at video how it works (video in some moments is lagging, but it is recording lag, not of the game): [pacman_showcase.webm](#) [pacman_showcase.mp4](#)

1.2.0.1 Installing Dependencies

For running the game:

```
sudo apt install libsdl2-dev libsdl2-image-dev libsdl2-ttf-dev
```

For building documentation:

```
sudo apt install doxygen
```

1.2.0.2 How to run:

```
# build
make all
# run
make run
```

1.2.0.3 How to play:

- Hold arrows to change direction of the pacman.
- User portals to slow down the ghost and to teleport.
- Pick up all the scores to win.
- [Game](#) is over when you lose all the lives.

1.2.0.4 Documentation:

To generate documentation run:

```
make doc
```

Then open doc/html/index.html to read generated documentation.

For pdf, there is already generated pdf file, you can read it. For manual generation the texlive (on Linux) is required, install the following packages (note, it will take several GBs of memory):

```
texlive-scheme-full
```

Go to the ./doc/latex directory and run make. The pdf file should be generated. If any latex errors appear, probably you will have to install some packages, otherwise it is up to you to troubleshoot them. Short showcase of documentation:

1.2.0.5 Source Code Structure:

```
src
  components
    AICommands.cpp
    AICommands.hpp
    AI_components
      AIComponent.cpp
      AIComponent.hpp
      Blinky.cpp
      Blinky.hpp
      Clyde.cpp
      Clyde.hpp
      Inky.cpp
      Inky.hpp
```



```
Pinky.cpp
Pinky.hpp
AnimationComponent.cpp
AnimationComponent.hpp
Command.cpp
Command.hpp
ControllerComponent.cpp
ControllerComponent.hpp
GraphicsComponent.cpp
GraphicsComponent.hpp
PhysicsComponent.cpp
PhysicsComponent.hpp
PlayerController.cpp
PlayerController.hpp
TransformComponent.cpp
TransformComponent.hpp
game_engine
GameAudio.cpp
GameAudio.hpp
Game.cpp
GameEvents.cpp
GameEvents.hpp
GameFilesystem.cpp
GameFilesystem.hpp
GameGraphics.cpp
GameGraphics.hpp
Game.hpp
GameLocator.cpp
GameLocator.hpp
GameManager.cpp
GameManager.hpp
GameMap.cpp
GameMap.hpp
GameTime.cpp
GameTime.hpp
game_objects
Bonus.cpp
Bonus.hpp
Controller.cpp
Controller.hpp
CrossTile.cpp
CrossTile.hpp
EmptyTile.cpp
EmptyTile.hpp
GameActor.cpp
GameActor.hpp
GameObject.cpp
GameObject.hpp
Ghost.cpp
Ghost.hpp
NullTile.cpp
NullTile.hpp
Pacman.cpp
Pacman.hpp
PortalTile.cpp
PortalTile.hpp
Tile.cpp
Tile.hpp
TunnelTile.cpp
TunnelTile.hpp
WallTile.cpp
WallTile.hpp
game_states
GameState.cpp
GameState.hpp
GameStateNormal.cpp
GameStateNormal.hpp
GameStateOver.cpp
GameStateOver.hpp
GameStateReset.cpp
GameStateReset.hpp
main.cpp
structures
MapParser.cpp
MapParser.hpp
MapReader.cpp
MapReader.hpp
NullTexture.cpp
NullTexture.hpp
Text.cpp
Text.hpp
Texture.cpp
Texture.hpp
TextureSize.hpp
TextureSize.inl
TileNodeBuilder.cpp
TileNodeBuilder.hpp
```

- TileNode.cpp
- TileNode.hpp
- Vector2.hpp
- Vector2.inl

- UI

- Canvas.cpp
 - Canvas.hpp
 - UIElement.cpp
 - UIElement.hpp

7 directories, 101 files

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

[TextureSizes](#)

Namespace of all standard texture sizes used for game actors, tiles and other game object [15](#)

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Command	25
MoveCommand	66
StopMoveCommand	85
TurnDownCommand	108
TurnLeftCommand	109
TurnRightCommand	110
TurnUpCommand	111
ControllerComponent	28
AIComponent	17
PlayerController	78
Game	37
GameAudio	41
GameEvents	41
GameFilesystem	42
GameGraphics	42
GameLocator	43
GameManager	45
GameMap	47
GameObject	49
Bonus	19
Controller	26
GameActor	38
Ghost	58
Pacman	72
Tile	93
EmptyTile	33
CrossTile	30
TunnelTile	105
PortalTile	81
NullTile	69
WallTile	118
UIElement	112
Canvas	21
GameState	49
GameStateNormal	51
GameStateReset	56
GameStateOver	54
GameTime	58
GraphicsComponent	62

MapHeader	63
MapInfo	64
MapParser	64
MapReader	65
PhysicsComponent	76
TargetCommand	85
CatchPlayerCommand	22
ChasePlayerCommand	23
Texture	89
NullTexture	67
Text	86
TextureSize< T_ >	92
TileData	98
TileInfo	98
TileNode	99
TileNodeBuilder	102
TransformComponent	103
Vector2< T_ >	113

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AIComponent	Class that represents AI behavior component, defined through different commands . . .	17
Bonus	Class that represents bonus game object	19
Canvas		21
CatchPlayerCommand	Command to try to catch target. If caught - report to game manager about the catch	22
ChasePlayerCommand	TargetCommand to chase player given player object	23
Command	Class which represents commands executed with given game actor object	25
Controller	Game Object subclass which controls game actor objects. Uses controller component which controls assigned game actor to it	26
ControllerComponent	Controller component class which controls given game object to this component . . .	28
CrossTile	Class that represents Cross tile subclass of the Tile superclass	30
EmptyTile	Class that represents Empty tile subclass of the Tile superclass	33
Game	Class which represents the game, initializing, starting, updating, rendering and quitting the game	37
GameActor	GameActor abstract class specification of the GameObject	38
GameAudio		41
GameEvents	Low level class that handles SDL events	41
GameFilesystem	Game Filesystem Class which handles media loading and storage	42
GameGraphics	Class that initialize overall graphics for the game. Initialize window, renderer, and clearing and rendering on the screen	42
GameLocator	Locator class which provides global access to global game services	43
GameManager	Class which manages game, what happens there and affect game objects	45
GameMap	Class that represents game maps in the game world	47

GameObject	Interface for all game objects in the game	49
GameState	Class that represents game state	49
GameStateNormal	Represents normal state of the game	51
GameStateOver	Represents game over state of the game, when player wins	54
GameStateReset	Class that represents game state on reset after collision	56
GameTime	Game Time class which handles time across all the game	58
Ghost	Pacman class - concrete specification of the GameActor , which defines its moving algorithms and what it does	58
GraphicsComponent		62
MapHeader	Structure that represents header info in the game map file	63
MapInfo	Structure that represents general information about the map	64
MapParser	Class that parses tile data objects to tile info objects, used for building tiles	64
MapReader	Class that reads map file and stores map chars in the 2D vector for further parsing or processing, checking file stream and map size from map header information	65
MoveCommand		66
NullTexture		67
NullTile	Class that represents Null tile subclass of the Tile superclass	69
Pacman	Pacman class - concrete specification of the GameActor , which defines its moving algorithms and what it does	72
PhysicsComponent	Component which resolves physics aspects of the game object. Resolves collisions and interaction with other game objects, while transform component only represents position in the world	76
PlayerController	Controller to get user input and control given actor	78
PortalTile		81
StopMoveCommand		85
TargetCommand	Represent command where are interacting two objects	85
Text		86
Texture	Texture wrapper class around SDL's texture	89
TextureSize< T_ >	Template which represents size of the texture	92
Tile	Abstract class representing Tiles in the map	93
TileData	Structure that represents read data about the tile from each layer of the game map	98
TileInfo	Structure that represents information about tile	98
TileNode	Represents connections between each adjacent tile in the map	99
TileNodeBuilder	Represents class that builds all tile nodes using tile info and connects them with each other	102

TransformComponent	
Class which represents coordinates and direction in the game world	103
TunnelTile	105
TurnDownCommand	108
TurnLeftCommand	109
TurnRightCommand	110
TurnUpCommand	111
UIElement	112
Vector2< T_ >	113
WallTile	
Class that represents Wall tile subclass of the Tile superclass	118

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

src/components/ AICommands.hpp	124
src/components/ AnimationComponent.hpp	124
src/components/ Command.hpp	124
src/components/ ControllerComponent.hpp	125
src/components/ GraphicsComponent.hpp	125
src/components/ PhysicsComponent.hpp	125
src/components/ PlayerController.hpp	126
src/components/ TransformComponent.hpp	126
src/components/AI_components/ AIComponent.hpp	123
src/components/AI_components/ Blinky.hpp	123
src/components/AI_components/ Clyde.hpp	123
src/components/AI_components/ Inky.hpp	123
src/components/AI_components/ Pinky.hpp	123
src/game_engine/ Game.hpp	127
src/game_engine/ GameAudio.hpp	127
src/game_engine/ GameEvents.hpp	127
src/game_engine/ GameFilesystem.hpp	128
src/game_engine/ GameGraphics.hpp	129
src/game_engine/ GameLocator.hpp	129
src/game_engine/ GameManager.hpp	131
src/game_engine/ GameMap.hpp	131
src/game_engine/ GameTime.hpp	132
src/game_objects/ Bonus.hpp	132
src/game_objects/ Controller.hpp	133
src/game_objects/ CrossTile.hpp	133
src/game_objects/ EmptyTile.hpp	133
src/game_objects/ GameActor.hpp	134
src/game_objects/ GameObject.hpp	135
src/game_objects/ Ghost.hpp	135
src/game_objects/ NullTile.hpp	136
src/game_objects/ Pacman.hpp	136
src/game_objects/ PortalTile.hpp	137
src/game_objects/ Tile.hpp	137
src/game_objects/ TunnelTile.hpp	138
src/game_objects/ WallTile.hpp	138
src/game_states/ GameState.hpp	138
src/game_states/ GameStateNormal.hpp	139
src/game_states/ GameStateOver.hpp	139
src/game_states/ GameStateReset.hpp	140
src/structures/ MapParser.hpp	140

src/structures/MapReader.hpp	141
src/structures/NullTexture.hpp	142
src/structures/Text.hpp	142
src/structures/Texture.hpp	143
src/structures/TextureSize.hpp	143
src/structures/TextureSize.inl	144
src/structures/TileNode.hpp	144
src/structures/TileNodeBuilder.hpp	146
src/structures/Vector2.hpp	146
src/structures/Vector2.inl	148
src/UI/Canvas.hpp	151
src/UI/UIElement.hpp	151

Chapter 6

Namespace Documentation

6.1 TextureSizes Namespace Reference

Namespace of all standard texture sizes used for game actors, tiles and other game object.

Variables

- const [TextureSize](#)< float > ACTOR (40, 40)
- const [TextureSize](#)< float > TILE (22, 22)
- const [TextureSize](#)< float > BONUS (5, 5)

6.1.1 Detailed Description

Namespace of all standard texture sizes used for game actors, tiles and other game object.

Chapter 7

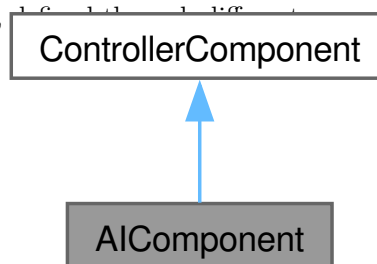
Class Documentation

7.1 AIComponent Class Reference

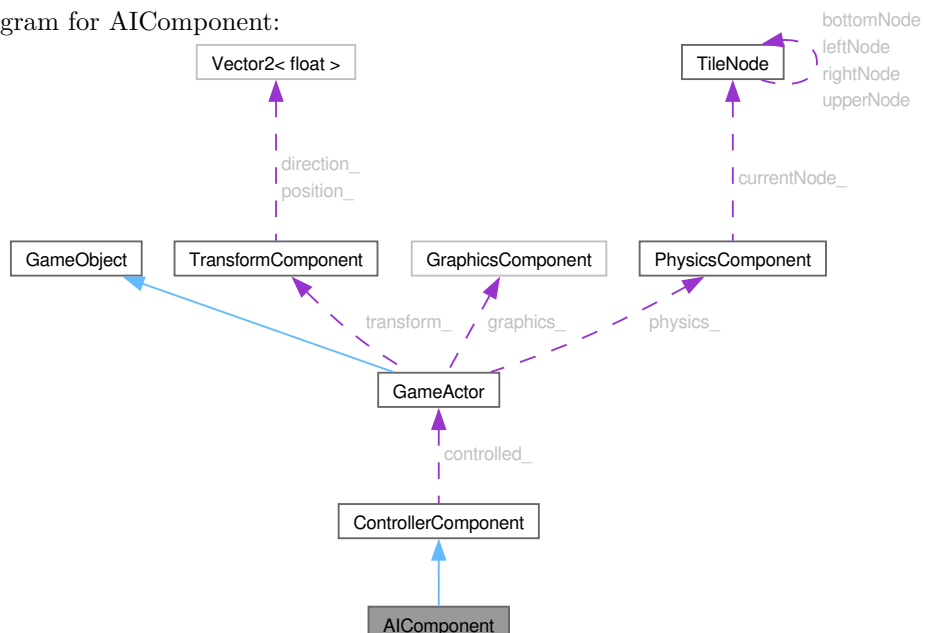
Class that represents AI behavior component, `AIComponent` and `ControllerComponent` depends.

`#include <AIComponent.hpp>`

Inheritance diagram for AIComponent:



Collaboration diagram for AIComponent:



Public Member Functions

- `AIComponent` (`GameActor` *controlled, `GameActor` *player)

- Constructs [AIComponent](#).
- `~AIComponent ()` override
Destroys AI component with its allocated commands.
- `void Update ()` override
Updates controller component.
- `void FixedUpdate ()` override
- `void ResetControlled ()` override
Resets controlled game actor to some status.

Public Member Functions inherited from [ControllerComponent](#)

- `ControllerComponent (GameActor *actor)`
Constructs controller component.
- `virtual ~ControllerComponent ()=default`
Destroys component but not the actor. Actor is destroyed by game itself when closing.

Additional Inherited Members

Protected Attributes inherited from [ControllerComponent](#)

- `GameActor * controlled_`
`Game` Actor under control of the given object.

7.1.1 Detailed Description

Class that represents AI behavior component, defined through different commands.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 AIComponent()

`AIComponent::AIComponent (`
 [GameActor](#) * controlled,
 [GameActor](#) * player)
Constructs [AIComponent](#).

Parameters

controlled	object that is controlled by the AI.
player	object of the player, used in target commands.

7.1.3 Member Function Documentation

7.1.3.1 FixedUpdate()

`void AIComponent::FixedUpdate ()` [override], [virtual]
Implements [ControllerComponent](#).

7.1.3.2 ResetControlled()

`void AIComponent::ResetControlled ()` [override], [virtual]
Resets controlled game actor to some status.
Implements [ControllerComponent](#).

7.1.3.3 Update()

```
void AIComponent::Update () [override], [virtual]
```

Updates controller component.

Implements [ControllerComponent](#).

The documentation for this class was generated from the following files:

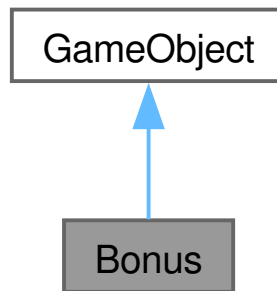
- src/components/AI_components/AIComponent.hpp
- src/components/AI_components/AIComponent.cpp

7.2 Bonus Class Reference

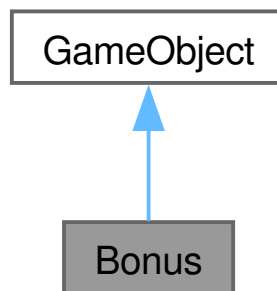
Class that represents bonus game object.

```
#include <Bonus.hpp>
```

Inheritance diagram for Bonus:



Collaboration diagram for Bonus:



Public Member Functions

- [Bonus](#) ([GraphicsComponent](#) *graphics)
Constructs bonus game object.
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- void [Render](#) (const [Vector2](#)< float > &position)
Shows bonus graphics on the screen each frame.

Public Member Functions inherited from [GameObject](#)

- virtual [~GameObject](#) ()=default
Default destructor of the game object.

7.2.1 Detailed Description

Class that represents bonus game object.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 Bonus()

Bonus::Bonus (
 [GraphicsComponent](#) * graphics) [explicit]
Constructs bonus game object.

Parameters

graphics	component of the bonus game object.
----------	-------------------------------------

7.2.3 Member Function Documentation

7.2.3.1 FixedUpdate()

void Bonus::FixedUpdate () [override], [virtual]
Implements [GameObject](#).

7.2.3.2 Render()

void Bonus::Render (
 const [Vector2](#)< float > & position)
Shows bonus graphics on the screen each frame.

Parameters

position	of the tile where this bonus attached.
----------	--

7.2.3.3 Update()

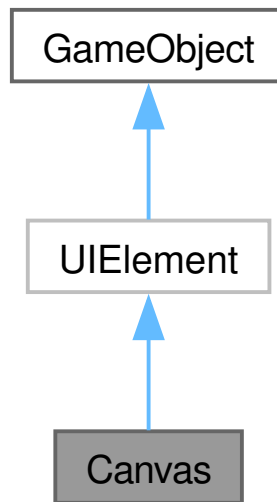
void Bonus::Update () [override], [virtual]
Updates this game object in the game.
Implements [GameObject](#).

The documentation for this class was generated from the following files:

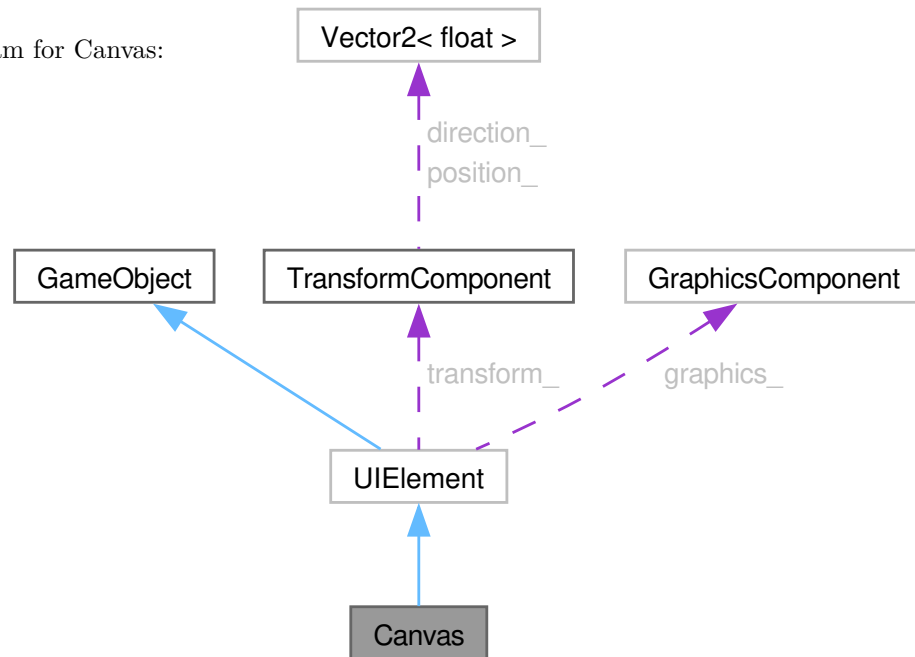
- src/game_objects/Bonus.hpp
- src/game_objects/Bonus.cpp

7.3 Canvas Class Reference

Inheritance diagram for Canvas:



Collaboration diagram for Canvas:



Public Member Functions

- Canvas ([TransformComponent](#) *transform, [GraphicsComponent](#) *graphics)
- void [Render](#) () const override
- void [Render](#) (const [Vector2](#)< float > &position) const override
- void RenderScore () const
- void DecreaseLives ()
- void [Update](#) () override

Updates this game object in the game.

Public Member Functions inherited from [UIElement](#)

- [UIElement](#) ([TransformComponent](#) *transform, [GraphicsComponent](#) *graphics)
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- [Vector2](#)< float > GetPosition () const
- void ChangeGraphicsTexture (const string &pathOrMessage)

Public Member Functions inherited from [GameObject](#)

- virtual ~[GameObject](#) ()=default
Default destructor of the game object.

Additional Inherited Members

Protected Attributes inherited from [UIElement](#)

- [TransformComponent](#) * transform__
- [GraphicsComponent](#) * graphics__

7.3.1 Member Function Documentation

7.3.1.1 [Render\(\)](#) [1/2]

void [Canvas::Render](#) () const [override], [virtual]
Reimplemented from [UIElement](#).

7.3.1.2 [Render\(\)](#) [2/2]

void [Canvas::Render](#) (
const [Vector2](#)< float > & position) const [override], [virtual]
Reimplemented from [UIElement](#).

7.3.1.3 [Update\(\)](#)

void [Canvas::Update](#) () [override], [virtual]
Updates this game object in the game.
Implements [GameObject](#).

The documentation for this class was generated from the following files:

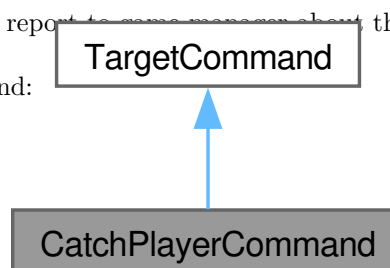
- src/UI/Canvas.hpp
- src/UI/Canvas.cpp

7.4 [CatchPlayerCommand](#) Class Reference

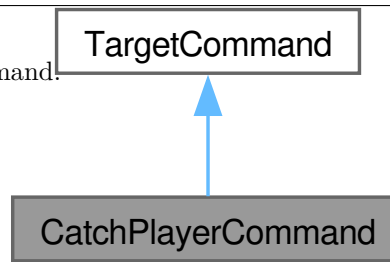
[Command](#) to try to catch target. If caught - report the catch.

#include <AICommands.hpp>

Inheritance diagram for [CatchPlayerCommand](#):



Collaboration diagram for CatchPlayerCommand:



Public Member Functions

- void [Execute](#) ([GameActor](#) *controlled, [GameActor](#) *target) override
Executes command with give game actor objects.

7.4.1 Detailed Description

[Command](#) to try to catch target. If caught - report to game manager about the catch.

7.4.2 Member Function Documentation

7.4.2.1 Execute()

void `CatchPlayerCommand::Execute` (
[GameActor](#) * controlled,
[GameActor](#) * target) [override], [virtual]
 Executes command with give game actor objects.

Parameters

controlled	first object which does something.
target	which is affected by the controlled object.

Implements [TargetCommand](#).

The documentation for this class was generated from the following files:

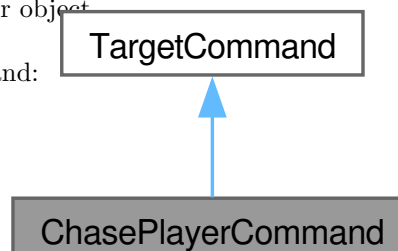
- `src/components/AICommands.hpp`
- `src/components/AICommands.cpp`

7.5 ChasePlayerCommand Class Reference

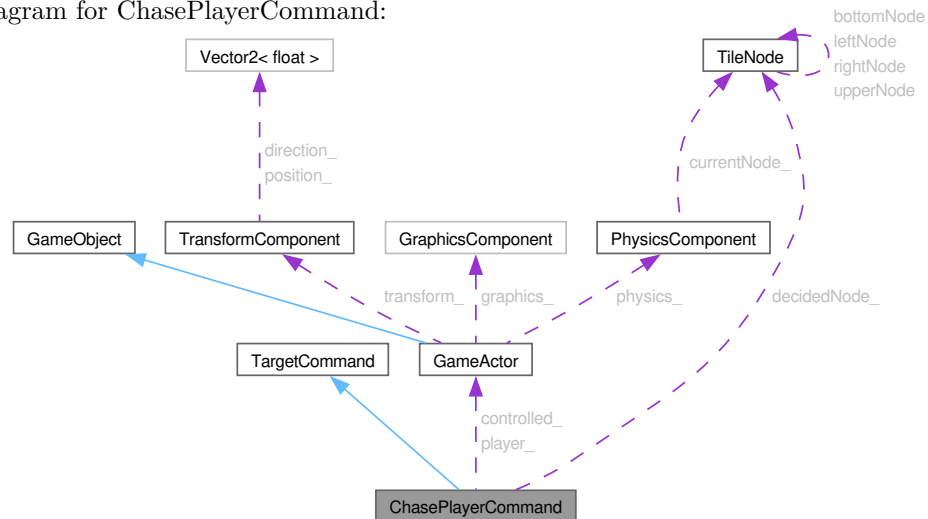
[TargetCommand](#) to chase player given player object.

`#include <AICommands.hpp>`

Inheritance diagram for ChasePlayerCommand:



Collaboration diagram for ChasePlayerCommand:



Public Member Functions

- void [Execute](#) ([GameActor](#) *controlled, [GameActor](#) *target) override
Executes command with give game actor objects.

Protected Member Functions

- void [CheckTile](#) ()
Check whether current tile is cross tile where can decide where to go.
- [Vector2< float >](#) [DecideDirection](#) ()
Returns direction where to turn.
- void [CheckDirection](#) (const [Vector2< float >](#) &direction, pair< [Vector2< float >](#), float > &decision) const
Check if specified direction is better that previously best decision.

Protected Attributes

- [GameActor](#) * player_
Player game actor object.
- [GameActor](#) * controlled_
[Game](#) actor object under control of the command.
- const [TileNode](#) * decidedNode_
Represents last tile node where decision already has been made.

7.5.1 Detailed Description

[TargetCommand](#) to chase player given player object.

7.5.2 Member Function Documentation

7.5.2.1 CheckDirection()

```
void ChasePlayerCommand::CheckDirection (
    const Vector2< float > & direction,
    pair< Vector2< float >, float > & decision) const    [protected]
```

Check if specified direction is better that previously best decision.

Parameters

direction	where to check if is better direction to turn.
decision	currently best decision where to turn.

7.5.2.2 DecideDirection()

[Vector2](#)< float > ChasePlayerCommand::DecideDirection () [protected]

Returns direction where to turn.

Returns

direction to turn.

7.5.2.3 Execute()

void ChasePlayerCommand::Execute (
[GameActor](#) * controlled,
[GameActor](#) * target) [override], [virtual]

Executes command with give game actor objects.

Parameters

controlled	first object which does something.
target	which is affected by the controlled object.

Implements [TargetCommand](#).

The documentation for this class was generated from the following files:

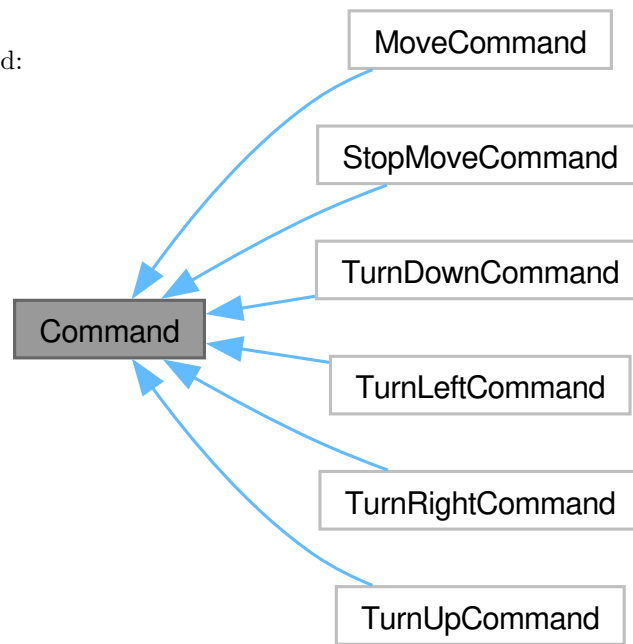
- src/components/AICommands.hpp
- src/components/AICommands.cpp

7.6 Command Class Reference

Class which represents commands executed with given game actor object.

#include <Command.hpp>

Inheritance diagram for Command:



Public Member Functions

- virtual void [Execute](#) ([GameActor](#) *actor)=0
Executes command with give actor.

7.6.1 Detailed Description

Class which represents commands executed with given game actor object.

7.6.2 Member Function Documentation

7.6.2.1 Execute()

virtual void [Command::Execute](#) (
[GameActor](#) * actor) [pure virtual]

Executes command with give actor.

Parameters

actor	which should execute the command.
-------	-----------------------------------

Implemented in [MoveCommand](#), [StopMoveCommand](#), [TurnDownCommand](#), [TurnLeftCommand](#), [TurnRightCommand](#), and [TurnUpCommand](#).

The documentation for this class was generated from the following file:

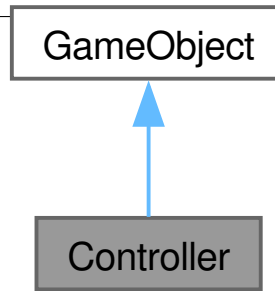
- src/components/Command.hpp

7.7 Controller Class Reference

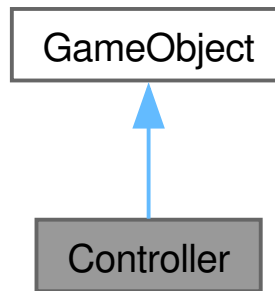
[Game](#) Object subclass which controls game actor objects. Uses controller component which controls assigned game actor to it.

```
#include <Controller.hpp>
```


Inheritance diagram for Controller:



Collaboration diagram for Controller:



Public Types

- enum State { ENABLED , DISABLED }

Public Member Functions

- [Controller](#) ([ControllerComponent](#) *controller)
Construct this controller object.
- [~Controller](#) () override
Destructor - destroys controller and its components.
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- [ControllerComponent](#) * [GetController](#) ()
Get controller component.
- void [SetState](#) (int value)
Set controller state.

Public Member Functions inherited from [GameObject](#)

- virtual [~GameObject](#) ()=default
Default destructor of the game object.

7.7.1 Detailed Description

[Game](#) Object subclass which controls game actor objects. Uses controller component which controls assigned game actor to it.

7.7.2 Constructor & Destructor Documentation

7.7.2.1 Controller()

Controller::Controller (
[ControllerComponent](#) * controller) [explicit]
 Construct this controller object.

Parameters

controller	component which defines controller behavior.
------------	--

7.7.3 Member Function Documentation

7.7.3.1 FixedUpdate()

void Controller::FixedUpdate () [override], [virtual]
 Implements [GameObject](#).

7.7.3.2 Update()

void Controller::Update () [override], [virtual]
 Updates this game object in the game.
 Implements [GameObject](#).

The documentation for this class was generated from the following files:

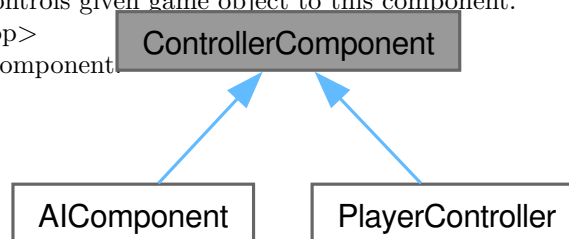
- src/game_objects/Controller.hpp
- src/game_objects/Controller.cpp

7.8 ControllerComponent Class Reference

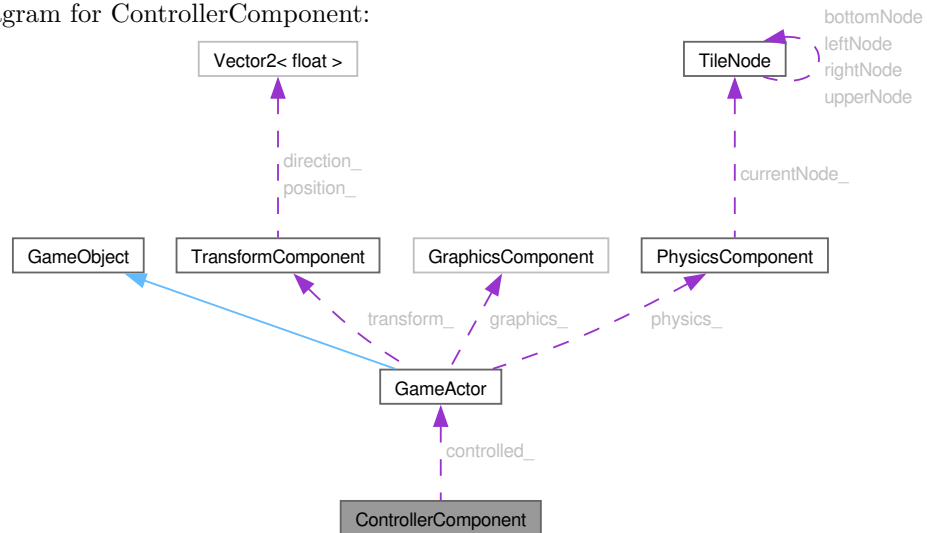
[Controller](#) component class which controls given game object to this component.

#include <ControllerComponent.hpp>

Inheritance diagram for ControllerComponent



Collaboration diagram for ControllerComponent:



Public Member Functions

- [ControllerComponent](#) ([GameActor](#) *actor)
Constructs controller component.
- virtual [~ControllerComponent](#) ()=default
Destroys component but not the actor. Actor is destroyed by game itself when closing.
- virtual void [Update](#) ()=0
Updates controller component.
- virtual void [FixedUpdate](#) ()=0
- virtual void [ResetControlled](#) ()=0
Resets controlled game actor to some status.

Protected Attributes

- [GameActor](#) * controlled_
[Game](#) Actor under control of the given object.

7.8.1 Detailed Description

[Controller](#) component class which controls given game object to this component.

7.8.2 Constructor & Destructor Documentation

7.8.2.1 ControllerComponent()

`ControllerComponent::ControllerComponent (`
 [GameActor](#) * actor) [explicit]

Constructs controller component.

Parameters

actor	to be controlled by this controller.
-------	--------------------------------------

7.8.3 Member Function Documentation

7.8.3.1 ResetControlled()

virtual void ControllerComponent::ResetControlled () [pure virtual]

Resets controlled game actor to some status.

Implemented in [AIComponent](#), and [PlayerController](#).

7.8.3.2 Update()

virtual void ControllerComponent::Update () [pure virtual]

Updates controller component.

Implemented in [AIComponent](#), and [PlayerController](#).

The documentation for this class was generated from the following files:

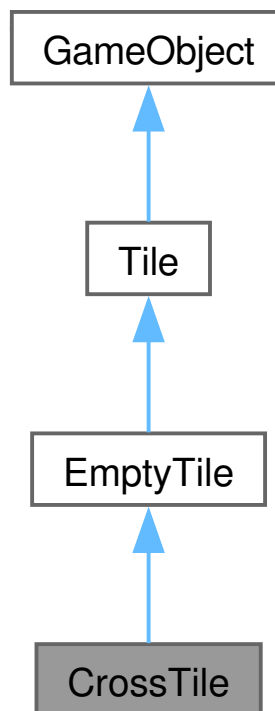
- src/components/ControllerComponent.hpp
- src/components/ControllerComponent.cpp

7.9 CrossTile Class Reference

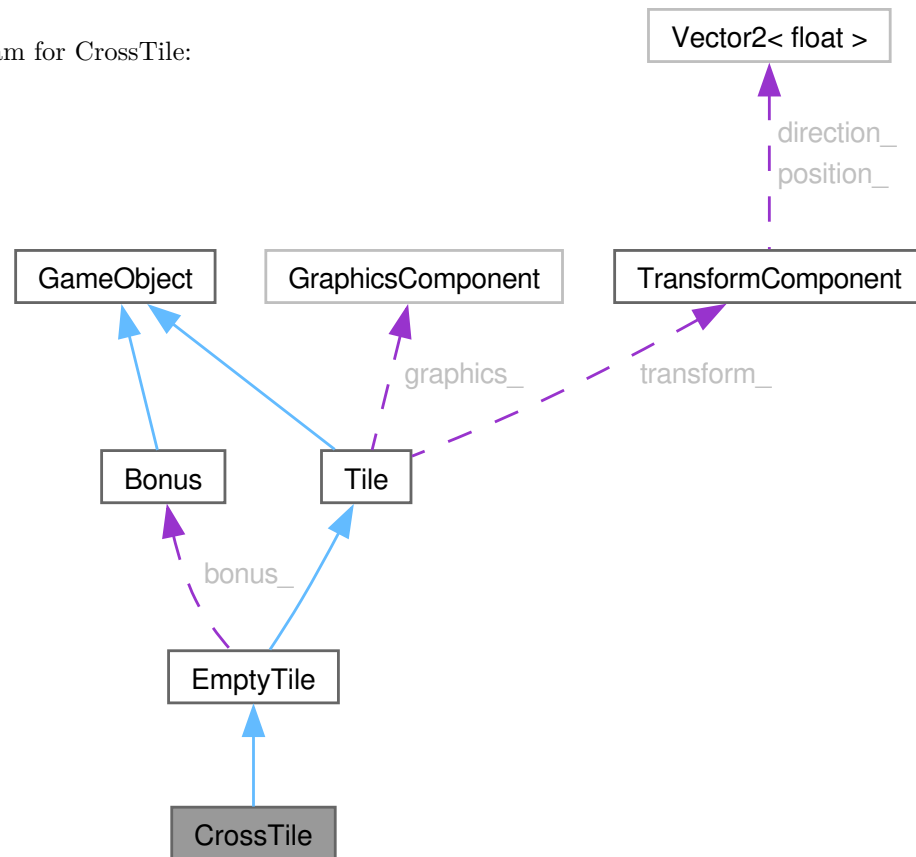
Class that represents Cross tile subclass of the [Tile](#)

`#include <CrossTile.hpp>`

Inheritance diagram for CrossTile:



Collaboration diagram for CrossTile:



Public Member Functions

- [CrossTile](#) ([Texture](#) *texture, const [Vector2< float >](#) &position)
Construct a new Empty Cross object.
- void [Render](#) () const override
Render the tile on the game map. O(1).
- bool [CanTurnAside](#) (const [GameActor](#) &gameActor) const override
Check if game object standing on this tile can turn to the side, right or left or not. O(1).
- TileType [GetType](#) () const override
Get the type of the tile. O(1).

Public Member Functions inherited from [EmptyTile](#)

- [EmptyTile](#) ([Texture](#) *texture, const [Vector2< float >](#) &position)
Construct a new Empty [Tile](#) object.
- void [Render](#) () const override
Render the tile on the game map. O(1).
- bool [AddBonus](#) ([Bonus](#) *bonus) override
Add bonus to tile.
- bool [CollectBonus](#) () override
Collect bonus from tile.
- TileType [GetType](#) () const override
Get the type of the tile. O(1).
- bool [CanTurnAside](#) (const [GameActor](#) &gameActor) const override
Check if game object standing on this tile can turn to the side, right or left or not. O(1).

Public Member Functions inherited from [Tile](#)

- [Tile](#) ([Texture](#) *texture, const [Vector2](#)< float > &position)
Construct a new [Tile](#) object. O(1).
- [Tile](#) ([Texture](#) *texture, [TransformComponent](#) *transform)
- [~Tile](#) () override
Destroys the [Tile](#) object. O(1).
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- virtual bool [isObstacle](#) () const
Check if this tile is obstacle or not. O(1).
- virtual bool [isTunnel](#) () const
Check if this tile is tunnel.
- virtual bool [isPortal](#) () const
Check if this tile is portal.
- virtual void [Teleport](#) ([GameActor](#) &gameActor, [TileNode](#) &portalNode)
Moves game actor transform and its current tile to another connected portalNode.
- [TransformComponent](#) * [GetTransform](#) ()
Get Transform component of the tile. O(1).
- const [TransformComponent](#) * [GetTransform](#) () const
Get Transform component of the tile. O(1).

Public Member Functions inherited from [GameObject](#)

- virtual [~GameObject](#) ()=default
Default destructor of the game object.

Additional Inherited Members

Static Public Member Functions inherited from [Tile](#)

- static [Vector2](#)< float > [GetOffsetFromTileCenter](#) (const [GameActor](#) &gameActor, const [Tile](#) *tile)
Get distance between game object and its tile center according to the current direction. O(1).

Protected Attributes inherited from [EmptyTile](#)

- [Bonus](#) * bonus_
[Bonus](#) on the tile.

Protected Attributes inherited from [Tile](#)

- [GraphicsComponent](#) * graphics_
Graphics component of the tile class.
- [TransformComponent](#) * transform_
Transform component of the tile class.

7.9.1 Detailed Description

Class that represents Cross tile subclass of the [Tile](#) superclass.

7.9.2 Constructor & Destructor Documentation

7.9.2.1 CrossTile()

CrossTile::CrossTile (
 [Texture](#) * texture,
 const [Vector2](#)< float > & position)
 Construct a new Empty Cross object.

Parameters

texture	for graphics component to render the tile.
position	of the tile in the world.

7.9.3 Member Function Documentation

7.9.3.1 CanTurnAside()

bool CrossTile::CanTurnAside (
 const [GameActor](#) & gameActor) const [nodiscard], [override], [virtual]
 Check if game object standing on this tile can turn to the side, right or left or not. O(1).

Parameters

gameActor	standing on this tile and trying to turn to another direction.
-----------	--

Returns

true if game object can turn aside.

Reimplemented from [Tile](#).

7.9.3.2 GetType()

TileType CrossTile::GetType () const [nodiscard], [override], [virtual]
 Get the type of the tile. O(1).

Returns

type of the tile.

Implements [Tile](#).

7.9.3.3 Render()

void CrossTile::Render () const [override], [virtual]
 Render the tile on the game map. O(1).

Reimplemented from [Tile](#).

The documentation for this class was generated from the following files:

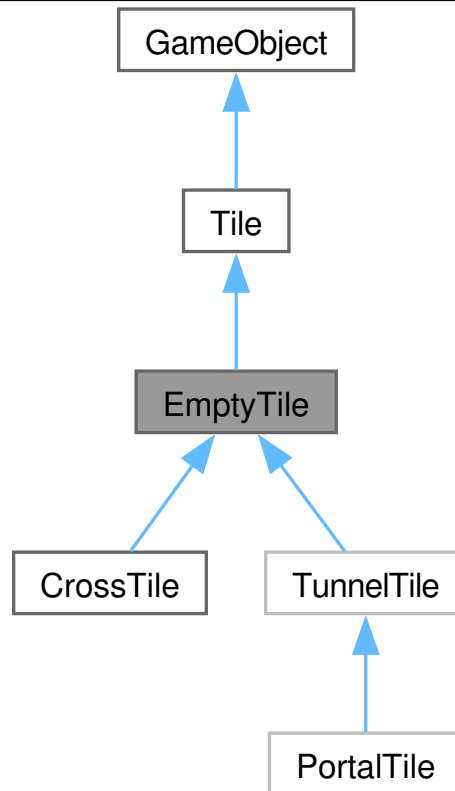
- src/game_objects/CrossTile.hpp
- src/game_objects/CrossTile.cpp

7.10 EmptyTile Class Reference

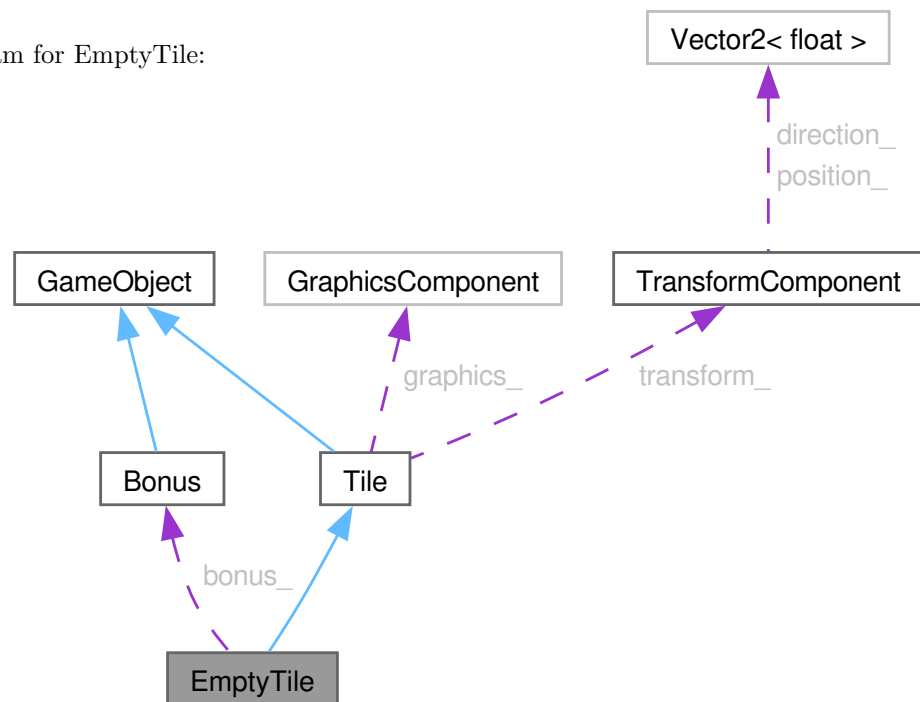
Class that represents Empty tile subclass of the [Tile](#) superclass.

#include <EmptyTile.hpp>

Inheritance diagram for EmptyTile:



Collaboration diagram for EmptyTile:



Public Member Functions

- [EmptyTile](#) ([Texture](#) *texture, const [Vector2< float >](#) &position)

- Construct a new Empty [Tile](#) object.
- void [Render](#) () const override
Render the tile on the game map. O(1).
- bool [AddBonus](#) ([Bonus](#) *bonus) override
Add bonus to tile.
- bool [CollectBonus](#) () override
Collect bonus from tile.
- [TileType](#) [GetType](#) () const override
Get the type of the tile. O(1).
- bool [CanTurnAside](#) (const [GameActor](#) &gameActor) const override
Check if game object standing on this tile can turn to the side, right or left or not. O(1).

Public Member Functions inherited from [Tile](#)

- [Tile](#) ([Texture](#) *texture, const [Vector2](#)< float > &position)
Construct a new [Tile](#) object. O(1).
- [Tile](#) ([Texture](#) *texture, [TransformComponent](#) *transform)
- [~Tile](#) () override
Destroys the [Tile](#) object. O(1).
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- virtual bool [isObstacle](#) () const
Check if this tile is obstacle or not. O(1).
- virtual bool [isTunnel](#) () const
Check if this tile is tunnel.
- virtual bool [isPortal](#) () const
Check if this tile is portal.
- virtual void [Teleport](#) ([GameActor](#) &gameActor, [TileNode](#) &portalNode)
Moves game actor transform and its current tile to another connected portalNode.
- [TransformComponent](#) * [GetTransform](#) ()
Get Transform component of the tile. O(1).
- const [TransformComponent](#) * [GetTransform](#) () const
Get Transform component of the tile. O(1).

Public Member Functions inherited from [GameObject](#)

- virtual [~GameObject](#) ()=default
Default destructor of the game object.

Protected Attributes

- [Bonus](#) * bonus_
[Bonus](#) on the tile.

Protected Attributes inherited from [Tile](#)

- [GraphicsComponent](#) * graphics_
Graphics component of the tile class.
- [TransformComponent](#) * transform_
Transform component of the tile class.

Additional Inherited Members

Static Public Member Functions inherited from [Tile](#)

- static [Vector2](#)< float > [GetOffsetFromTileCenter](#) (const [GameActor](#) &gameActor, const [Tile](#) *tile)
Get distance between game object and its tile center according to the current direction. O(1).

7.10.1 Detailed Description

Class that represents Empty tile subclass of the [Tile](#) superclass.

7.10.2 Constructor & Destructor Documentation

7.10.2.1 EmptyTile()

EmptyTile::EmptyTile (
[Texture](#) * texture,
 const [Vector2](#)< float > & position)

Construct a new Empty [Tile](#) object.

Parameters

texture	for graphics component to render the tile.
position	where tile will be rendered in the world.

7.10.3 Member Function Documentation

7.10.3.1 AddBonus()

bool EmptyTile::AddBonus (
[Bonus](#) * bonus) [override], [virtual]

Add bonus to tile.

Parameters

bonus	to be added to tile.
-------	----------------------

Returns

true if bonus was successfully added.

Reimplemented from [Tile](#).

7.10.3.2 CanTurnAside()

bool EmptyTile::CanTurnAside (
 const [GameActor](#) & gameActor) const [nodiscard], [override], [virtual]

Check if game object standing on this tile can turn to the side, right or left or not. O(1).

Parameters

gameActor	standing on this tile and trying to turn to another direction.
-----------	--

Returns

true if game object can turn aside.

Reimplemented from [Tile](#).

7.10.3.3 CollectBonus()

bool EmptyTile::CollectBonus () [override], [virtual]
Collect bonus from tile.

Returns

true if successfully collected bonus.

Reimplemented from [Tile](#).

7.10.3.4 GetType()

TileType EmptyTile::GetType () const [nodiscard], [override], [virtual]
Get the type of the tile. O(1).

Returns

type of the tile.

Implements [Tile](#).

Reimplemented in [PortalTile](#), and [TunnelTile](#).

7.10.3.5 Render()

void EmptyTile::Render () const [override], [virtual]
Render the tile on the game map. O(1).

Reimplemented from [Tile](#).

The documentation for this class was generated from the following files:

- src/game_objects/EmptyTile.hpp
- src/game_objects/EmptyTile.cpp

7.11 Game Class Reference

Class which represents the game, initializing, starting, updating, rendering and quitting the game.

#include <Game.hpp>

Public Member Functions

- Game ()=default
Constructs [Game](#) object.
- ~Game ()
Destroy the [Game](#) object. Use gameClose() method to clean the game.
- void GameStart ()
Starts the game - game loop and run until quit.
- void QuitGame ()
Quits game.

Public Attributes

- vector< [Controller](#) * > gameControllers_
Container for all GameControllers in the game.

7.11.1 Detailed Description

Class which represents the game, initializing, starting, updating, rendering and quitting the game.

The documentation for this class was generated from the following files:

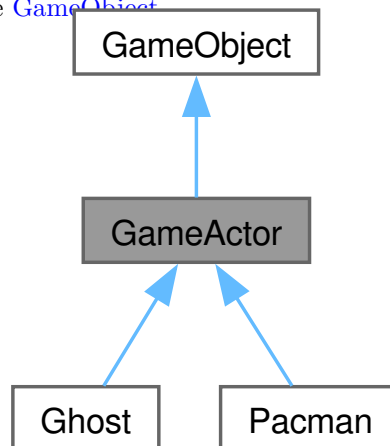
- src/game_engine/Game.hpp
- src/game_engine/Game.cpp

7.12 GameActor Class Reference

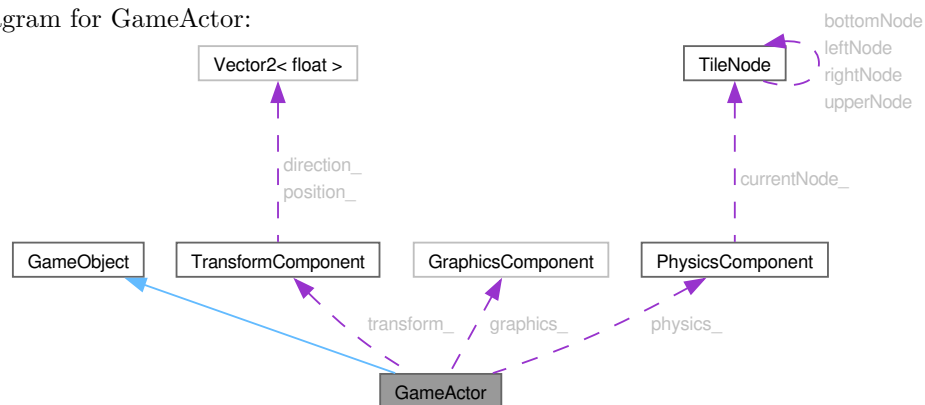
[GameActor](#) abstract class specification of the [GameObject](#)

```
#include <GameActor.hpp>
```

Inheritance diagram for GameActor:



Collaboration diagram for GameActor:



Public Member Functions

- [GameActor](#) ([TransformComponent](#) *transform, [GraphicsComponent](#) *graphics, [GameActor](#) &actor)
Construct a new [Game](#) Actor object and its components.
- [~GameActor](#) () override
Destroy the [Game](#) Actor object and its components.
- virtual void [Render](#) () const =0
Render object on the screen.
- virtual void [TurnUp](#) ()=0
Change game object's moving direction to up.
- virtual void [TurnDown](#) ()=0
Change game object's moving direction to down.
- virtual void [TurnRight](#) ()=0
Change game object's moving direction to right.
- virtual void [TurnLeft](#) ()=0
Change game object's moving direction to left.

- virtual void [StopMoving](#) ()=0
Change game object's moving to stop and don't move.
- virtual void [Move](#) ()=0
Moves game actor in its current direction.
- virtual void [TurnTo](#) (const [Vector2](#)< float > &direction)=0
Change game object's moving direction to specified direction.
- const [GraphicsComponent](#) * [GetGraphics](#) () const
Get the Graphics component.
- [PhysicsComponent](#) * [GetPhysics](#) ()
Get the Physics component. O(1).
- const [PhysicsComponent](#) * [GetPhysics](#) () const
- [TransformComponent](#) * [GetTransform](#) ()
Get the Transform component.
- const [TransformComponent](#) * [GetTransform](#) () const
- float [GetVelocity](#) () const
Get velocity of the game actor.
- void [SetVelocity](#) (float value)
Set velocity of the game actor.
- int [GetState](#) () const
Get state of the game actor.
- void [SetState](#) (int value)
Set state of the game actor.

Public Member Functions inherited from [GameObject](#)

- virtual ~[GameObject](#) ()=default
Default destructor of the game object.
- virtual void [Update](#) ()=0
Updates this game object in the game.
- virtual void [FixedUpdate](#) ()=0

Protected Attributes

- [TransformComponent](#) * transform__
Transform component of the game object.
- [GraphicsComponent](#) * graphics__
Graphics component of the game object.
- [PhysicsComponent](#) * physics__
Physics component of the game object.
- float velocity__
Current game actor's velocity in the world.
- int state__
Represents current state of the game actor.

7.12.1 Detailed Description

[GameActor](#) abstract class specification of the [GameObject](#).

Represents all gameObjects that are playable and controllable objects. Are used by the controller component to control such objects using unified interface.

7.12.2 Constructor & Destructor Documentation

7.12.2.1 GameActor()

GameActor::GameActor (
 [TransformComponent](#) * transform,
 [GraphicsComponent](#) * graphics,
 [GameActor](#) & actor)

Construct a new [Game](#) Actor object and its components.

Parameters

transform	- initial transform component for the game object.
graphics	- initial graphics component for the game object.

7.12.3 Member Function Documentation

7.12.3.1 Move()

virtual void GameActor::Move () [pure virtual]
 Moves game actor in its current direction.
 Implemented in [Ghost](#), and [Pacman](#).

7.12.3.2 Render()

virtual void GameActor::Render () const [pure virtual]
 Render object on the screen.
 Implemented in [Ghost](#), and [Pacman](#).

7.12.3.3 StopMoving()

virtual void GameActor::StopMoving () [pure virtual]
 Change game object's moving to stop and don't move.
 Implemented in [Ghost](#), and [Pacman](#).

7.12.3.4 TurnDown()

virtual void GameActor::TurnDown () [pure virtual]
 Change game object's moving direction to down.
 Implemented in [Ghost](#), and [Pacman](#).

7.12.3.5 TurnLeft()

virtual void GameActor::TurnLeft () [pure virtual]
 Change game object's moving direction to left.
 Implemented in [Ghost](#), and [Pacman](#).

7.12.3.6 TurnRight()

virtual void GameActor::TurnRight () [pure virtual]
 Change game object's moving direction to right.
 Implemented in [Ghost](#), and [Pacman](#).

7.12.3.7 TurnTo()

virtual void GameActor::TurnTo (
 const [Vector2](#)< float > & direction) [pure virtual]
 Change game object's moving direction to specified direction.

Parameters

direction	where to turn.
-----------	----------------

Implemented in [Ghost](#), and [Pacman](#).

7.12.3.8 TurnUp()

virtual void GameActor::TurnUp () [pure virtual]

Change game object's moving direction to up.

Implemented in [Ghost](#), and [Pacman](#).

The documentation for this class was generated from the following files:

- src/game_objects/GameActor.hpp
- src/game_objects/GameActor.cpp

7.13 GameAudio Class Reference

The documentation for this class was generated from the following file:

- src/game_engine/GameAudio.hpp

7.14 GameEvents Class Reference

Low level class that handles SDL events.

```
#include <GameEvents.hpp>
```

Public Member Functions

- GameEvents ()
Constructs the game events object.
- void ProcessInput ()
Handles input if game should quit or not.
- void QuitGame ()
Sets game to quit on the next frame, not immediately.
- bool [isQuit](#) () const
Checks whether game should stop and quit or not.
- bool [isQuitEvent](#) () const

7.14.1 Detailed Description

Low level class that handles SDL events.

7.14.2 Member Function Documentation

7.14.2.1 isQuit()

bool GameEvents::isQuit () const [inline]

Checks whether game should stop and quit or not.

Returns

true if is quit event.

false if is NOT quit event.

7.14.2.2 isQuitEvent()

bool GameEvents::isQuitEvent () const [inline]

Returns

- ture if current event is quit event.
- false if current event is NOT quit event.

The documentation for this class was generated from the following files:

- src/game_engine/GameEvents.hpp
- src/game_engine/GameEvents.cpp

7.15 GameFilesystem Class Reference

Game Filesystem Class which handles media loading and storage.

#include <GameFilesystem.hpp>

Public Member Functions

- GameFilesystem ()
Constructs **Game** Filesystem and instantly loads all resources.
- Texture * GetTexture (TextureType type)
Gets texture from filesystem of some texture type.

7.15.1 Detailed Description

Game Filesystem Class which handles media loading and storage.

Stores textures and other flyweight data used across the game. Handles loading and freeing data used in the game.

The documentation for this class was generated from the following files:

- src/game_engine/GameFilesystem.hpp
- src/game_engine/GameFilesystem.cpp

7.16 GameGraphics Class Reference

Class that initialize overall graphics for the game. Initialize window, renderer, and clearing and rendering on the screen.

#include <GameGraphics.hpp>

Public Member Functions

- GameGraphics ()
Construct a new **Game** Graphics object. Initiates graphics of the game - window, renderer etc.
- ~GameGraphics ()
Destroy game graphics object.
- void ClearRender ()
Clear the screen from all rendered objects.
- void RenderGame ()
Shows all current rendered game objects on the screen.
- SDL_Renderer * GetRenderer ()
Gets the renderer.

7.16.1 Detailed Description

Class that initialize overall graphics for the game. Initialize window, renderer, and clearing and rendering on the screen.

The documentation for this class was generated from the following files:

- src/game_engine/GameGraphics.hpp
- src/game_engine/GameGraphics.cpp

7.17 GameLocator Class Reference

Locator class which provides global access to global game services.

```
#include <GameLocator.hpp>
```

Static Public Member Functions

- static void InitLocator ()
Initializes locators members with nullptr.
- static void QuitLocator ()
Deletes allocated services objects avoiding possibly lost memory.
- static void ProvideTime (GameTime *time)
Initializes a Game Time service with provided object to locator.
- static void ProvideMap (GameMap *map)
Initializes a Game Map service with provided object to locator.
- static void ProvideFilesystem (GameFilesystem *filesystem)
Initializes a Game Filesystem service with provided object to locator.
- static void ProvideAudio (GameAudio *audio)
Initializes a Game Audio service with provided object to locator.
- static void ProvideManager (GameManager *manager)
Initializes a Game Manager service with provided object to locator.
- static void ProvideGraphics (GameGraphics *graphics)
Initializes a Game Graphics service with provided object to locator.
- static GameTime & GetTime ()
Get reference to the service.
- static GameMap & GetMap ()
Get reference to the service.
- static GameFilesystem & GetFilesystem ()
Get reference to the service.
- static GameAudio & GetAudio ()
Get reference to the service.
- static GameManager & GetManager ()
Get reference to the service.
- static GameGraphics & GetGraphics ()
Get reference to the service.
- static SDL_Renderer * GetRenderer ()
Get pointer to the renderer.

7.17.1 Detailed Description

Locator class which provides global access to global game services.

Provides global access to instances to game services - classes that are used widely by the game and have only one instance.

7.17.2 Member Function Documentation

7.17.2.1 GetFilesystem()

[GameFilesystem](#) & GameLocator::GetFilesystem () [inline], [static]

Get reference to the service.

Returns

[GameFilesystem](#)& reference to the [GameFilesystem](#) object.

7.17.2.2 GetMap()

[GameMap](#) & GameLocator::GetMap () [inline], [static]

Get reference to the service.

Returns

[GameMap](#)& reference to the [GameMap](#) object.

7.17.2.3 GetTime()

[GameTime](#) & GameLocator::GetTime () [inline], [static]

Get reference to the service.

Returns

[GameTime](#)& reference to the [GameTime](#) object.

7.17.2.4 ProvideAudio()

void GameLocator::ProvideAudio (
[GameAudio](#) * audio) [inline], [static]

Initializes a [Game](#) Audio service with provided object to locator.

Parameters

audio	GameAudio object to play audio in the game.
-------	---

7.17.2.5 ProvideFilesystem()

void GameLocator::ProvideFilesystem (
[GameFilesystem](#) * filesystem) [inline], [static]

Initializes a [Game](#) Filesystem service with provided object to locator.

Parameters

filesystem	filesystem of the game - stores all textures and tiles.
------------	---

7.17.2.6 ProvideGraphics()

void GameLocator::ProvideGraphics (
[GameGraphics](#) * graphics) [inline], [static]

Initializes a [Game](#) Graphics service with provided object to locator.

Parameters

graphics	graphics of the game - window and renderer.
----------	---

7.17.2.7 ProvideManager()

void GameLocator::ProvideManager (
 [GameManager](#) * manager) [inline], [static]

Initializes a [Game](#) Manager service with provided object to locator.

Parameters

manager	GameManager object to manage game state, score and modes.
---------	---

7.17.2.8 ProvideMap()

void GameLocator::ProvideMap (
 [GameMap](#) * map) [inline], [static]

Initializes a [Game](#) Map service with provided object to locator.

Parameters

map	GameMap object to represent game map from tiles in the game.
-----	--

7.17.2.9 ProvideTime()

void GameLocator::ProvideTime (
 [GameTime](#) * time) [inline], [static]

Initializes a [Game](#) Time service with provided object to locator.

Parameters

time	GameTime object for tracking time in the game.
------	--

The documentation for this class was generated from the following files:

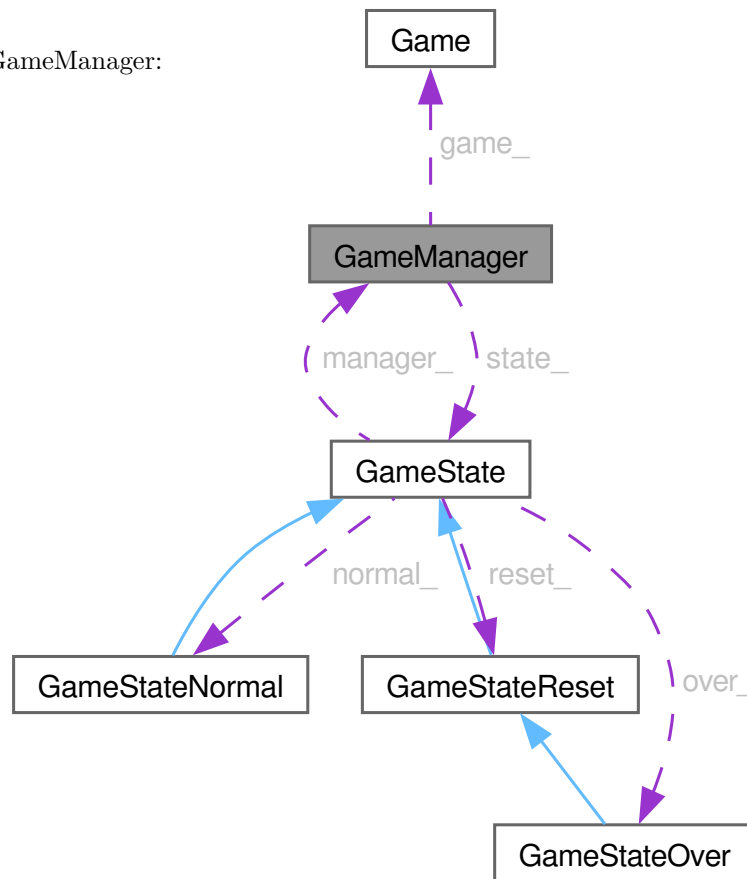
- src/game_engine/GameLocator.hpp
- src/game_engine/GameLocator.cpp

7.18 GameManager Class Reference

Class which manages game, what happens there and affect game objects.

#include <GameManager.hpp>

Collaboration diagram for GameManager:



Public Member Functions

- **GameManager** (**Game** &game)
Constructs game manager object.
- void Update ()
Checks game state each frame.
- void FixedUpdate ()
- void ReportPlayerCollision ()
Reports that player was collided with ghost.

Static Public Member Functions

- static void ReportBonusCollection ()
Reports bonus collection to current state.

Public Attributes

- **GameState** * state_
Represents current game state.
- **Game** & game_
Currently running game.

Static Public Attributes

- static constexpr float GAME_SPEED = 200.0f
Represents default game speed.

7.18.1 Detailed Description

Class which manages game, what happens there and affect game objects.

7.18.2 Constructor & Destructor Documentation

7.18.2.1 GameManager()

GameManager::GameManager (
[Game](#) & game) [explicit]

Constructs game manager object.

Parameters

game	which is managed by game manager.
------	-----------------------------------

The documentation for this class was generated from the following files:

- src/game_engine/GameManager.hpp
- src/game_engine/GameManager.cpp

7.19 GameMap Class Reference

Class that represents game maps in the game world.

#include <GameMap.hpp>

Public Member Functions

- [GameMap](#) (const string &path)
Constructs game map from the .map file. O(n).
- ~GameMap ()
Deletes all created tile from map. O(n).
- void RenderMap () const
Render on tiles on the map. O(n).
- void [LoadMap](#) (const string &path)
Loads game map. O(n), n - number of tiles. O(n).
- void ClearMap ()
Clears the map - both array and map collections of tiles. O(n).
- [MapInfo](#) GetMapInfo ()
Get map info member.

Static Public Member Functions

- static [Vector2](#)< int > [CentralizePosition](#) (const [Vector2](#)< int > &position)
Get tile center from current position of the game object. O(1).
- static [Vector2](#)< int > [GetPixelsPosition](#) (const [Vector2](#)< int > &position)
Convert position to pixels representation. O(1).

7.19.1 Detailed Description

Class that represents game maps in the game world.

7.19.2 Constructor & Destructor Documentation

7.19.2.1 GameMap()

GameMap::GameMap (
 const string & path) [explicit]
 Constructs game map from the .map file. $O(n)$.

Parameters

path	to the .map file to be loaded.
------	--------------------------------

7.19.3 Member Function Documentation

7.19.3.1 CentralizePosition()

[Vector2](#)< int > GameMap::CentralizePosition (
 const [Vector2](#)< int > & position) [static]
 Get tile center from current position of the game object. $O(1)$.

Parameters

position	of the game object in pixels.
----------	-------------------------------

Returns

center of the supposed tile on which game object currently is.

7.19.3.2 GetPixelsPosition()

[Vector2](#)< int > GameMap::GetPixelsPosition (
 const [Vector2](#)< int > & position) [static]
 Convert position to pixels representation. $O(1)$.

Parameters

position	of the tile in integer coordinates representation.
----------	--

Returns

position in pixels.

7.19.3.3 LoadMap()

void GameMap::LoadMap (
 const string & path)
 Loads game map. $O(n)$, n - number of tiles. $O(n)$.

Parameters

path	to the map file to read and load map from it.
------	---

The documentation for this class was generated from the following files:

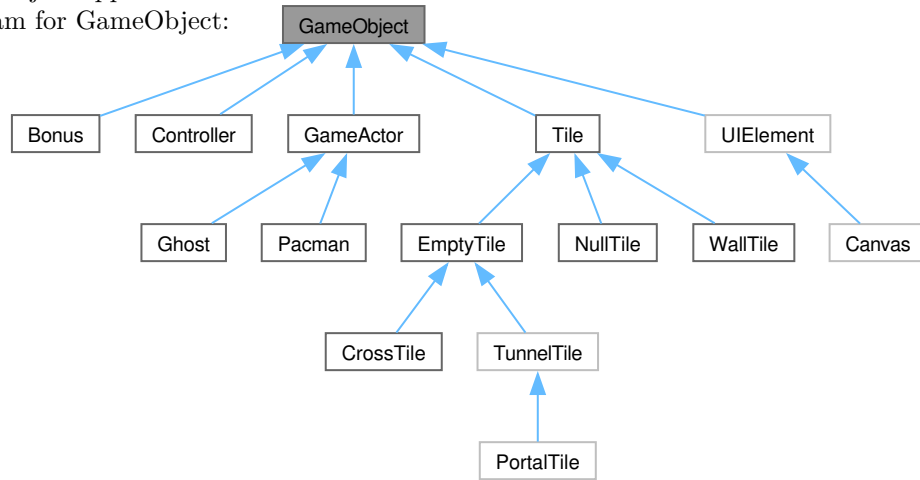
- src/game_engine/GameMap.hpp
- src/game_engine/GameMap.cpp

7.20 GameObject Class Reference

Interface for all game objects in the game.

#include <GameObject.hpp>

Inheritance diagram for GameObject:



Public Member Functions

- virtual `~GameObject()` = default
Default destructor of the game object.
- virtual void `Update()` = 0
Updates this game object in the game.
- virtual void `FixedUpdate()` = 0

7.20.1 Detailed Description

Interface for all game objects in the game.

7.20.2 Member Function Documentation

7.20.2.1 Update()

virtual void `GameObject::Update()` [pure virtual]

Updates this game object in the game.

Implemented in [Bonus](#), [Canvas](#), [Controller](#), [Ghost](#), [Pacman](#), [Tile](#), and [UIElement](#).

The documentation for this class was generated from the following file:

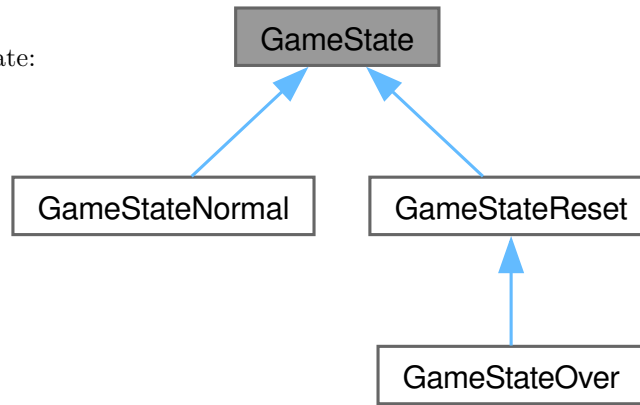
- `src/game_objects/GameObject.hpp`

7.21 GameState Class Reference

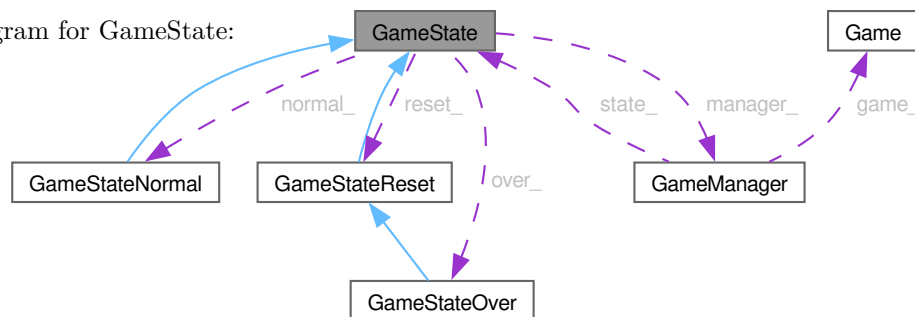
Class that represents game state.

#include <GameState.hpp>

Inheritance diagram for GameState:



Collaboration diagram for GameState:



Public Member Functions

- virtual `~GameState ()`=default
Destructs game state object.
- virtual void `Update ()`=0
Update state each frame.
- virtual void `FixedUpdate ()`=0
- virtual void `PlayerCollision ()`=0
Reacts on player collision event.
- virtual void `InitGameState ()`=0
Init's game state with its initial settings or values.

Static Public Member Functions

- static void `ProvideGameManager (GameManager &manager)`
Provide pointer to game manager.

Static Public Attributes

- static `GameStateNormal normal__`
Normal state of the game.
- static `GameStateReset reset__`
Reset state of the game.
- static `GameStateOver over__`
`Game` over state of the game.

Static Protected Attributes

- static [GameManager](#) * manager__
[Game](#) Manager object of the game.

7.21.1 Detailed Description

Class that represents game state.

7.21.2 Member Function Documentation

7.21.2.1 InitGameState()

virtual void GameState::InitGameState () [pure virtual]

Init's game state with its initial settings or values.

Implemented in [GameStateNormal](#), [GameStateOver](#), and [GameStateReset](#).

7.21.2.2 PlayerCollision()

virtual void GameState::PlayerCollision () [pure virtual]

Reacts on player collision event.

Implemented in [GameStateNormal](#), and [GameStateReset](#).

7.21.2.3 Update()

virtual void GameState::Update () [pure virtual]

Update state each frame.

Implemented in [GameStateNormal](#), [GameStateOver](#), and [GameStateReset](#).

The documentation for this class was generated from the following files:

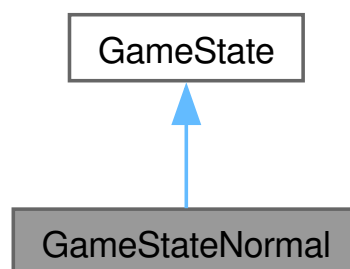
- src/game__states/GameState.hpp
- src/game__states/GameState.cpp
- src/game__states/GameStateNormal.cpp
- src/game__states/GameStateOver.cpp
- src/game__states/GameStateReset.cpp

7.22 GameStateNormal Class Reference

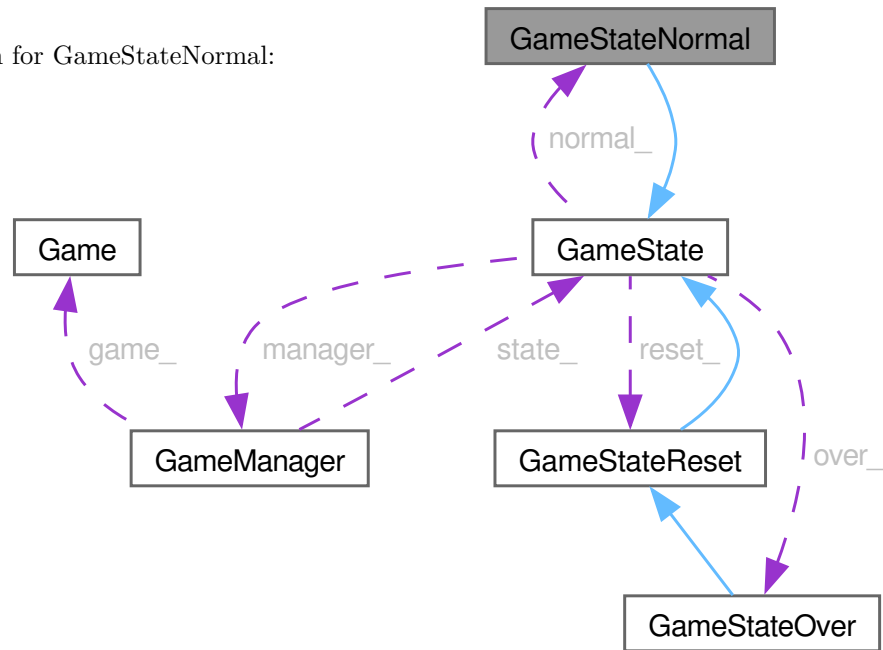
Represents normal state of the game.

```
#include <GameStateNormal.hpp>
```

Inheritance diagram for GameStateNormal:



Collaboration diagram for GameStateNormal:



Public Member Functions

- `GameStateNormal ()`=default
Constructs normal game state object.
- `void Update ()` override
Update state each frame.
- `void FixedUpdate ()` override
- `void PlayerCollision ()` override
Reacts on player collision event.
- `void InitGameState ()` override
Init's game state with its initial settings or values.

Public Member Functions inherited from [GameState](#)

- `virtual ~GameState ()`=default
Destructs game state object.

Static Public Member Functions

- `static void SetBonusAmount (int amount)`
Set total amount of bonuses in the game.
- `static void DecrementBonus ()`
Decrement amount of bonuses by one.
- `static void AddScore (int score)`
- `static int GetGameScore ()`
- `static int GetPlayerLives ()`

Static Public Member Functions inherited from [GameState](#)

- `static void ProvideGameManager (GameManager &manager)`
Provide pointer to game manager.

Additional Inherited Members

Static Public Attributes inherited from [GameState](#)

- static [GameStateNormal](#) normal__
Normal state of the game.
- static [GameStateReset](#) reset__
Reset state of the game.
- static [GameStateOver](#) over__
[Game](#) over state of the game.

Static Protected Attributes inherited from [GameState](#)

- static [GameManager](#) * manager__
[Game](#) Manager object of the game.

7.22.1 Detailed Description

Represents normal state of the game.

7.22.2 Member Function Documentation

7.22.2.1 FixedUpdate()

void GameStateNormal::FixedUpdate () [override], [virtual]
Implements [GameState](#).

7.22.2.2 InitGameState()

void GameStateNormal::InitGameState () [override], [virtual]
Inits game state with its initial settings or values.
Implements [GameState](#).

7.22.2.3 PlayerCollision()

void GameStateNormal::PlayerCollision () [override], [virtual]
Reacts on player collision event.
Implements [GameState](#).

7.22.2.4 SetBonusAmount()

void GameStateNormal::SetBonusAmount (
int amount) [static]
Set total amount of bonuses in the game.

Parameters

amount	of bonuses to be set.
--------	-----------------------

7.22.2.5 Update()

void GameStateNormal::Update () [override], [virtual]
Update state each frame.
Implements [GameState](#).

The documentation for this class was generated from the following files:

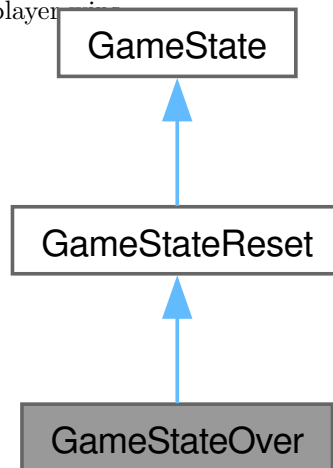
- src/game_states/GameStateNormal.hpp
- src/game_states/GameStateNormal.cpp

7.23 GameStateOver Class Reference

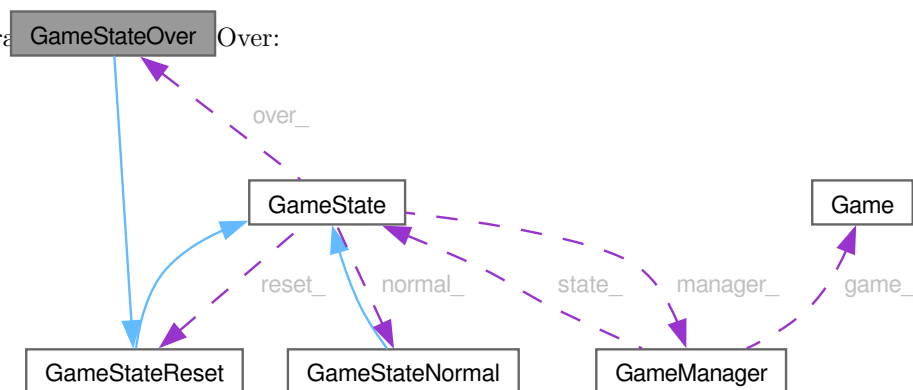
Represents game over state of the game, when player dies.

```
#include <GameStateOver.hpp>
```

Inheritance diagram for GameStateOver:



Collaboration diagram for GameStateOver:



Public Member Functions

- void [Update](#) () override
Update state each frame.
- void [FixedUpdate](#) () override
- void [InitGameState](#) () override
Inits game state with its initial settings or values.

Public Member Functions inherited from [GameStateReset](#)

- [GameStateReset](#) ()
Constructs reset game state.
- void [Update](#) () override
Update state each frame.
- void [FixedUpdate](#) () override
- void [PlayerCollision](#) () override
Reacts on player collision event.

- void [InitGameState](#) () override
Init's game state with its initial settings or values.

Public Member Functions inherited from [GameState](#)

- virtual [~GameState](#) ()=default
Destructs game state object.

Additional Inherited Members

Static Public Member Functions inherited from [GameState](#)

- static void [ProvideGameManager](#) ([GameManager](#) &manager)
Provide pointer to game manager.

Static Public Attributes inherited from [GameState](#)

- static [GameStateNormal](#) normal__
Normal state of the game.
- static [GameStateReset](#) reset__
Reset state of the game.
- static [GameStateOver](#) over__
[Game](#) over state of the game.

Protected Attributes inherited from [GameStateReset](#)

- float delay__
Delay before reset to give player some time to consider about what happened, about his life and other things.

Static Protected Attributes inherited from [GameState](#)

- static [GameManager](#) * manager__
[Game](#) Manager object of the game.

7.23.1 Detailed Description

Represents game over state of the game, when player wins.

7.23.2 Member Function Documentation

7.23.2.1 FixedUpdate()

void [GameStateOver::FixedUpdate](#) () [override], [virtual]
Implements [GameState](#).

7.23.2.2 InitGameState()

void [GameStateOver::InitGameState](#) () [override], [virtual]
Init's game state with its initial settings or values.
Implements [GameState](#).

7.23.2.3 Update()

void [GameStateOver::Update](#) () [override], [virtual]
Update state each frame.
Implements [GameState](#).

The documentation for this class was generated from the following files:

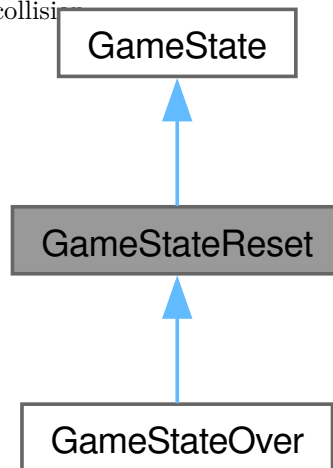
- [src/game_states/GameStateOver.hpp](#)
- [src/game_states/GameStateOver.cpp](#)

7.24 GameStateReset Class Reference

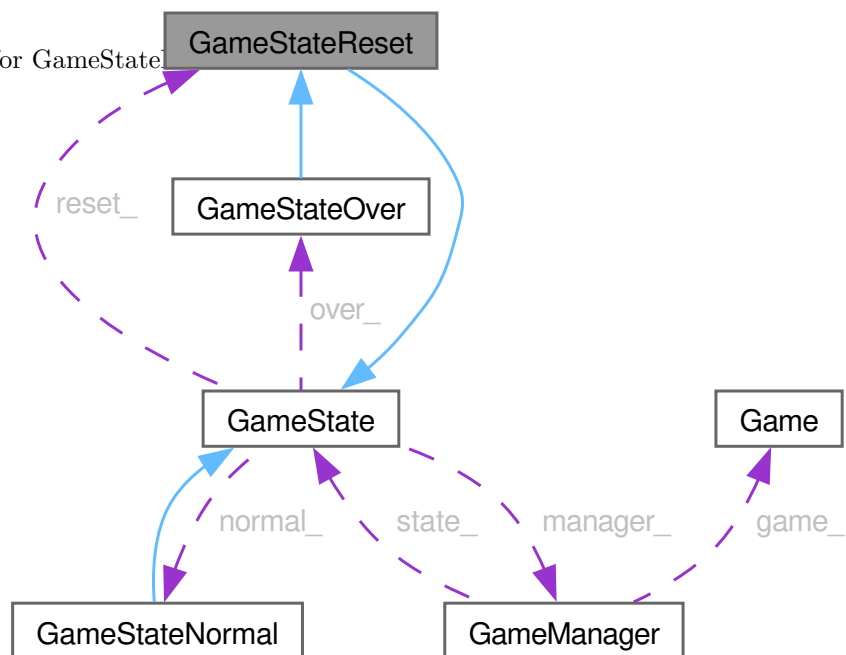
Class that represents game state on reset after collision.

```
#include <GameStateReset.hpp>
```

Inheritance diagram for GameStateReset:



Collaboration diagram for GameStateReset



Public Member Functions

- `GameStateReset ()`
Constructs reset game state.
- `void Update ()` override
Update state each frame.
- `void FixedUpdate ()` override
- `void PlayerCollision ()` override
Reacts on player collision event.

- void [InitGameState](#) () override
 Inits game state with its initial settings or values.

Public Member Functions inherited from [GameState](#)

- virtual [~GameState](#) ()=default
 Destructs game state object.

Protected Attributes

- float delay_
 Delay before reset to give player some time to consider about what happened, about his life and other things.

Additional Inherited Members

Static Public Member Functions inherited from [GameState](#)

- static void ProvideGameManager ([GameManager](#) &manager)
 Provide pointer to game manager.

Static Public Attributes inherited from [GameState](#)

- static [GameStateNormal](#) normal_
 Normal state of the game.
- static [GameStateReset](#) reset_
 Reset state of the game.
- static [GameStateOver](#) over_
 [Game](#) over state of the game.

Static Protected Attributes inherited from [GameState](#)

- static [GameManager](#) * manager_
 [Game](#) Manager object of the game.

7.24.1 Detailed Description

Class that represents game state on reset after collision.

7.24.2 Member Function Documentation

7.24.2.1 FixedUpdate()

void GameStateReset::FixedUpdate () [override], [virtual]
Implements [GameState](#).

7.24.2.2 InitGameState()

void GameStateReset::InitGameState () [override], [virtual]
Inits game state with its initial settings or values.
Implements [GameState](#).

7.24.2.3 PlayerCollision()

void GameStateReset::PlayerCollision () [override], [virtual]
Reacts on player collision event.
Implements [GameState](#).

7.24.2.4 Update()

void GameStateReset::Update () [override], [virtual]

Update state each frame.

Implements [GameState](#).

The documentation for this class was generated from the following files:

- src/game_states/GameStateReset.hpp
- src/game_states/GameStateReset.cpp

7.25 GameTime Class Reference

[Game](#) Time class which handles time across all the game.

#include <GameTime.hpp>

Public Member Functions

- [GameTime](#) ()
Constructs game time object.
- void InitGameTicks ()
Initialized game timer with 0 ticks, and last ticks with time since game start.
- void UpdateCurrentTicks ()
Update current ticks with actual SDL time in ticks and calculate delta ticks between current and last ticks.
- void UpdateLastTicks ()
Update last ticks with current ticks Sets to the value from the start of the current game loop frame.
- uint GetDeltaTicks () const
Get the delta ticks in milliseconds.
- float GetDeltaTime () const
Get the delta ticks in seconds.

Static Public Attributes

- static constexpr uint MS_PER_UPDATE = 16
- static constexpr float FMS_PER_UPDATE = 0.016f

7.25.1 Detailed Description

[Game](#) Time class which handles time across all the game.

The documentation for this class was generated from the following files:

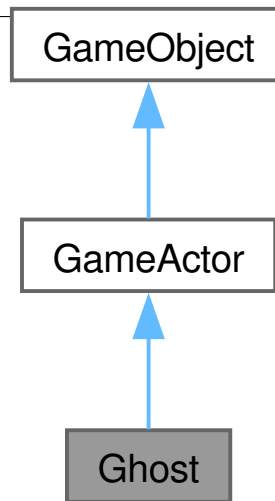
- src/game_engine/GameTime.hpp
- src/game_engine/GameTime.cpp

7.26 Ghost Class Reference

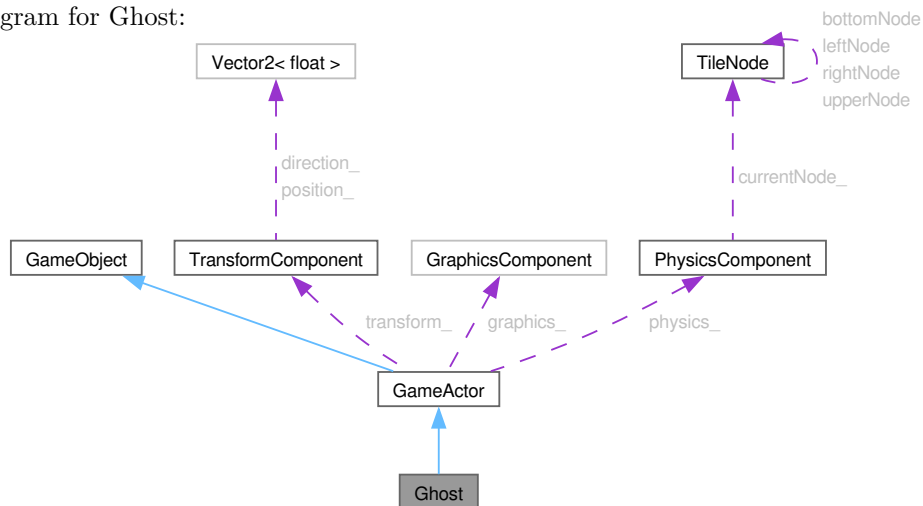
[Pacman](#) class - concrete specification of the [GameActor](#), which defines its moving algorithms and what it does.

#include <Ghost.hpp>

Inheritance diagram for Ghost:



Collaboration diagram for Ghost:



Public Member Functions

- [Ghost](#) ([TransformComponent](#) *transform, [GraphicsComponent](#) *graphics)
[Ghost](#) constructor - takes components to construct ghost object.
- void [TurnUp](#) () override
 Change game object's moving direction to up.
- void [TurnDown](#) () override
 Change game object's moving direction to down.
- void [TurnRight](#) () override
 Change game object's moving direction to right.
- void [TurnLeft](#) () override
 Change game object's moving direction to left.
- void [StopMoving](#) () override
 Change game object's moving to stop and don't move.
- void [Move](#) () override
 Moves game actor in its current direction.

- void [TurnTo](#) (const [Vector2](#)< float > &direction) override
Change game object's moving direction to specified direction.
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- void [Render](#) () const override
Render object on the screen.

Public Member Functions inherited from [GameActor](#)

- [GameActor](#) ([TransformComponent](#) *transform, [GraphicsComponent](#) *graphics, [GameActor](#) &actor)
Construct a new [Game](#) Actor object and its components.
- ~[GameActor](#) () override
Destroy the [Game](#) Actor object and its components.
- const [GraphicsComponent](#) * [GetGraphics](#) () const
Get the Graphics component.
- [PhysicsComponent](#) * [GetPhysics](#) ()
Get the Physics component. O(1).
- const [PhysicsComponent](#) * [GetPhysics](#) () const
- [TransformComponent](#) * [GetTransform](#) ()
Get the Transform component.
- const [TransformComponent](#) * [GetTransform](#) () const
- float [GetVelocity](#) () const
Get velocity of the game actor.
- void [SetVelocity](#) (float value)
Set velocity of the game actor.
- int [GetState](#) () const
Get state of the game actor.
- void [SetState](#) (int value)
Set state of the game actor.

Public Member Functions inherited from [GameObject](#)

- virtual ~[GameObject](#) ()=default
Default destructor of the game object.

Additional Inherited Members

Protected Attributes inherited from [GameActor](#)

- [TransformComponent](#) * transform_
Transform component of the game object.
- [GraphicsComponent](#) * graphics_
Graphics component of the game object.
- [PhysicsComponent](#) * physics_
Physics component of the game object.
- float velocity_
Current game actor's velocity in the world.
- int state_
Represents current state of the game actor.

7.26.1 Detailed Description

[Pacman](#) class - concrete specification of the [GameActor](#), which defines its moving algorithms and what it does.

7.26.2 Constructor & Destructor Documentation

7.26.2.1 Ghost()

```
Ghost::Ghost (
    TransformComponent * transform,
    GraphicsComponent * graphics)
```

[Ghost](#) constructor - takes components to construct ghost object.

Parameters

transform	transform component of the object.
graphics	graphics component of the object.

7.26.3 Member Function Documentation

7.26.3.1 FixedUpdate()

```
void Ghost::FixedUpdate () [override], [virtual]
Implements GameObject.
```

7.26.3.2 Move()

```
void Ghost::Move () [override], [virtual]
Moves game actor in its current direction.
Implements GameActor.
```

7.26.3.3 Render()

```
void Ghost::Render () const [override], [virtual]
Render object on the screen.
Implements GameActor.
```

7.26.3.4 StopMoving()

```
void Ghost::StopMoving () [override], [virtual]
Change game object's moving to stop and don't move.
Implements GameActor.
```

7.26.3.5 TurnDown()

```
void Ghost::TurnDown () [override], [virtual]
Change game object's moving direction to down.
Implements GameActor.
```

7.26.3.6 TurnLeft()

```
void Ghost::TurnLeft () [override], [virtual]
Change game object's moving direction to left.
Implements GameActor.
```

7.26.3.7 TurnRight()

```
void Ghost::TurnRight () [override], [virtual]
Change game object's moving direction to right.
```

Implements [GameActor](#).

7.26.3.8 TurnTo()

```
void Ghost::TurnTo (
    const Vector2< float > & direction)  [override], [virtual]
```

Change game object's moving direction to specified direction.

Parameters

direction	where to turn.
-----------	----------------

Implements [GameActor](#).

7.26.3.9 TurnUp()

```
void Ghost::TurnUp ()  [override], [virtual]
```

Change game object's moving direction to up.

Implements [GameActor](#).

7.26.3.10 Update()

```
void Ghost::Update ()  [override], [virtual]
```

Updates this game object in the game.

Implements [GameObject](#).

The documentation for this class was generated from the following files:

- src/game_objects/Ghost.hpp
- src/game_objects/Ghost.cpp

7.27 GraphicsComponent Class Reference

Public Member Functions

- [GraphicsComponent](#) ([Texture](#) *texture)
Construct a new Graphics Component object. O(1).
- virtual [~GraphicsComponent](#) ()=default
Destruct graphics component. O(1).
- void ChangeGraphicsTexture (const string &pathOrMessage)
- virtual void [Render](#) (const [Vector2](#)< float > &position) const
Render image each game frame onto the screen. O(1).
- virtual void [Render](#) (const [Vector2](#)< float > &position, double angle) const
- const [TextureSize](#)< float > & [GetTextureSize](#) () const
Get the size of the texture graphics component is holding. O(1).

7.27.1 Constructor & Destructor Documentation

7.27.1.1 GraphicsComponent()

```
GraphicsComponent::GraphicsComponent (
    Texture * texture)  [explicit]
```

Construct a new Graphics Component object. O(1).

Parameters

texture	- texture provided by the filesystem.
---------	---------------------------------------

7.27.2 Member Function Documentation

7.27.2.1 GetTextureSize()

const [TextureSize](#)< float > & GraphicsComponent::GetTextureSize () const [inline]

Get the size of the texture graphics component is holding. O(1).

Returns

reference to const texture size.

7.27.2.2 Render()

void GraphicsComponent::Render (const [Vector2](#)< float > & position) const [virtual]

Render image each game frame onto the screen. O(1).

Parameters

position	- position where to render on the screen in pixels.
----------	---

The documentation for this class was generated from the following files:

- src/components/GraphicsComponent.hpp
- src/components/GraphicsComponent.cpp

7.28 MapHeader Struct Reference

Structure that represents header info in the game map file.

#include <MapReader.hpp>

Public Attributes

- pair< size_t, size_t > mapSize__ = {0, 0}
Represents map size, first is rows, second is columns.
- pair< [Vector2](#)< int >, [Vector2](#)< int > > portalsPair__
Represents two connected portals in the game map.
- pair< [Vector2](#)< int >, [Vector2](#)< int > > spawnPoints__
- bool noPortals__
Stores true if there is no portals in the map.

7.28.1 Detailed Description

Structure that represents header info in the game map file.

7.28.2 Member Data Documentation

7.28.2.1 spawnPoints__

pair<[Vector2](#)<int>, [Vector2](#)<int> > MapHeader::spawnPoints__

Represents spawn points for pacman and ghost, first is pacman or player, second is ghosts.

The documentation for this struct was generated from the following file:

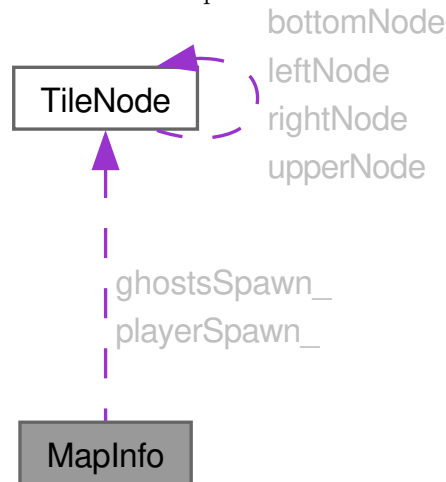
- src/structures/MapReader.hpp

7.29 MapInfo Struct Reference

Structure that represents general information about the map.

#include <TileNodeBuilder.hpp>

Collaboration diagram for MapInfo:



Public Attributes

- `int bonuses__ = 0`
Represents total number of bonuses in the map.
- `TileNode * playerSpawn__ = nullptr`
Represents player or pacman spawn point.
- `TileNode * ghostsSpawn__ = nullptr`
Represents ghost spawn point.

7.29.1 Detailed Description

Structure that represents general information about the map.

The documentation for this struct was generated from the following file:

- `src/structures/TileNodeBuilder.hpp`

7.30 MapParser Class Reference

Class that parses tile data objects to tile info objects, used for building tiles.

#include <MapParser.hpp>

Public Member Functions

- `MapParser ()=default`
Constructs map parser object.
- `MapParser (const vector< vector< TileData > > &tilesData, const MapHeader &header)`
Constructs map parser object and parses tile data to tile info.
- `bool ParseMap (const vector< vector< TileData > > &tilesData, const MapHeader &header)`
Parses tile data collection to tile info collection.
- `const vector< vector< TileInfo > > & GetTilesInfo () const`
Gets parsed tiles info collection.

7.30.1 Detailed Description

Class that parses tile data objects to tile info objects, used for building tiles.

7.30.2 Constructor & Destructor Documentation

7.30.2.1 MapParser()

```
MapParser::MapParser (
    const vector< vector< TileData > > & tilesData,
    const MapHeader & header) [explicit]
```

Constructs map parser object and parses tile data to tile info.

Parameters

tilesData	represents collection of tile data objects used for parsing.
-----------	--

7.30.3 Member Function Documentation

7.30.3.1 ParseMap()

```
bool MapParser::ParseMap (
    const vector< vector< TileData > > & tilesData,
    const MapHeader & header)
```

Parses tile data collection to tile info collection.

Parameters

tilesData	represents collection of tile data objects used for parsing.
-----------	--

Returns

true if successfully parsed.
false if failed to parse.

The documentation for this class was generated from the following files:

- src/structures/MapParser.hpp
- src/structures/MapParser.cpp

7.31 MapReader Class Reference

Class that reads map file and stores map chars in the 2D vector for further parsing or processing, checking file stream and map size from map header information.

```
#include <MapReader.hpp>
```

Public Member Functions

- [MapReader](#) ()=default
Constructs map reader object.
- [MapReader](#) (const string &path)
Constructor, that reads and loads map.
- [~MapReader](#) ()
Destructor of the map reader object.
- bool [Read](#) (const string &path)
Opens, reads header and reads map from the file map.
- const vector< vector< [TileData](#) > > & GetTilesData () const

- Get 2D collection of [TileData](#) objects.
- `const MapHeader GetMapHeader () const`
Get map header of the read map..

7.31.1 Detailed Description

Class that reads map file and stores map chars in the 2D vector for further parsing or processing, checking file stream and map size from map header information.

7.31.2 Constructor & Destructor Documentation

7.31.2.1 MapReader()

`MapReader::MapReader (`
 `const string & path) [explicit]`
 Constructor, that reads and loads map.

Parameters

path	to the map file.
------	------------------

7.31.3 Member Function Documentation

7.31.3.1 Read()

`bool MapReader::Read (`
 `const string & path)`
 Opens, reads header and reads map from the file map.

Parameters

path	to map file.
------	--------------

Returns

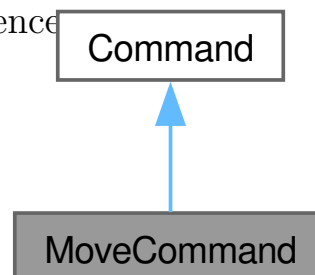
 true if successfully read file.
 false if failed to read file.

The documentation for this class was generated from the following files:

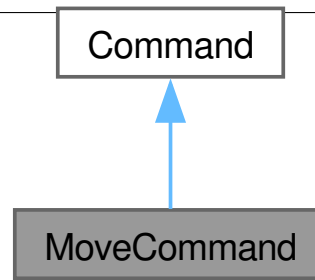
- `src/structures/MapReader.hpp`
- `src/structures/MapReader.cpp`

7.32 MoveCommand Class Reference

Inheritance diagram for MoveCommand:



Collaboration diagram for MoveCommand:



Public Member Functions

- void [Execute](#) ([GameActor](#) *actor) override
Executes command with give actor.

7.32.1 Member Function Documentation

7.32.1.1 Execute()

void MoveCommand::Execute (
[GameActor](#) * actor) [override], [virtual]
 Executes command with give actor.

Parameters

actor	which should execute the command.
-------	-----------------------------------

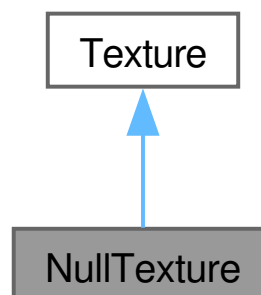
Implements [Command](#).

The documentation for this class was generated from the following files:

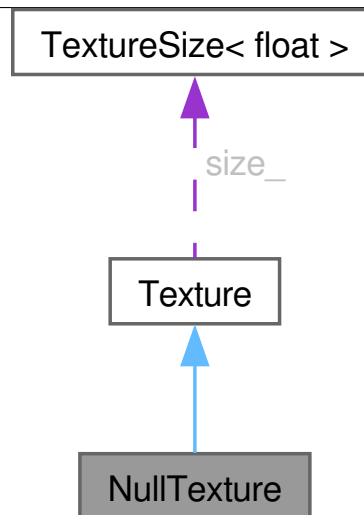
- src/components/Command.hpp
- src/components/Command.cpp

7.33 NullTexture Class Reference

Inheritance diagram for NullTexture:



Collaboration diagram for NullTexture:



Public Member Functions

- `bool LoadTexture (const std::string &path) override`
- `bool LoadTexture (const std::string &path, const TextureSize< float > &size) override`
- `void RenderTexture (const Vector2< float > &position) const override`
Render this texture on the screen by renderer. O(1).
- `bool SetNativeTextureSize () override`
Queries texture for its native image dimensions and sets the size of the texture in pixels. O(1).

Public Member Functions inherited from [Texture](#)

- `Texture ()`
Construct and inits sdl texture with nullptr and width and height with 0. O(1).
- `virtual ~Texture () noexcept`
Deletes texture. Use `DestroyTexture` to safely destroy sdl texture. O(1).
- `Texture (const Texture &src)=delete`
Deleted copy constructor.
- `Texture (Texture &&src) noexcept`
Move constructor.
- `virtual bool LoadTexture (const string &path)`
Loads texture from .png image and applies it for renderer. O(1).
- `virtual bool LoadTexture (const string &path, const TextureSize< float > &size)`
Loads texture from .png image and applies it for renderer. O(1).
- `virtual void RenderTexture (const Vector2< float > &position, double angle) const`
- `void SetTextureSize (const TextureSize< float > &size)`
Set texture dimensions using user defined size in pixels. O(1).
- `const TextureSize< float > & GetSize () const`
Get texture size of this texture. O(1).

Additional Inherited Members

Protected Member Functions inherited from [Texture](#)

- `SDL_FRect CreateRenderRectCentered (const Vector2< float > &position) const`

Create SDL float rectangle object from center position. Uses texture size to find left upper corner where SDL rect should be defined for rendering. O(1).

- void DestroyTexture ()

Destroys current texture and prepares for loading another texture. O(1).

Protected Attributes inherited from [Texture](#)

- SDL_Texture * texture_
Object to store and handle textures in SDL.
- [TextureSize](#)< float > size_
Dimensions of the loaded texture (of the image) in pixels.

7.33.1 Member Function Documentation

7.33.1.1 RenderTexture()

void NullTexture::RenderTexture (const [Vector2](#)< float > & position) const [override], [virtual]

Render this texture on the screen by renderer. O(1).

Parameters

position	in pixels where to show image on the screen.
----------	--

Reimplemented from [Texture](#).

7.33.1.2 SetNativeTextureSize()

bool NullTexture::SetNativeTextureSize () [override], [virtual]

Queries texture for its native image dimensions and sets the size of the texture in pixels. O(1).

Returns

true if successfully set up native size.

false if failed to query image size and set it up.

Reimplemented from [Texture](#).

The documentation for this class was generated from the following files:

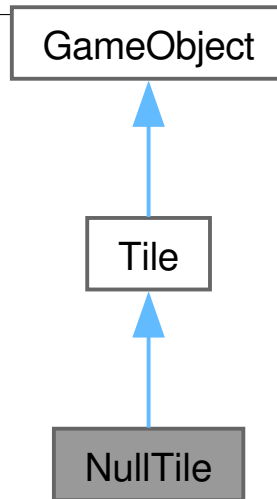
- src/structures/NullTexture.hpp
- src/structures/NullTexture.cpp

7.34 NullTile Class Reference

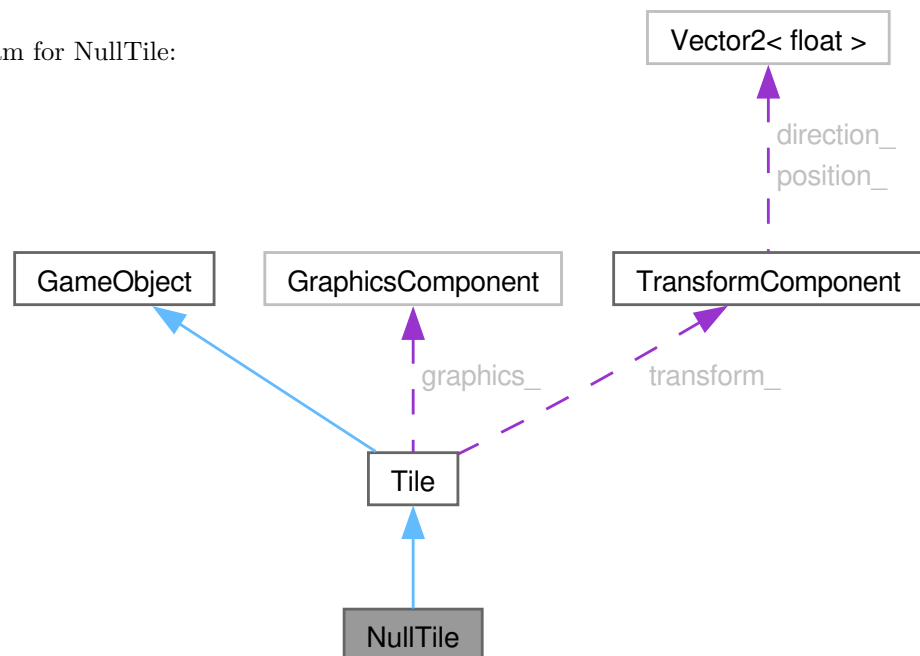
Class that represents Null tile subclass of the [Tile](#) superclass.

#include <NullTile.hpp>

Inheritance diagram for NullTile:



Collaboration diagram for NullTile:



Public Member Functions

- **NullTile** (const **Vector2**< float > &position)
Construct a new Null **Tile** object.
- void **Render** () const override
Render the tile on the game map. O(1).
- TileType **GetType** () const override
Get the type of the tile. O(1).
- bool **isObstacle** () const override
Check if this tile is obstacle or not. O(1).

Public Member Functions inherited from [Tile](#)

- [Tile](#) ([Texture](#) *texture, const [Vector2](#)< float > &position)
Construct a new [Tile](#) object. O(1).
- [Tile](#) ([Texture](#) *texture, [TransformComponent](#) *transform)
- [~Tile](#) () override
Destroys the [Tile](#) object. O(1).
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- virtual bool [isTunnel](#) () const
Check if this tile is tunnel.
- virtual bool [isPortal](#) () const
Check if this tile is portal.
- virtual void [Teleport](#) ([GameActor](#) &gameActor, [TileNode](#) &portalNode)
Moves game actor transform and its current tile to another connected portalNode.
- virtual bool [CanTurnAside](#) (const [GameActor](#) &gameActor) const
Check if game object standing on this tile can turn to the side, right or left or not. O(1).
- virtual bool [AddBonus](#) ([Bonus](#) *bonus)
Add bonus to tile.
- virtual bool [CollectBonus](#) ()
Collect bonus from tile.
- [TransformComponent](#) * [GetTransform](#) ()
Get Transform component of the tile. O(1).
- const [TransformComponent](#) * [GetTransform](#) () const
Get Transform component of the tile. O(1).

Public Member Functions inherited from [GameObject](#)

- virtual [~GameObject](#) ()=default
Default destructor of the game object.

Additional Inherited Members

Static Public Member Functions inherited from [Tile](#)

- static [Vector2](#)< float > [GetOffsetFromTileCenter](#) (const [GameActor](#) &gameActor, const [Tile](#) *tile)
Get distance between game object and its tile center according to the current direction. O(1).

Protected Attributes inherited from [Tile](#)

- [GraphicsComponent](#) * graphics_
Graphics component of the tile class.
- [TransformComponent](#) * transform_
Transform component of the tile class.

7.34.1 Detailed Description

Class that represents Null tile subclass of the [Tile](#) superclass.

7.34.2 Constructor & Destructor Documentation

7.34.2.1 NullTile()

NullTile::NullTile (
 const [Vector2](#)< float > & position)

Construct a new Null [Tile](#) object.

Parameters

position	where tile will be rendered in the world.
----------	---

7.34.3 Member Function Documentation

7.34.3.1 GetType()

TileType NullTile::GetType () const [nodiscard], [override], [virtual]

Get the type of the tile. O(1).

Returns

type of the tile.

Implements [Tile](#).

7.34.3.2 isObstacle()

bool NullTile::isObstacle () const [nodiscard], [override], [virtual]

Check if this tile is obstacle or not. O(1).

Returns

true if tile is obstacle.

Reimplemented from [Tile](#).

7.34.3.3 Render()

void NullTile::Render () const [override], [virtual]

Render the tile on the game map. O(1).

Reimplemented from [Tile](#).

The documentation for this class was generated from the following files:

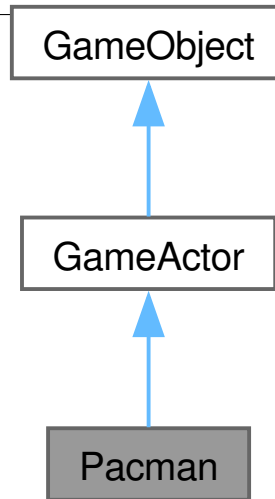
- src/game_objects/NullTile.hpp
- src/game_objects/NullTile.cpp

7.35 Pacman Class Reference

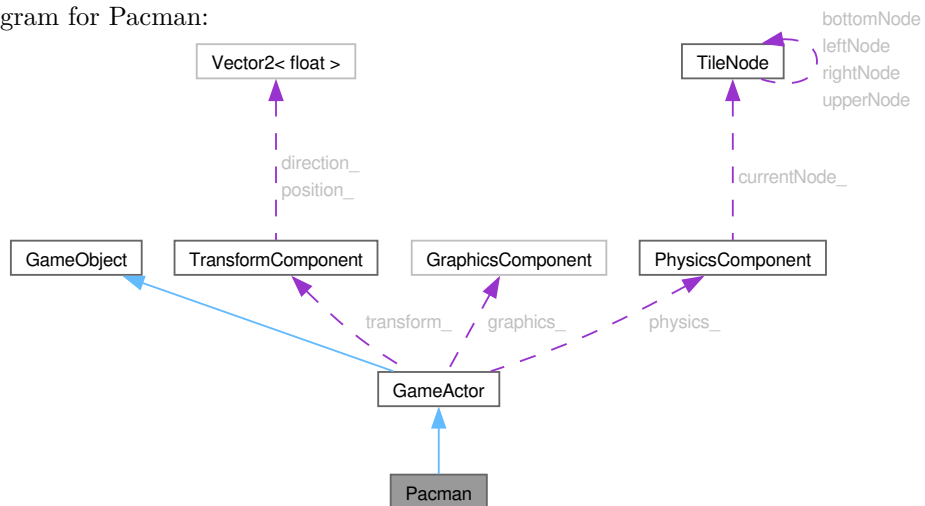
[Pacman](#) class - concrete specification of the [GameActor](#), which defines its moving algorithms and what it does.

```
#include <Pacman.hpp>
```

Inheritance diagram for Pacman:



Collaboration diagram for Pacman:



Public Member Functions

- **Pacman** ([TransformComponent](#) *transform, [GraphicsComponent](#) *graphics)
[Pacman](#) constructor - takes components to construct pacman object. O(1).
- void **TurnUp** () override
 Change game object's moving direction to up.
- void **TurnDown** () override
 Change game object's moving direction to down.
- void **TurnRight** () override
 Change game object's moving direction to right.
- void **TurnLeft** () override
 Change game object's moving direction to left.
- void **StopMoving** () override
 Change game object's moving to stop and don't move.
- void **Move** () override
 Moves game actor in its current direction.

- void [TurnTo](#) (const [Vector2](#)< float > &direction) override
Change game object's moving direction to specified direction.
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- void [Render](#) () const override
Render object on the screen.
- void [TryChangeDirection](#) (const [Vector2](#)< float > &newDirection)
Change direction if there are no possible collisions with other game objects. O(1).
- void [CollectBonus](#) ()
Collects bonus on the current tile node.

Public Member Functions inherited from [GameActor](#)

- [GameActor](#) ([TransformComponent](#) *transform, [GraphicsComponent](#) *graphics, [GameActor](#) &actor)
Construct a new [Game](#) Actor object and its components.
- ~[GameActor](#) () override
Destroy the [Game](#) Actor object and its components.
- const [GraphicsComponent](#) * [GetGraphics](#) () const
Get the Graphics component.
- [PhysicsComponent](#) * [GetPhysics](#) ()
Get the Physics component. O(1).
- const [PhysicsComponent](#) * [GetPhysics](#) () const
- [TransformComponent](#) * [GetTransform](#) ()
Get the Transform component.
- const [TransformComponent](#) * [GetTransform](#) () const
- float [GetVelocity](#) () const
Get velocity of the game actor.
- void [SetVelocity](#) (float value)
Set velocity of the game actor.
- int [GetState](#) () const
Get state of the game actor.
- void [SetState](#) (int value)
Set state of the game actor.

Public Member Functions inherited from [GameObject](#)

- virtual ~[GameObject](#) ()=default
Default destructor of the game object.

Additional Inherited Members

Protected Attributes inherited from [GameActor](#)

- [TransformComponent](#) * transform__
Transform component of the game object.
- [GraphicsComponent](#) * graphics__
Graphics component of the game object.
- [PhysicsComponent](#) * physics__
Physics component of the game object.
- float velocity__
Current game actor's velocity in the world.
- int state__
Represents current state of the game actor.

7.35.1 Detailed Description

[Pacman](#) class - concrete specification of the [GameActor](#), which defines its moving algorithms and what it does.

7.35.2 Constructor & Destructor Documentation

7.35.2.1 Pacman()

Pacman::Pacman (
 [TransformComponent](#) * transform,
 [GraphicsComponent](#) * graphics)

[Pacman](#) constructor - takes components to construct pacman object. O(1).

Parameters

transform	transform component of the object.
graphics	graphics component of the object.

7.35.3 Member Function Documentation

7.35.3.1 FixedUpdate()

void Pacman::FixedUpdate () [override], [virtual]
 Implements [GameObject](#).

7.35.3.2 Move()

void Pacman::Move () [override], [virtual]
 Moves game actor in its current direction.
 Implements [GameActor](#).

7.35.3.3 Render()

void Pacman::Render () const [override], [virtual]
 Render object on the screen.
 Implements [GameActor](#).

7.35.3.4 StopMoving()

void Pacman::StopMoving () [override], [virtual]
 Change game object's moving to stop and don't move.
 Implements [GameActor](#).

7.35.3.5 TryChangeDirection()

void Pacman::TryChangeDirection (
 const [Vector2](#)< float > & newDirection)
 Change direction if there are no possible collisions with other game objects. O(1).

Parameters

direction	new direction of the game object.
-----------	-----------------------------------

7.35.3.6 TurnDown()

void Pacman::TurnDown () [override], [virtual]
 Change game object's moving direction to down.

Implements [GameActor](#).

7.35.3.7 TurnLeft()

void Pacman::TurnLeft () [override], [virtual]

Change game object's moving direction to left.

Implements [GameActor](#).

7.35.3.8 TurnRight()

void Pacman::TurnRight () [override], [virtual]

Change game object's moving direction to right.

Implements [GameActor](#).

7.35.3.9 TurnTo()

void Pacman::TurnTo (

const [Vector2](#)< float > & direction) [override], [virtual]

Change game object's moving direction to specified direction.

Parameters

direction	where to turn.
-----------	----------------

Implements [GameActor](#).

7.35.3.10 TurnUp()

void Pacman::TurnUp () [override], [virtual]

Change game object's moving direction to up.

Implements [GameActor](#).

7.35.3.11 Update()

void Pacman::Update () [override], [virtual]

Updates this game object in the game.

Implements [GameObject](#).

The documentation for this class was generated from the following files:

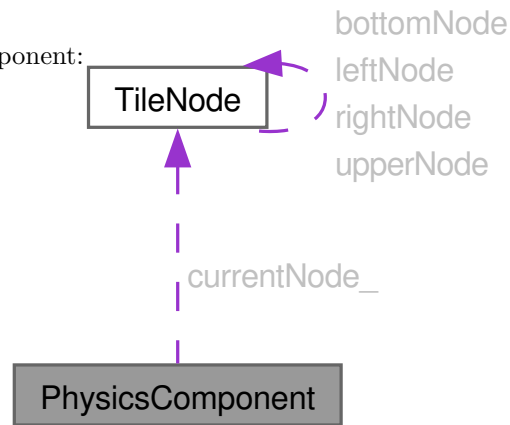
- src/game_objects/Pacman.hpp
- src/game_objects/Pacman.cpp

7.36 PhysicsComponent Class Reference

Component which resolves physics aspects of the game object. Resolves collisions and interaction with other game objects, while transform component only represents position in the world.

#include <PhysicsComponent.hpp>

Collaboration diagram for PhysicsComponent:



Public Member Functions

- [PhysicsComponent](#) ([GameActor](#) &gameActor)
Constructs this physics component. O(1).
- bool [TouchesObstacle](#) (const [Vector2](#)< float > &direction) const
Check if next tile is obstacle. O(1).
- bool [isOnNextTile](#) () const
Checks if current game object is on the next tile. O(1).
- void [Move](#) ([Vector2](#)< float > newPosition)
Translates object to its current direction resolving collisions. O(1).

Public Attributes

- [TileNode](#) * currentNode_
Current tile node where the game actor object is standing now.

7.36.1 Detailed Description

Component which resolves physics aspects of the game object. Resolves collisions and interaction with other game objects, while transform component only represents position in the world.

7.36.2 Constructor & Destructor Documentation

7.36.2.1 PhysicsComponent()

PhysicsComponent::PhysicsComponent (
 [GameActor](#) & gameActor) [explicit]
Constructs this physics component. O(1).

Parameters

gameActor	reference to game object where this component belongs to.
-----------	---

7.36.3 Member Function Documentation

7.36.3.1 isOnNextTile()

bool PhysicsComponent::isOnNextTile () const [nodiscard]
Checks if current game object is on the next tile. O(1).

Returns

true if game object is on the next tile.
false if game object is NOT on the next tile.

7.36.3.2 Move()

```
void PhysicsComponent::Move (
    Vector2< float > newPosition)
```

Translates object to its current direction resolving collisions. $O(1)$.

Parameters

newPosition	of the game object if no collisions occurs.
-------------	---

7.36.3.3 TouchesObstacle()

```
bool PhysicsComponent::TouchesObstacle (
    const Vector2< float > & direction) const [nodiscard]
```

Check if next tile is obstacle. $O(1)$.

Parameters

direction	from which to check next tile to this position.
-----------	---

Returns

true if next tile is obstacle.
false if next tile is NOT obstacle.

The documentation for this class was generated from the following files:

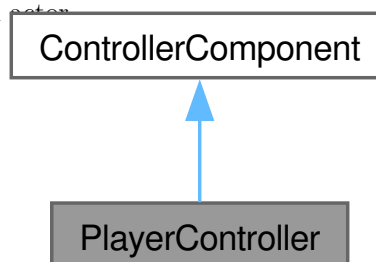
- src/components/PhysicsComponent.hpp
- src/components/PhysicsComponent.cpp

7.37 PlayerController Class Reference

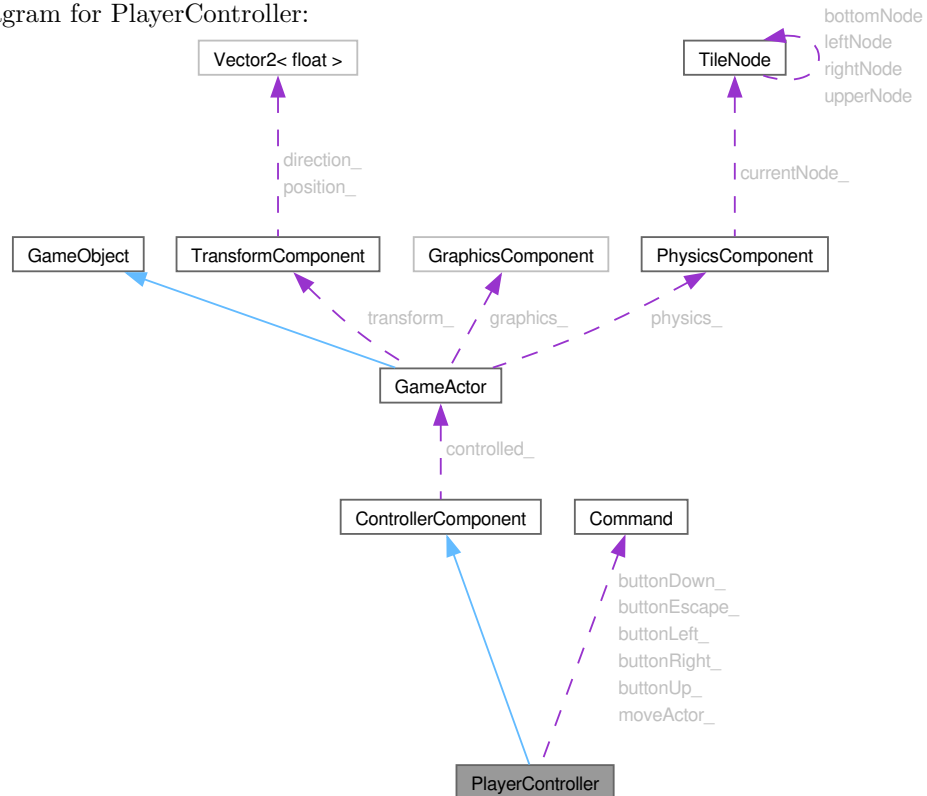
[Controller](#) to get user input and control given action

```
#include <PlayerController.hpp>
```

Inheritance diagram for PlayerController:



Collaboration diagram for PlayerController:



Public Member Functions

- [PlayerController](#) ([GameActor](#) *actor)
Constructs controller with injected actor.
- [~PlayerController](#) () override
Destroys controller and its commands.
- void [Update](#) () override
Updates controller component.
- void [FixedUpdate](#) () override
- void [ResetControlled](#) () override
Resets controlled game actor to some status.

Public Member Functions inherited from [ControllerComponent](#)

- [ControllerComponent](#) ([GameActor](#) *actor)
Constructs controller component.
- virtual [~ControllerComponent](#) ()=default
Destroys component but not the actor. Actor is destroyed by game itself when closing.

Protected Attributes

- [Command](#) * buttonUp_
[Command](#) executed by up arrow button.
- [Command](#) * buttonDown_
[Command](#) executed by down arrow button.
- [Command](#) * buttonLeft_
[Command](#) executed by left arrow button.

- [Command](#) * buttonRight_
[Command](#) executed by right arrow button.
- [Command](#) * buttonEscape_
[Command](#) executed by escape button.
- [Command](#) * moveActor_
[Command](#) to move actor further.

Protected Attributes inherited from [ControllerComponent](#)

- [GameActor](#) * controlled_
[Game](#) Actor under control of the given object.

7.37.1 Detailed Description

[Controller](#) to get user input and control given actor.

7.37.2 Constructor & Destructor Documentation

7.37.2.1 PlayerController()

PlayerController::PlayerController (
[GameActor](#) * actor) [explicit]

Constructs controller with injected actor.

Parameters

actor	to take control of by player.
-------	-------------------------------

7.37.3 Member Function Documentation

7.37.3.1 FixedUpdate()

void PlayerController::FixedUpdate () [override], [virtual]
 Implements [ControllerComponent](#).

7.37.3.2 ResetControlled()

void PlayerController::ResetControlled () [override], [virtual]
 Resets controlled game actor to some status.
 Implements [ControllerComponent](#).

7.37.3.3 Update()

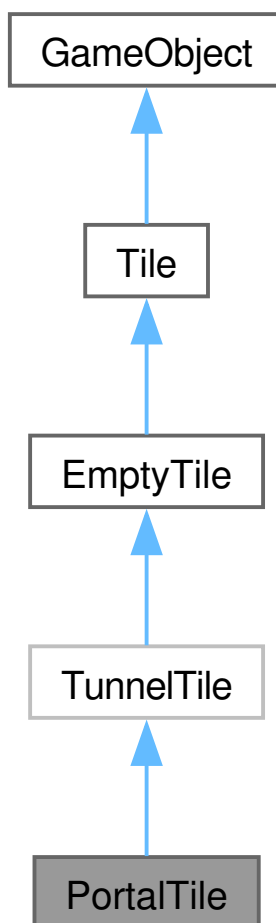
void PlayerController::Update () [override], [virtual]
 Updates controller component.
 Implements [ControllerComponent](#).

The documentation for this class was generated from the following files:

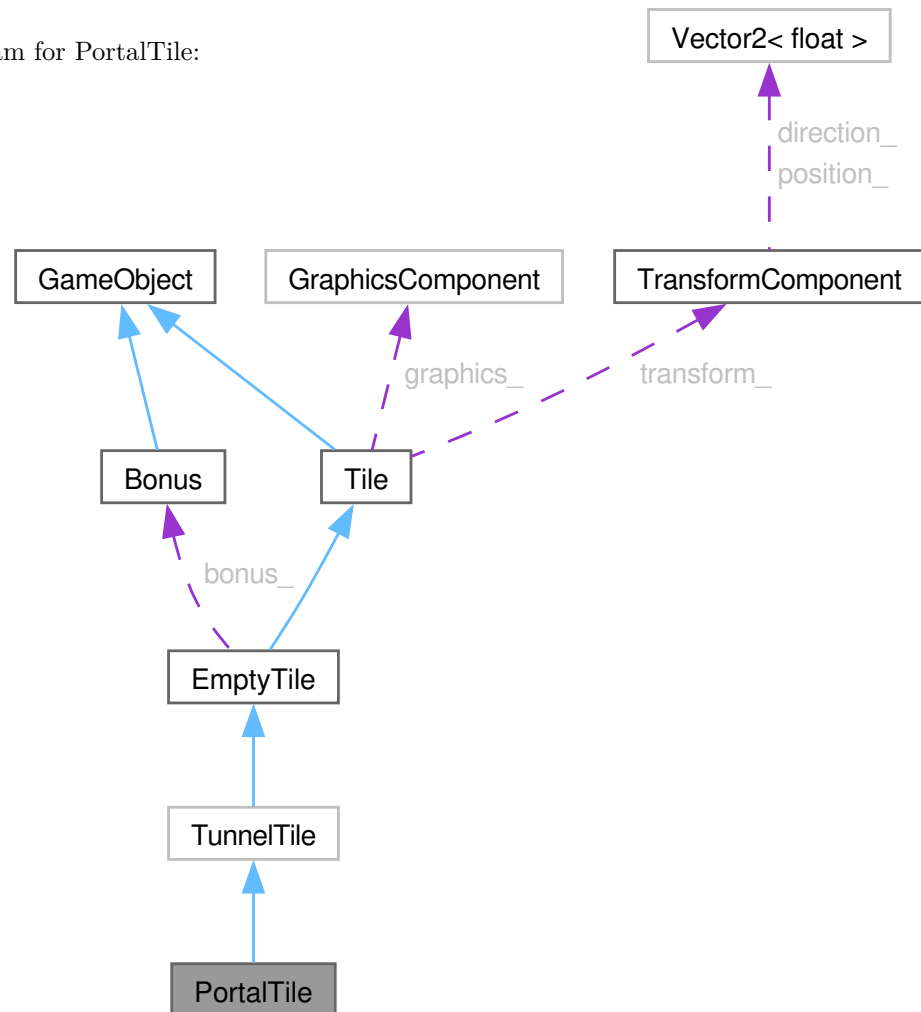
- src/components/PlayerController.hpp
- src/components/PlayerController.cpp

7.38 PortalTile Class Reference

Inheritance diagram for PortalTile:



Collaboration diagram for PortalTile:



Public Member Functions

- [PortalTile](#) ([Texture](#) *texture, const [Vector2< float >](#) &position, const [Vector2< float >](#) &direction)
Constructs portal tile.
- TileType [GetType](#) () const override
Get the type of the tile. O(1).
- bool [isPortal](#) () const override
Check if this tile is portal.
- void [Teleport](#) ([GameActor](#) &gameActor, [TileNode](#) &portal) override
Moves game actor transform and its current tile to another connected portalNode.

Public Member Functions inherited from [TunnelTile](#)

- [TunnelTile](#) ([Texture](#) *texture, const [Vector2< float >](#) &position)
Construct a new Tunnel [Tile](#) object.
- TileType [GetType](#) () const override
Get the type of the tile. O(1).
- bool [isTunnel](#) () const override
Check if this tile is tunnel.

Public Member Functions inherited from [EmptyTile](#)

- [EmptyTile](#) ([Texture](#) *texture, const [Vector2](#)< float > &position)
Construct a new Empty [Tile](#) object.
- void [Render](#) () const override
Render the tile on the game map. O(1).
- bool [AddBonus](#) ([Bonus](#) *bonus) override
Add bonus to tile.
- bool [CollectBonus](#) () override
Collect bonus from tile.
- bool [CanTurnAside](#) (const [GameActor](#) &gameActor) const override
Check if game object standing on this tile can turn to the side, right or left or not. O(1).

Public Member Functions inherited from [Tile](#)

- [Tile](#) ([Texture](#) *texture, const [Vector2](#)< float > &position)
Construct a new [Tile](#) object. O(1).
- [Tile](#) ([Texture](#) *texture, [TransformComponent](#) *transform)
- ~[Tile](#) () override
Destroys the [Tile](#) object. O(1).
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- virtual bool [isObstacle](#) () const
Check if this tile is obstacle or not. O(1).
- [TransformComponent](#) * [GetTransform](#) ()
Get Transform component of the tile. O(1).
- const [TransformComponent](#) * [GetTransform](#) () const
Get Transform component of the tile. O(1).

Public Member Functions inherited from [GameObject](#)

- virtual ~[GameObject](#) ()=default
Default destructor of the game object.

Additional Inherited Members

Static Public Member Functions inherited from [Tile](#)

- static [Vector2](#)< float > [GetOffsetFromTileCenter](#) (const [GameActor](#) &gameActor, const [Tile](#) *tile)
Get distance between game object and its tile center according to the current direction. O(1).

Protected Attributes inherited from [EmptyTile](#)

- [Bonus](#) * bonus_
[Bonus](#) on the tile.

Protected Attributes inherited from [Tile](#)

- [GraphicsComponent](#) * graphics_
Graphics component of the tile class.
- [TransformComponent](#) * transform_
Transform component of the tile class.

7.38.1 Constructor & Destructor Documentation

7.38.1.1 PortalTile()

PortalTile::PortalTile (
 [Texture](#) * texture,
 const [Vector2](#)< float > & position,
 const [Vector2](#)< float > & direction)

Constructs portal tile.

Parameters

texture	of the tile.
position	of the tile in the game map.
direction	where is the exit from the portal, not entrance.

7.38.2 Member Function Documentation

7.38.2.1 GetType()

TileType PortalTile::GetType () const [override], [virtual]
 Get the type of the tile. O(1).

Returns

type of the tile.

Reimplemented from [EmptyTile](#).

7.38.2.2 isPortal()

bool PortalTile::isPortal () const [override], [virtual]
 Check if this tile is portal.

Returns

true if is portal.

Reimplemented from [Tile](#).

7.38.2.3 Teleport()

void PortalTile::Teleport (
 [GameActor](#) & gameActor,
 [TileNode](#) & portalNode) [override], [virtual]

Moves game actor transform and its current tile to another connected portalNode.

Parameters

gameActor	to be moved to another portalNode.
portalNode	represents tile node of the portalNode, which teleports now the game actor.

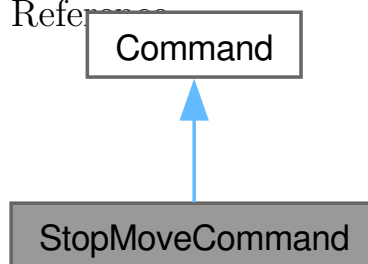
Reimplemented from [Tile](#).

The documentation for this class was generated from the following files:

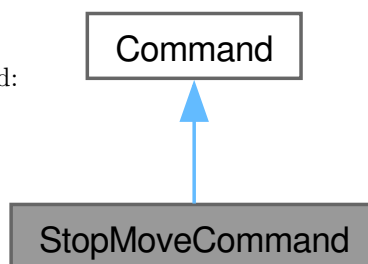
- src/game_objects/PortalTile.hpp
- src/game_objects/PortalTile.cpp

7.39 StopMoveCommand Class Reference

Inheritance diagram for StopMoveCommand:



Collaboration diagram for StopMoveCommand:



Public Member Functions

- void [Execute](#) ([GameActor](#) *actor) override
Executes command with give actor.

7.39.1 Member Function Documentation

7.39.1.1 Execute()

void StopMoveCommand::Execute (
[GameActor](#) * actor) [override], [virtual]
 Executes command with give actor.

Parameters

actor	which should execute the command.
-------	-----------------------------------

Implements [Command](#).

The documentation for this class was generated from the following files:

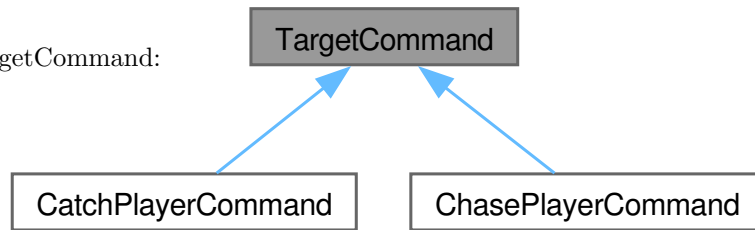
- src/components/Command.hpp
- src/components/Command.cpp

7.40 TargetCommand Class Reference

Represent command where are interacting two objects.

```
#include <AICommands.hpp>
```

Inheritance diagram for TargetCommand:



Public Member Functions

- virtual void [Execute](#) ([GameActor](#) *controlled, [GameActor](#) *target)=0
Executes command with give game actor objects.

7.40.1 Detailed Description

Represent command where are interacting two objects.

7.40.2 Member Function Documentation

7.40.2.1 Execute()

virtual void TargetCommand::Execute (
[GameActor](#) * controlled,
[GameActor](#) * target) [pure virtual]
 Executes command with give game actor objects.

Parameters

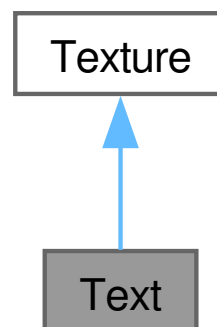
controlled	first object which does something.
target	which is affected by the controlled object.

Implemented in [CatchPlayerCommand](#), and [ChasePlayerCommand](#).
 The documentation for this class was generated from the following file:

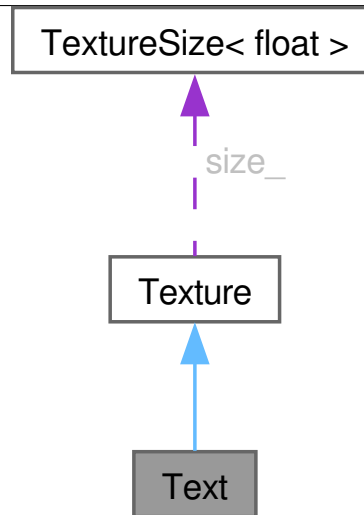
- src/components/AICommands.hpp

7.41 Text Class Reference

Inheritance diagram for Text:



Collaboration diagram for Text:



Public Member Functions

- `Text (const Text &src)=delete`
- `bool LoadFont (const string &path, size_t fontSize=DefaultFontSize)`
- `bool LoadText (const string &msg, SDL_Color color)`
- `bool LoadText (const string &msg, SDL_Color color, const TextureSize< float > &size)`
- `bool LoadTexture (const string &msg) override`
Loads texture from .png image and applies it for renderer. O(1).
- `bool LoadTexture (const string &msg, const TextureSize< float > &size) override`
Loads texture from .png image and applies it for renderer. O(1).
- `void RenderTexture (const Vector2< float > &position) const override`
Render this texture on the screen by renderer. O(1).
- `void RenderTexture (const Vector2< float > &position, double angle) const override`

Public Member Functions inherited from [Texture](#)

- `Texture ()`
Construct and inits sdl texture with nullptr and width and height with 0. O(1).
- `virtual ~Texture () noexcept`
Deletes texture. Use DestroyTexture to safely destroy sdl texture. O(1).
- `Texture (const Texture &src)=delete`
Deleted copy constructor.
- `Texture (Texture &&src) noexcept`
Move constructor.
- `virtual bool SetNativeTextureSize ()`
Queries texture for its native image dimensions and sets the size of the texture in pixels. O(1).
- `void SetTextureSize (const TextureSize< float > &size)`
Set texture dimensions using user defined size in pixels. O(1).
- `const TextureSize< float > & GetSize () const`
Get texture size of this texture. O(1).

Static Public Attributes

- `static constexpr size_t DefaultFontSize = 16`

Additional Inherited Members

Protected Member Functions inherited from [Texture](#)

- `SDL_FRect CreateRenderRectCentered (const Vector2< float > &position) const`
Create SDL float rectangle object from center position. Uses texture size to find left upper corner where SDL rect should be defined for rendering. O(1).
- `void DestroyTexture ()`
Destroys current texture and prepares for loading another texture. O(1).

Protected Attributes inherited from [Texture](#)

- `SDL_Texture * texture__`
Object to store and handle textures in SDL.
- `TextureSize< float > size__`
Dimensions of the loaded texture (of the image) in pixels.

7.41.1 Member Function Documentation

7.41.1.1 LoadTexture() [1/2]

```
bool Text::LoadTexture (
    const string & path) [override], [virtual]
```

Loads texture from .png image and applies it for renderer. O(1).

Parameters

path	to the loaded image.
------	----------------------

Returns

true if successfully loaded texture.

false if failed to load texture.

Reimplemented from [Texture](#).

7.41.1.2 LoadTexture() [2/2]

```
bool Text::LoadTexture (
    const string & path,
    const TextureSize< float > & size) [override], [virtual]
```

Loads texture from .png image and applies it for renderer. O(1).

Parameters

path	to the loaded image.
size	of the texture.

Returns

true if successfully loaded texture.

false if failed to load texture.

Reimplemented from [Texture](#).

7.41.1.3 RenderTexture() [1/2]

```
void Text::RenderTexture (
    const Vector2< float > & position) const [override], [virtual]
```

Render this texture on the screen by renderer. O(1).

Parameters

position	in pixels where to show image on the screen.
----------	--

Reimplemented from [Texture](#).

7.41.1.4 RenderTexture() [2/2]

```
void Text::RenderTexture (
    const Vector2< float > & position,
    double angle) const [override], [virtual]
```

Reimplemented from [Texture](#).

The documentation for this class was generated from the following files:

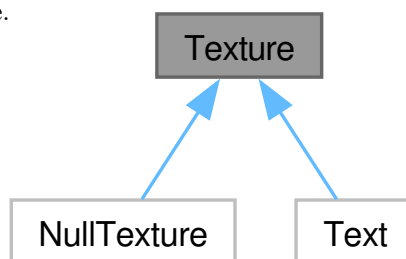
- src/structures/Text.hpp
- src/structures/Text.cpp

7.42 Texture Class Reference

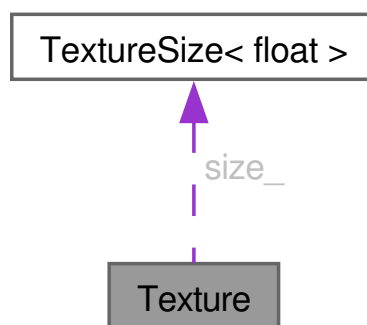
[Texture](#) wrapper class around SDL's texture.

```
#include <Texture.hpp>
```

Inheritance diagram for Texture:



Collaboration diagram for Texture:



Public Member Functions

- `Texture ()`
Construct and inits sdl texture with nullptr and width and height with 0. O(1).

- virtual `~Texture () noexcept`
Deletes texture. Use `DestroyTexture` to safely destroy sdl texture. $O(1)$.
- `Texture (const Texture &src)=delete`
Deleted copy constructor.
- `Texture (Texture &&src) noexcept`
Move constructor.
- virtual bool `LoadTexture (const string &path)`
Loads texture from .png image and applies it for renderer. $O(1)$.
- virtual bool `LoadTexture (const string &path, const TextureSize< float > &size)`
Loads texture from .png image and applies it for renderer. $O(1)$.
- virtual void `RenderTexture (const Vector2< float > &position) const`
Render this texture on the screen by renderer. $O(1)$.
- virtual void `RenderTexture (const Vector2< float > &position, double angle) const`
- virtual bool `SetNativeTextureSize ()`
Queries texture for its native image dimensions and sets the size of the texture in pixels. $O(1)$.
- void `SetTextureSize (const TextureSize< float > &size)`
Set texture dimensions using user defined size in pixels. $O(1)$.
- const `TextureSize< float > & GetSize () const`
Get texture size of this texture. $O(1)$.

Protected Member Functions

- `SDL_FRect CreateRenderRectCentered (const Vector2< float > &position) const`
Create SDL float rectangle object from center position. Uses texture size to find left upper corner where SDL rect should be defined for rendering. $O(1)$.
- void `DestroyTexture ()`
Destroys current texture and prepares for loading another texture. $O(1)$.

Protected Attributes

- `SDL_Texture * texture__`
Object to store and handle textures in SDL.
- `TextureSize< float > size__`
Dimensions of the loaded texture (of the image) in pixels.

7.42.1 Detailed Description

`Texture` wrapper class around SDL's texture.

7.42.2 Member Function Documentation

7.42.2.1 CreateRenderRectCentered()

`SDL_FRect Texture::CreateRenderRectCentered (const Vector2< float > & position) const` [nodiscard], [protected]

Create SDL float rectangle object from center position. Uses texture size to find left upper corner where SDL rect should be defined for rendering. $O(1)$.

Parameters

position	center of the game object position.
----------	-------------------------------------

Returns

`SDL_FRect` rectangle structure prepared for rendering.

7.42.2.2 GetSize()

const [TextureSize](#)< float > & Texture::GetSize () const [inline], [nodiscard]
 Get texture size of this texture. O(1).

Returns

reference to const texture size.

7.42.2.3 LoadTexture() [1/2]

bool Texture::LoadTexture (
 const string & path) [virtual]

Loads texture from .png image and applies it for renderer. O(1).

Parameters

path	to the loaded image.
------	----------------------

Returns

true if successfully loaded texture.

false if failed to load texture.

Reimplemented in [Text](#).

7.42.2.4 LoadTexture() [2/2]

bool Texture::LoadTexture (
 const string & path,
 const [TextureSize](#)< float > & size) [virtual]

Loads texture from .png image and applies it for renderer. O(1).

Parameters

path	to the loaded image.
size	of the texture.

Returns

true if successfully loaded texture.

false if failed to load texture.

Reimplemented in [Text](#).

7.42.2.5 RenderTexture()

void Texture::RenderTexture (
 const [Vector2](#)< float > & position) const [virtual]

Render this texture on the screen by renderer. O(1).

Parameters

position	in pixels where to show image on the screen.
----------	--

Reimplemented in [NullTexture](#), and [Text](#).

7.42.2.6 SetNativeTextureSize()

bool Texture::SetNativeTextureSize () [virtual]

Queries texture for its native image dimensions and sets the size of the texture in pixels. O(1).

Returns

- true if successfully set up native size.
- false if failed to query image size and set it up.

Reimplemented in [NullTexture](#).

7.42.2.7 SetTextureSize()

void Texture::SetTextureSize (
const [TextureSize](#)< float > & size) [inline]

Set texture dimensions using user defined size in pixels. O(1).

Parameters

size	to set to this texture.
------	-------------------------

The documentation for this class was generated from the following files:

- src/structures/Texture.hpp
- src/structures/Texture.cpp

7.43 TextureSize< T__ > Struct Template Reference

Template which represents size of the texture.

#include <TextureSize.hpp>

Public Member Functions

- [TextureSize](#) (T__ w=0, T__ h=0)
Construct a new [Texture](#) Size object. O(1).
- [TextureSize](#) (const [TextureSize](#) &src)
Copy constructor. O(1).
- [TextureSize](#) & operator= (const [TextureSize](#) &rhs)
Copy-assignment constructor. O(1).

Static Public Member Functions

- static [TextureSize Zero](#) ()

Public Attributes

- T__ width__
Size of the texture - its width and height.
- T__ height__

Friends

- template<typename U__>
[TextureSize](#)< U__ > operator/ (const [TextureSize](#)< U__ > &lhs, const U__ &rhs)
- template<typename U__, typename L__>
[TextureSize](#)< U__ > operator/ (const [TextureSize](#)< U__ > &lhs, const L__ &rhs)

7.43.1 Detailed Description

```
template<typename T_>
struct TextureSize< T_ >
```

Template which represents size of the texture.

Template Parameters

T↔	type of the width and height - float, int or double.
—	

7.43.2 Constructor & Destructor Documentation

7.43.2.1 TextureSize() [1/2]

```
template<typename T_>
TextureSize< T_ >::TextureSize (
    T_ w = 0,
    T_ h = 0)
```

Construct a new [Texture](#) Size object. O(1).

Parameters

w	width of the texture, default 0.
h	height of the texture, default 0.

7.43.2.2 TextureSize() [2/2]

```
template<typename T_>
TextureSize< T_ >::TextureSize (
    const TextureSize< T_ > & src)
```

Copy constructor. O(1).

Parameters

src	source texture size from which construct copy of this object.
-----	---

7.43.3 Member Function Documentation

7.43.3.1 Zero()

```
template<typename T_>
TextureSize< T_ > TextureSize< T_ >::Zero () [static]
```

Returns

texture size object with (width = 0, height = 0). O(1).

The documentation for this struct was generated from the following files:

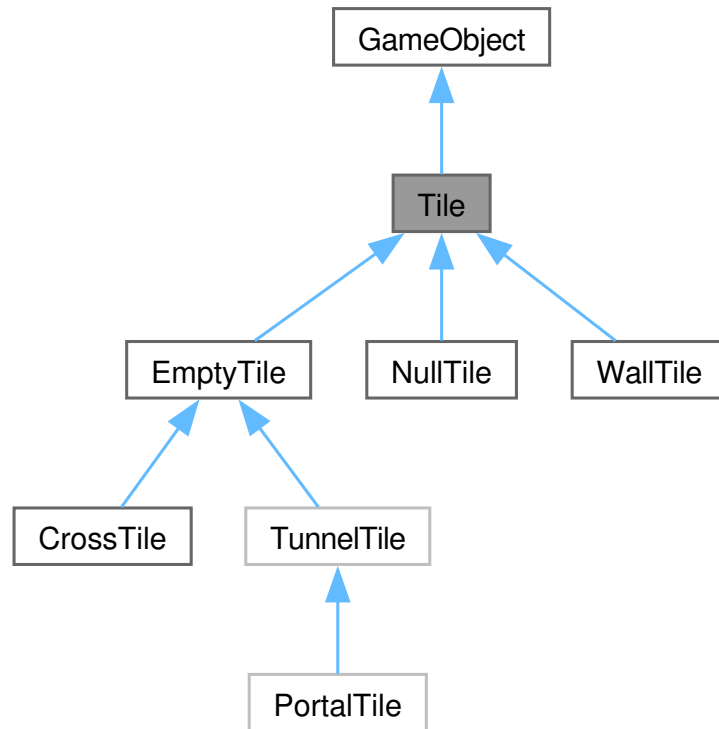
- src/structures/TextureSize.hpp
- src/structures/TextureSize.inl

7.44 Tile Class Reference

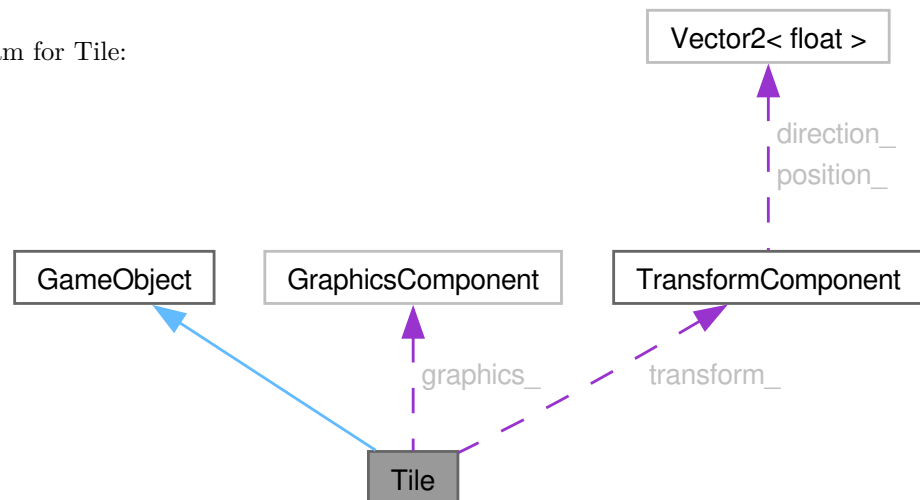
Abstract class representing Tiles in the map.

```
#include <Tile.hpp>
```

Inheritance diagram for Tile:



Collaboration diagram for Tile:



Public Member Functions

- **Tile** (**Texture** *texture, const **Vector2**< float > &position)
Construct a new **Tile** object. O(1).
- **Tile** (**Texture** *texture, **TransformComponent** *transform)
- **~Tile** () override
Destroys the **Tile** object. O(1).
- virtual **TileType** **GetType** () const =0
Get the type of the tile. O(1).

- virtual void [Render](#) () const
Render the tile on the game map. O(1).
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- virtual bool [isObstacle](#) () const
Check if this tile is obstacle or not. O(1).
- virtual bool [isTunnel](#) () const
Check if this tile is tunnel.
- virtual bool [isPortal](#) () const
Check if this tile is portal.
- virtual void [Teleport](#) ([GameActor](#) &gameActor, [TileNode](#) &portalNode)
Moves game actor transform and its current tile to another connected portalNode.
- virtual bool [CanTurnAside](#) (const [GameActor](#) &gameActor) const
Check if game object standing on this tile can turn to the side, right or left or not. O(1).
- virtual bool [AddBonus](#) ([Bonus](#) *bonus)
Add bonus to tile.
- virtual bool [CollectBonus](#) ()
Collect bonus from tile.
- [TransformComponent](#) * GetTransform ()
Get Transform component of the tile. O(1).
- const [TransformComponent](#) * GetTransform () const
Get Transform component of the tile. O(1).

Public Member Functions inherited from [GameObject](#)

- virtual ~GameObject ()=default
Default destructor of the game object.

Static Public Member Functions

- static [Vector2](#)< float > [GetOffsetFromTileCenter](#) (const [GameActor](#) &gameActor, const [Tile](#) *tile)
Get distance between game object and its tile center according to the current direction. O(1).

Protected Attributes

- [GraphicsComponent](#) * graphics_
Graphics component of the tile class.
- [TransformComponent](#) * transform_
Transform component of the tile class.

7.44.1 Detailed Description

Abstract class representing Tiles in the map.

7.44.2 Constructor & Destructor Documentation

7.44.2.1 Tile()

Tile::Tile (

[Texture](#) * texture,
const [Vector2](#)< float > & position)

Construct a new [Tile](#) object. O(1).

Parameters

texture	texture to construct graphics component.
position	of the tile in the map in pixels.

7.44.3 Member Function Documentation

7.44.3.1 AddBonus()

bool Tile::AddBonus (
[Bonus](#) * bonus) [virtual]
 Add bonus to tile.

Parameters

bonus	to be added to tile.
-------	----------------------

Returns

true if bonus was successfully added.

Reimplemented in [EmptyTile](#).

7.44.3.2 CanTurnAside()

bool Tile::CanTurnAside (
 const [GameActor](#) & gameActor) const [nodiscard], [virtual]
 Check if game object standing on this tile can turn to the side, right or left or not. O(1).

Parameters

gameActor	standing on this tile and trying to turn to another direction.
-----------	--

Returns

true if game object can turn aside.

Reimplemented in [CrossTile](#), and [EmptyTile](#).

7.44.3.3 CollectBonus()

bool Tile::CollectBonus () [virtual]
 Collect bonus from tile.

Returns

true if successfully collected bonus.

Reimplemented in [EmptyTile](#).

7.44.3.4 FixedUpdate()

void Tile::FixedUpdate () [override], [virtual]
 Implements [GameObject](#).

7.44.3.5 GetOffsetFromTileCenter()

[Vector2](#)< float > Tile::GetOffsetFromTileCenter (
 const [GameActor](#) & gameActor,
 const [Tile](#) * tile) [static], [nodiscard]

Get distance between game object and its tile center according to the current direction. O(1).

Returns

offset vector from tile center to game object position.

7.44.3.6 GetType()

virtual TileType Tile::GetType () const [nodiscard], [pure virtual]

Get the type of the tile. O(1).

Returns

type of the tile.

Implemented in [CrossTile](#), [EmptyTile](#), [NullTile](#), [PortalTile](#), [TunnelTile](#), and [WallTile](#).

7.44.3.7 isObstacle()

bool Tile::isObstacle () const [nodiscard], [virtual]

Check if this tile is obstacle or not. O(1).

Returns

true if tile is obstacle.

Reimplemented in [NullTile](#), and [WallTile](#).

7.44.3.8 isPortal()

bool Tile::isPortal () const [nodiscard], [virtual]

Check if this tile is portal.

Returns

true if is portal.

Reimplemented in [PortalTile](#).

7.44.3.9 isTunnel()

bool Tile::isTunnel () const [nodiscard], [virtual]

Check if this tile is tunnel.

Returns

true if is tunnel.

Reimplemented in [TunnelTile](#).

7.44.3.10 Render()

void Tile::Render () const [virtual]

Render the tile on the game map. O(1).

Reimplemented in [CrossTile](#), [EmptyTile](#), [NullTile](#), and [WallTile](#).

7.44.3.11 Teleport()

void Tile::Teleport (

[GameActor](#) & gameActor,

[TileNode](#) & portalNode) [virtual]

Moves game actor transform and its current tile to another connected portalNode.

Parameters

gameActor	to be moved to another portalNode.
-----------	------------------------------------

portalNode	represents tile node of the portalNode, which teleports now the game actor.
------------	---

Reimplemented in [PortalTile](#).

7.44.3.12 Update()

void Tile::Update () [override], [virtual]

Updates this game object in the game.

Implements [GameObject](#).

The documentation for this class was generated from the following files:

- src/game_objects/Tile.hpp
- src/game_objects/Tile.cpp

7.45 TileData Struct Reference

Structure that represents read data about the tile from each layer of the game map.

#include <MapReader.hpp>

Public Attributes

- char type_
Represents type of the tile - wall, empty, cross etc.
- char option_
Represents option on the tile - bonus, cherry, corner wall etc.
- char direction_
Represents direction of the tile in its transform component.

7.45.1 Detailed Description

Structure that represents read data about the tile from each layer of the game map.

The documentation for this struct was generated from the following file:

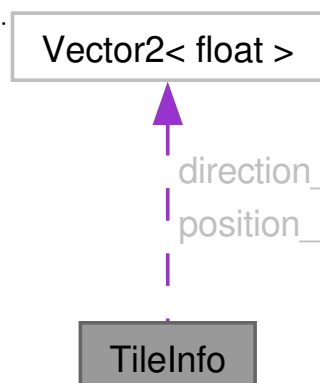
- src/structures/MapReader.hpp

7.46 TileInfo Struct Reference

Structure that represents information about tile.

#include <MapParser.hpp>

Collaboration diagram for TileInfo:



Public Attributes

- `TileType type_ = TileType::NULL_TILE`
Type of the tile.
- `TileOption option_ = TileOption::NONE`
Option feature of the tile.
- `Vector2< float > position_ = Vector2<float>::Zero()`
Position of the tile in pixels.
- `Vector2< float > direction_ = Vector2<float>::Zero()`
Direction of the tile.

7.46.1 Detailed Description

Structure that represents information about tile.

The documentation for this struct was generated from the following file:

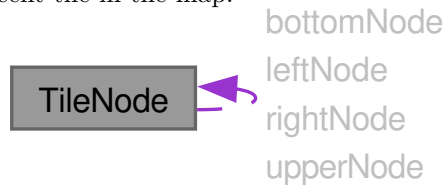
- `src/structures/MapParser.hpp`

7.47 TileNode Class Reference

Represents connections between each adjacent tile in the map.

`#include <TileNode.hpp>`

Collaboration diagram for TileNode:



Public Member Functions

- `TileNode (Tile *wrapped)`
Constructs tile node object wrapping tile with pointers to adjacent nodes. $O(1)$.
- `~TileNode ()`
Destroy tile node and wrapped tile. $O(1)$.
- `TileType GetType () const`
Get the type of the wrapped tile. $O(1)$.
- `void Render () const`
Render wrapped tile onto the screen. $O(1)$.
- `bool isObstacle () const`
Checks if wrapped tile is obstacle. $O(1)$.
- `bool isTunnel () const`
Checks if wrapped tile is tunnel.
- `bool isPortal () const`
Checks if wrapped tile is portal.
- `void Teleport (GameActor &gameActor)`
Teleports game actor to the another portal node in the game map.
- `bool CanTurnAside (const GameActor &gameActor) const`
Checks if game actor object can turn right or left on the wrapped tile. $O(1)$.
- `bool AddBonus (Bonus *bonus)`
Add bonus to the wrapped tile.

- `bool CollectBonus ()`
Collect bonus from the wrapped tile.
- `TransformComponent * GetTransform ()`
Get transform component of the wrapped tile.
- `const TransformComponent * GetTransform () const`
Get transform component of the wrapped tile.
- `Tile * GetTile ()`
Get wrapped tile.
- `const Tile * GetTile () const`
Get wrapped tile.
- `TileNode * NextNode (const Vector2< float > &direction)`
Get next node to specified direction.

Static Public Member Functions

- `static Vector2< float > GetOffsetFromNodeCenter (const GameActor &gameActor, const TileNode *node)`
Get distance between game object and its tile node center according to the current game actor object direction. O(1).

Public Attributes

- `TileNode * upperNode`
Next `TileNode` to up in the game map.
- `TileNode * bottomNode`
Next `TileNode` to down in the game map.
- `TileNode * rightNode`
Next `TileNode` to right in the game map.
- `TileNode * leftNode`
Next `TileNode` to left in the game map.

7.47.1 Detailed Description

Represents connections between each adjacent tile in the map.

7.47.2 Member Function Documentation

7.47.2.1 AddBonus()

`bool TileNode::AddBonus (`
`Bonus * bonus) [inline]`

Add bonus to the wrapped tile.

Parameters

bonus	to be added.
-------	--------------

Returns

true if bonus was added.
false if bonus wasn't added.

7.47.2.2 CanTurnAside()

```
bool TileNode::CanTurnAside (
    const GameActor & gameActor) const    [inline], [nodiscard]
```

Checks if game actor object can turn right or left on the wrapped tile. $O(1)$.

Parameters

gameActor	standing on the tile.
-----------	-----------------------

Returns

true if can turn aside.
false if can NOT turn aside.

7.47.2.3 CollectBonus()

```
bool TileNode::CollectBonus ()    [inline]
```

Collect bonus from the wrapped tile.

Returns

true if bonus was collected.
false if bonus wasn't collected.

7.47.2.4 GetOffsetFromNodeCenter()

```
Vector2< float > TileNode::GetOffsetFromNodeCenter (
    const GameActor & gameActor,
    const TileNode * node)    [static]
```

Get distance between game object and its tile node center according to the current game actor object direction. $O(1)$.

Returns

offset vector from the tile node center to game object position.

7.47.2.5 GetType()

```
TileType TileNode::GetType () const    [inline]
```

Get the type of the wrapped tile. $O(1)$.

Returns

wrapped tile type.

7.47.2.6 isObstacle()

```
bool TileNode::isObstacle () const    [inline], [nodiscard]
```

Checks if wrapped tile is obstacle. $O(1)$.

Returns

true if is obstacle.
false if is NOT obstacle.

7.47.2.7 isPortal()

`bool TileNode::isPortal () const [inline]`
Checks if wrapped tile is portal.

Returns

true if is portal.
false if is NOT portal.

7.47.2.8 isTunnel()

`bool TileNode::isTunnel () const [inline]`
Checks if wrapped tile is tunnel.

Returns

true if is tunnel.
false if is NOT tunnel.

7.47.2.9 NextNode()

`TileNode * TileNode::NextNode (`
 `const Vector2< float > & direction) [nodiscard]`
 Get next node to specified direction.

Parameters

direction	to which get next tile.
-----------	-------------------------

Returns

next tile node.

7.47.2.10 Teleport()

`void TileNode::Teleport (`
 `GameActor & gameActor) [inline]`
 Teleports game actor to the another portal node in the game map.

Parameters

gameActor	which is teleported to another tile node.
-----------	---

The documentation for this class was generated from the following files:

- `src/structures/TileNode.hpp`
- `src/structures/TileNode.cpp`

7.48 TileNodeBuilder Class Reference

Represents class that builds all tile nodes using tile info and connects them with each other.
`#include <TileNodeBuilder.hpp>`

Public Member Functions

- `TileNodeBuilder ()`
Constructs tile node builder object.
- `void LoadTileNodes (const vector< vector< TileInfo > > &tilesInfo, const MapHeader &header)`
Load tile nodes using tiles info to the `tileNodes_` 2D array.
- `void ClearTileNodes ()`
Destroys all loaded tiles if there are any. $O(n)$.
- `const vector< vector< TileNode * > > & GetTileNodesMap () const`
Get reference to the loaded collection of tile nodes.
- `MapInfo GetMapInfo () const`
Get map info object with information about the loaded map.

7.48.1 Detailed Description

Represents class that builds all tile nodes using tile info and connects them with each other.

7.48.2 Member Function Documentation

7.48.2.1 LoadTileNodes()

```
void TileNodeBuilder::LoadTileNodes (
    const vector< vector< TileInfo > > & tilesInfo,
    const MapHeader & header)
Load tile nodes using tiles info to the tileNodes_ 2D array.
```

Parameters

<code>tilesInfo</code>	represents collection of tile info objects.
<code>header</code>	represents map header with information about the map.

The documentation for this class was generated from the following files:

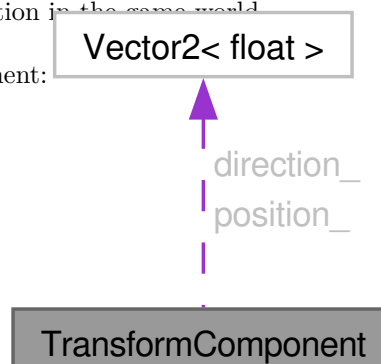
- `src/structures/TileNodeBuilder.hpp`
- `src/structures/TileNodeBuilder.cpp`

7.49 TransformComponent Class Reference

Class which represents coordinates and direction in the same world.

```
#include <TransformComponent.hpp>
```

Collaboration diagram for TransformComponent:



Public Member Functions

- `TransformComponent ()`
Construct Transform component.
- `TransformComponent (const Vector2< float > &initialPosition, const Vector2< float > &direction)`
Construct Transform component.
- `TransformComponent (const Vector2< float > &initialPosition)`
Constructs Transform component.
- `void Translate (const Vector2< float > &newPosition)`
Moves transform position to new position.

Public Attributes

- `Vector2< float > direction_`
direction of the gameObject which has this transform components.
- `Vector2< float > position_`
current gameObject's position in the world.

7.49.1 Detailed Description

Class which represents coordinates and direction in the game world.

7.49.2 Constructor & Destructor Documentation

7.49.2.1 TransformComponent() [1/2]

`TransformComponent::TransformComponent (`
 `const Vector2< float > & initialPosition,`
 `const Vector2< float > & direction)`
 Construct Transform component.

Parameters

<code>initialPosition</code>	represents initial position in the game world.
<code>direction</code>	sets the initial direction of the object.

7.49.2.2 TransformComponent() [2/2]

`TransformComponent::TransformComponent (`
 `const Vector2< float > & initialPosition)`
 Constructs Transform component.

Parameters

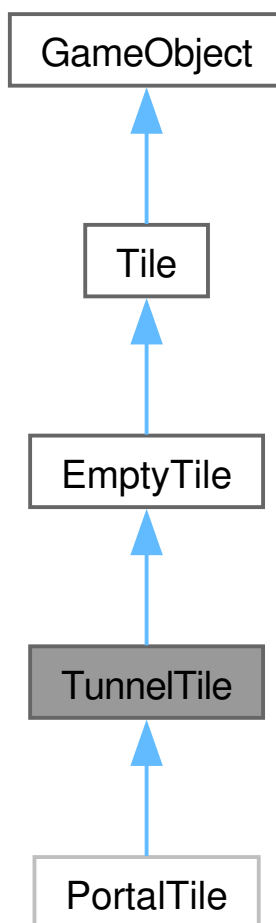
<code>initialPosition</code>	represents initial position in the game world.
------------------------------	--

The documentation for this class was generated from the following files:

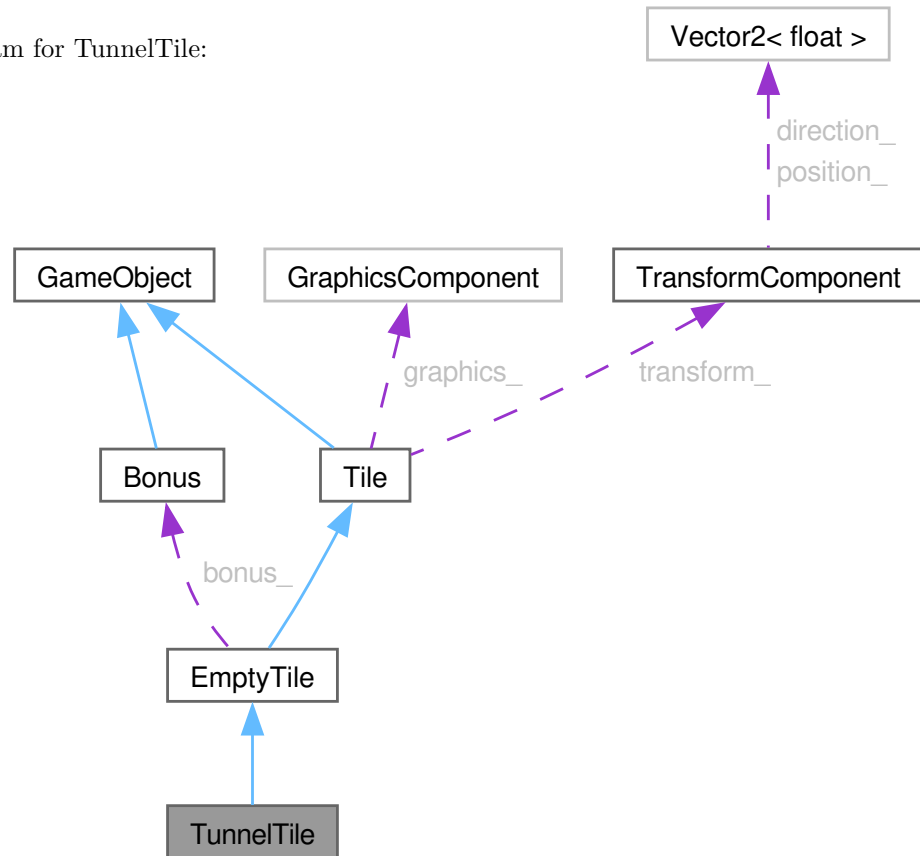
- `src/components/TransformComponent.hpp`
- `src/components/TransformComponent.cpp`

7.50 TunnelTile Class Reference

Inheritance diagram for TunnelTile:



Collaboration diagram for TunnelTile:



Public Member Functions

- [TunnelTile](#) ([Texture](#) *texture, const [Vector2< float >](#) &position)
Construct a new Tunnel [Tile](#) object.
- [TileType](#) [GetType](#) () const override
Get the type of the tile. O(1).
- bool [isTunnel](#) () const override
Check if this tile is tunnel.

Public Member Functions inherited from [EmptyTile](#)

- [EmptyTile](#) ([Texture](#) *texture, const [Vector2< float >](#) &position)
Construct a new Empty [Tile](#) object.
- void [Render](#) () const override
Render the tile on the game map. O(1).
- bool [AddBonus](#) ([Bonus](#) *bonus) override
Add bonus to tile.
- bool [CollectBonus](#) () override
Collect bonus from tile.
- bool [CanTurnAside](#) (const [GameActor](#) &gameActor) const override
Check if game object standing on this tile can turn to the side, right or left or not. O(1).

Public Member Functions inherited from [Tile](#)

- [Tile](#) ([Texture](#) *texture, const [Vector2](#)< float > &position)
Construct a new [Tile](#) object. O(1).
- [Tile](#) ([Texture](#) *texture, [TransformComponent](#) *transform)
- [~Tile](#) () override
Destroys the [Tile](#) object. O(1).
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- virtual bool [isObstacle](#) () const
Check if this tile is obstacle or not. O(1).
- virtual bool [isPortal](#) () const
Check if this tile is portal.
- virtual void [Teleport](#) ([GameActor](#) &gameActor, [TileNode](#) &portalNode)
Moves game actor transform and its current tile to another connected portalNode.
- [TransformComponent](#) * [GetTransform](#) ()
Get Transform component of the tile. O(1).
- const [TransformComponent](#) * [GetTransform](#) () const
Get Transform component of the tile. O(1).

Public Member Functions inherited from [GameObject](#)

- virtual [~GameObject](#) ()=default
Default destructor of the game object.

Additional Inherited Members

Static Public Member Functions inherited from [Tile](#)

- static [Vector2](#)< float > [GetOffsetFromTileCenter](#) (const [GameActor](#) &gameActor, const [Tile](#) *tile)
Get distance between game object and its tile center according to the current direction. O(1).

Protected Attributes inherited from [EmptyTile](#)

- [Bonus](#) * bonus_
[Bonus](#) on the tile.

Protected Attributes inherited from [Tile](#)

- [GraphicsComponent](#) * graphics_
Graphics component of the tile class.
- [TransformComponent](#) * transform_
Transform component of the tile class.

7.50.1 Constructor & Destructor Documentation

7.50.1.1 TunnelTile()

TunnelTile::TunnelTile (
 [Texture](#) * texture,
 const [Vector2](#)< float > & position)

Construct a new Tunnel [Tile](#) object.

Parameters

texture	for graphics component to render the tile.
position	where tile will be rendered in the world.

7.50.2 Member Function Documentation

7.50.2.1 GetType()

TileType TunnelTile::GetType () const [override], [virtual]
 Get the type of the tile. O(1).

Returns

type of the tile.

Reimplemented from [EmptyTile](#).

7.50.2.2 isTunnel()

bool TunnelTile::isTunnel () const [override], [virtual]
 Check if this tile is tunnel.

Returns

true if is tunnel.

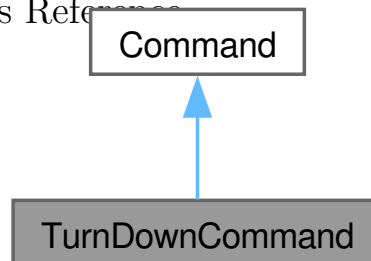
Reimplemented from [Tile](#).

The documentation for this class was generated from the following files:

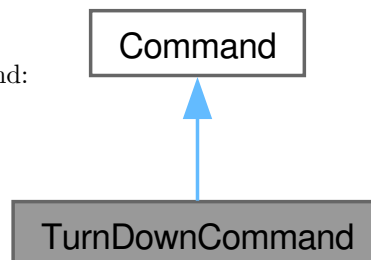
- src/game_objects/TunnelTile.hpp
- src/game_objects/TunnelTile.cpp

7.51 TurnDownCommand Class Reference

Inheritance diagram for TurnDownCommand:



Collaboration diagram for TurnDownCommand:



Public Member Functions

- void [Execute](#) ([GameActor](#) *actor) override
Executes command with give actor.

7.51.1 Member Function Documentation

7.51.1.1 Execute()

void TurnDownCommand::Execute (
[GameActor](#) * actor) [override], [virtual]
 Executes command with give actor.

Parameters

actor	which should execute the command.
-------	-----------------------------------

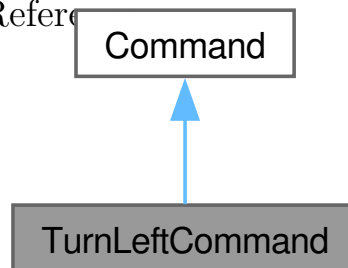
Implements [Command](#).

The documentation for this class was generated from the following files:

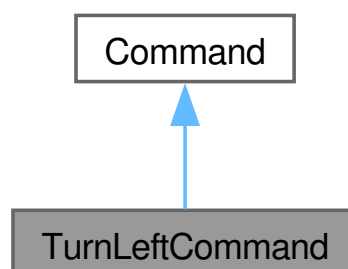
- src/components/Command.hpp
- src/components/Command.cpp

7.52 TurnLeftCommand Class Reference

Inheritance diagram for TurnLeftCommand:



Collaboration diagram for TurnLeftCommand:



Public Member Functions

- void [Execute](#) ([GameActor](#) *actor) override
Executes command with give actor.

7.52.1 Member Function Documentation

7.52.1.1 Execute()

void TurnLeftCommand::Execute (
 [GameActor](#) * actor) [override], [virtual]
 Executes command with give actor.

Parameters

actor	which should execute the command.
-------	-----------------------------------

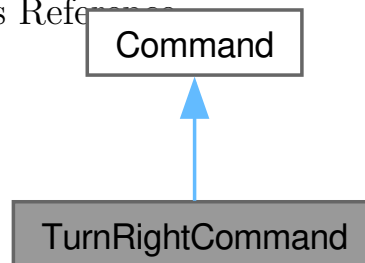
Implements [Command](#).

The documentation for this class was generated from the following files:

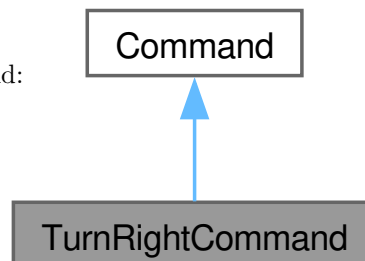
- src/components/Command.hpp
- src/components/Command.cpp

7.53 TurnRightCommand Class Reference

Inheritance diagram for TurnRightCommand:



Collaboration diagram for TurnRightCommand:



Public Member Functions

- void [Execute](#) ([GameActor](#) *actor) override
 Executes command with give actor.

7.53.1 Member Function Documentation

7.53.1.1 Execute()

void TurnRightCommand::Execute (
 [GameActor](#) * actor) [override], [virtual]
 Executes command with give actor.

Parameters

actor	which should execute the command.
-------	-----------------------------------

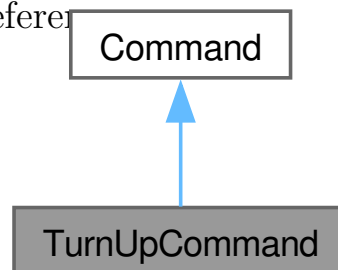
Implements [Command](#).

The documentation for this class was generated from the following files:

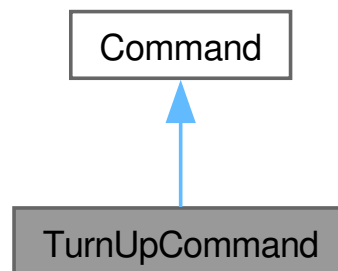
- src/components/Command.hpp
- src/components/Command.cpp

7.54 TurnUpCommand Class Reference

Inheritance diagram for TurnUpCommand:



Collaboration diagram for TurnUpCommand:



Public Member Functions

- void [Execute](#) ([GameActor](#) *actor) override
Executes command with give actor.

7.54.1 Member Function Documentation

7.54.1.1 Execute()

void TurnUpCommand::Execute (
[GameActor](#) * actor) [override], [virtual]
 Executes command with give actor.

Parameters

actor	which should execute the command.
-------	-----------------------------------

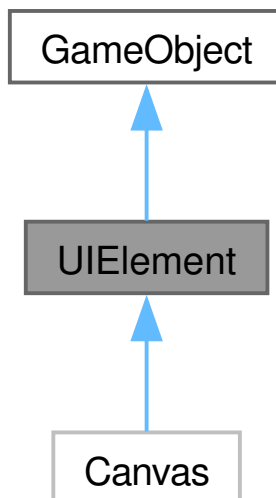
Implements [Command](#).

The documentation for this class was generated from the following files:

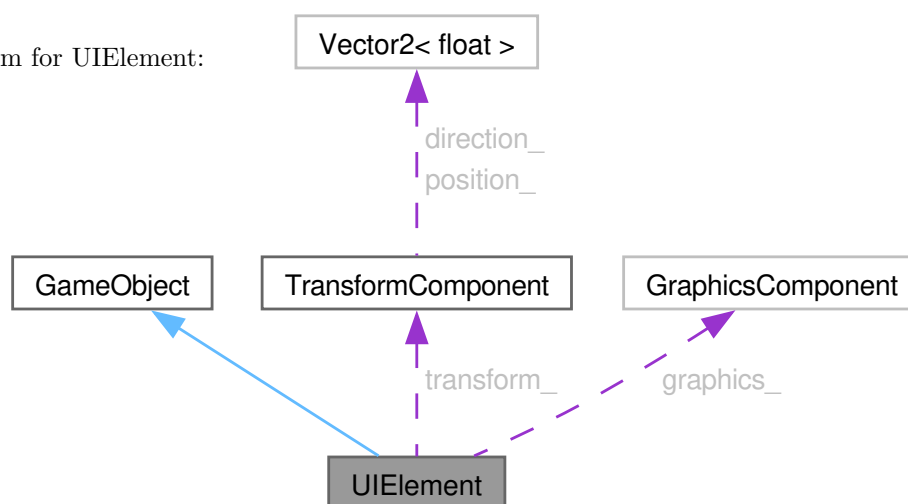
- `src/components/Command.hpp`
- `src/components/Command.cpp`

7.55 UIElement Class Reference

Inheritance diagram for UIElement:



Collaboration diagram for UIElement:



Public Member Functions

- UIElement ([TransformComponent](#) *transform, [GraphicsComponent](#) *graphics)
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- virtual void Render (const [Vector2](#)< float > &position) const
- virtual void Render () const
- [Vector2](#)< float > GetPosition () const
- void ChangeGraphicsTexture (const string &pathOrMessage)

Public Member Functions inherited from [GameObject](#)

- `virtual ~GameObject ()=default`
Default destructor of the game object.

Protected Attributes

- [TransformComponent](#) * transform__
- [GraphicsComponent](#) * graphics__

7.55.1 Member Function Documentation

7.55.1.1 FixedUpdate()

`void UElement::FixedUpdate () [override], [virtual]`

Implements [GameObject](#).

7.55.1.2 Update()

`void UElement::Update () [override], [virtual]`

Updates this game object in the game.

Implements [GameObject](#).

The documentation for this class was generated from the following files:

- `src/UI/UElement.hpp`
- `src/UI/UElement.cpp`

7.56 Vector2< T_ > Struct Template Reference

Public Member Functions

- [Vector2](#) (T_ x=0, T_ y=0)
Construct a new [Vector2](#) object.
- `template<typename U_>`
[Vector2](#) (U_ x=0, U_ y=0)
Construct a new [Vector2](#) object.
- [Vector2](#) (const [Vector2](#) &src)
Copy constructor.
- `template<typename U_>`
[Vector2](#) (const [Vector2](#)< U_ > &src)
Copy constructor from another type. Use `static_cast` to convert other type to this type.
- [Vector2](#) & operator= (const [Vector2](#) &src)
Copy-assignment constructor.
- `template<typename U_>`
[Vector2](#) & operator= (const [Vector2](#)< U_ > &src)
Copy assignment constructor from another type. Use `static_cast` to convert other type to this type.
- [Vector2 ToLeft](#) () const
Turn this vector to 90 degrees to the left.
- [Vector2 ToRight](#) () const
Turn this vector to 90 degrees to the right.
- [Vector2 ToBack](#) () const
Return vector to back direction.
- `float ToDegrees` () const
- `template<typename U_>`
[Vector2](#)< U_ > [ConvertTo](#) () const
Converts this object form one type to another.
- `template<typename U_>`
[Vector2](#)< T_ > & operator= (const [Vector2](#)< U_ > &src)

Static Public Member Functions

- static [Vector2 Zero](#) ()
- static [Vector2 Up](#) ()
- static [Vector2 Down](#) ()
- static [Vector2 Right](#) ()
- static [Vector2 Left](#) ()
- static [Vector2 One](#) ()
- static [Vector2 Abs](#) (const [Vector2](#) &src)
Calculate absolute value of the vector object.
- static [T__ Distance](#) (const [Vector2](#) &a, const [Vector2](#) &b)
Calculates distance between two vectors.

Public Attributes

- [T__ x__](#)
x and y axis in the 2D world
- [T__ y__](#)

Friends

- template<typename U__>
[Vector2](#)< U__ > operator+ (const [Vector2](#)< U__ > &lhs, const [Vector2](#)< U__ > &rhs)
- template<typename U__>
[Vector2](#)< U__ > operator+ (const [Vector2](#)< U__ > &lhs, const U__ &rhs)
- template<typename U__, typename L__>
[Vector2](#)< U__ > operator+ (const [Vector2](#)< U__ > &lhs, const L__ &rhs)
- template<typename U__>
[Vector2](#)< U__ > operator- (const [Vector2](#)< U__ > &src)
- template<typename U__>
[Vector2](#)< U__ > operator- (const [Vector2](#)< U__ > &lhs, const [Vector2](#)< U__ > &rhs)
- template<typename U__>
[Vector2](#)< U__ > operator- (const [Vector2](#)< U__ > &lhs, const U__ &rhs)
- template<typename U__, typename L__>
[Vector2](#)< U__ > operator- (const [Vector2](#)< U__ > &lhs, const L__ &rhs)
- template<typename U__>
[Vector2](#)< U__ > operator* (const [Vector2](#)< U__ > &lhs, const U__ &rhs)
- template<typename U__, typename L__>
[Vector2](#)< U__ > operator* (const [Vector2](#)< U__ > &lhs, const L__ &rhs)
- template<typename U__>
[Vector2](#)< U__ > operator* (const [Vector2](#)< U__ > &lhs, const [Vector2](#)< U__ > &rhs)
- template<typename U__>
[Vector2](#)< U__ > operator/ (const [Vector2](#)< U__ > &lhs, const U__ &rhs)
- template<typename U__, typename L__>
[Vector2](#)< U__ > operator/ (const [Vector2](#)< U__ > &lhs, const L__ &rhs)
- template<typename U__>
[Vector2](#)< U__ > operator% (const [Vector2](#)< U__ > &lhs, int rhs)
- template<typename U__>
bool operator== (const [Vector2](#)< U__ > &lhs, const [Vector2](#)< U__ > &rhs)
- template<typename U__>
bool operator!= (const [Vector2](#)< U__ > &lhs, const [Vector2](#)< U__ > &rhs)
- template<typename U__>
bool operator<= (const [Vector2](#)< U__ > &lhs, const [Vector2](#)< U__ > &rhs)
- template<typename U__>
bool operator>= (const [Vector2](#)< U__ > &lhs, const [Vector2](#)< U__ > &rhs)
- template<typename U__>
bool operator< (const [Vector2](#)< U__ > &lhs, const [Vector2](#)< U__ > &rhs)

- `template<typename U_>
bool operator> (const Vector2< U_ > &lhs, const Vector2< U_ > &rhs)`
- `template<typename U_>
std::ostream & operator<< (std::ostream &os, const Vector2< U_ > &src)`

7.56.1 Constructor & Destructor Documentation

7.56.1.1 Vector2() [1/3]

```
template<typename T_>
Vector2< T_ >::Vector2 (
    T_ x = 0,
    T_ y = 0)
```

Construct a new [Vector2](#) object.

Parameters

x	axis, default is 0.
y	axis, default is 0.

7.56.1.2 Vector2() [2/3]

```
template<typename T_>
template<typename U_>
Vector2< T_ >::Vector2 (
    U_ x = 0,
    U_ y = 0)
```

Construct a new [Vector2](#) object.

Template Parameters

U↔	another type of the parameter.
—	

Parameters

x	axis, default is 0.
y	axis, default is 0.

7.56.1.3 Vector2() [3/3]

```
template<typename T_>
template<typename U_>
Vector2< T_ >::Vector2 (
    const Vector2< U_ > &src)
```

Copy constructor from another type. Use `static_cast` to convert other type to this type.

Template Parameters

U↔	another type of the vector object used to initialize this object.
—	

7.56.2 Member Function Documentation

7.56.2.1 Abs()

```
template<typename T__>
Vector2< T__ > Vector2< T__ >::Abs (
    const Vector2< T__ > & src) [static]
```

Calculate absolute value of the vector object.

Returns

object with absolute values.

7.56.2.2 ConvertTo()

```
template<typename T__>
template<typename U__>
Vector2< U__ > Vector2< T__ >::ConvertTo () const
```

Converts this object form one type to another.

Template Parameters

U↔	type to which to convert.
—	

7.56.2.3 Distance()

```
template<typename T__>
T__ Vector2< T__ >::Distance (
    const Vector2< T__ > & a,
    const Vector2< T__ > & b) [static]
```

Calculates distance between two vectors.

Returns

distance between two vectors.

7.56.2.4 Down()

```
template<typename T__>
Vector2< T__ > Vector2< T__ >::Down () [static]
```

Returns

Vector2 object with values {0, 1}.

7.56.2.5 Left()

```
template<typename T__>
Vector2< T__ > Vector2< T__ >::Left () [static]
```

Returns

Vector2 object with values {-1, 1}.

7.56.2.6 One()

```
template<typename T__>
Vector2< T__ > Vector2< T__ >::One () [static]
```

Returns

Vector2 object with values {1, 1}.

7.56.2.7 operator=()

```
template<typename T_>
```

```
template<typename U_>
```

```
Vector2 & Vector2< T_ >::operator= (
    const Vector2< U_ > & src)
```

Copy assignment constructor from another type. Use `static_cast` to convert other type to this type.

Template Parameters

U↔	another type of the vector object used to assign this object.
—	

7.56.2.8 Right()

```
template<typename T_>
```

```
Vector2< T_ > Vector2< T_ >::Right () [static]
```

Returns

Vector2 object with values {1, 0}.

7.56.2.9 ToBack()

```
template<typename T_>
```

```
Vector2< T_ > Vector2< T_ >::ToBack () const
```

Return vector to back direction.

Returns

vector in opposite direction to current vector.

7.56.2.10 ToLeft()

```
template<typename T_>
```

```
Vector2< T_ > Vector2< T_ >::ToLeft () const
```

Turn this vector to 90 degrees to the left.

Returns

Vector turned to 90 degrees to the left.

7.56.2.11 ToRight()

```
template<typename T_>
```

```
Vector2< T_ > Vector2< T_ >::ToRight () const
```

Turn this vector to 90 degrees to the right.

Returns

Vector turned to 90 degrees to the right.

7.56.2.12 Up()

```
template<typename T_>
```

```
Vector2< T_ > Vector2< T_ >::Up () [static]
```

Returns

Vector2 object with values {0, -1}.

7.56.2.13 Zero()

```
template<typename T_>  
Vector2< T_ > Vector2< T_ >::Zero () [static]
```

Returns

Vector2 object with values {0, 0}.

The documentation for this struct was generated from the following files:

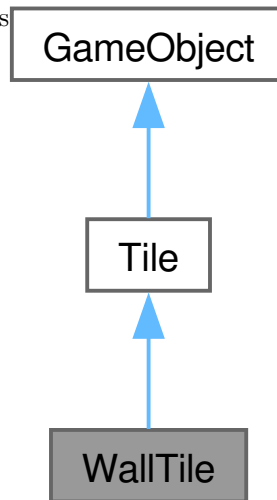
- src/structures/Vector2.hpp
- src/structures/Vector2.inl

7.57 WallTile Class Reference

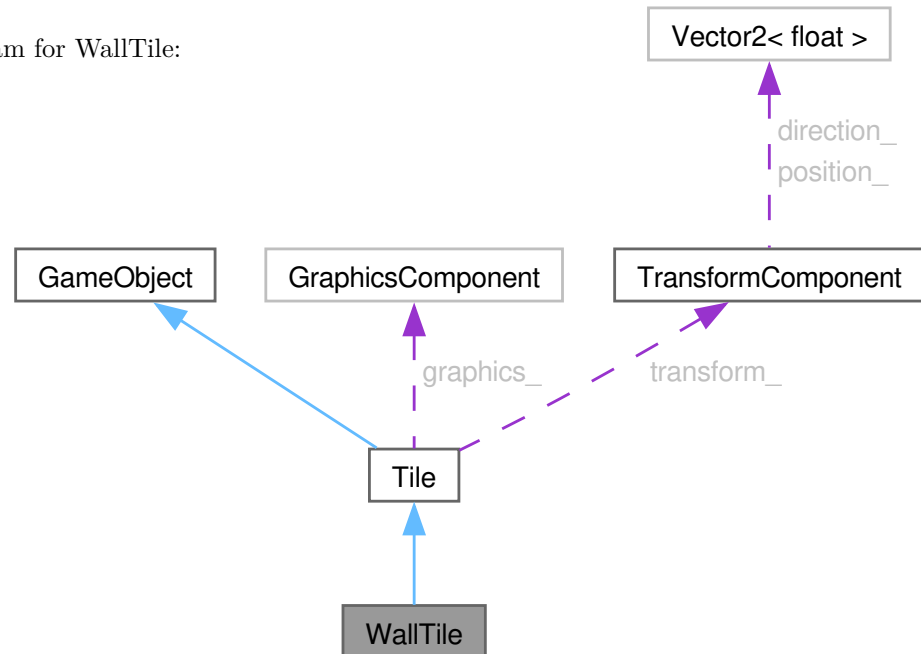
Class that represents Wall tile subclass of the [Tile](#) s

```
#include <WallTile.hpp>
```

Inheritance diagram for WallTile:



Collaboration diagram for WallTile:



Public Member Functions

- [WallTile](#) ([Texture](#) *texture, const [Vector2](#)< float > &position)
Construct a new Wall [Tile](#) object.
- [WallTile](#) ([Texture](#) *texture, [TransformComponent](#) *transform)
- void [Render](#) () const override
Render the tile on the game map. O(1).
- bool [isObstacle](#) () const override
Check if this tile is obstacle or not. O(1).
- [TileType](#) [GetType](#) () const override
Get the type of the tile. O(1).

Public Member Functions inherited from [Tile](#)

- [Tile](#) ([Texture](#) *texture, const [Vector2](#)< float > &position)
Construct a new [Tile](#) object. O(1).
- [Tile](#) ([Texture](#) *texture, [TransformComponent](#) *transform)
- ~[Tile](#) () override
Destroys the [Tile](#) object. O(1).
- void [Update](#) () override
Updates this game object in the game.
- void [FixedUpdate](#) () override
- virtual bool [isTunnel](#) () const
Check if this tile is tunnel.
- virtual bool [isPortal](#) () const
Check if this tile is portal.
- virtual void [Teleport](#) ([GameActor](#) &gameActor, [TileNode](#) &portalNode)
Moves game actor transform and its current tile to another connected portalNode.
- virtual bool [CanTurnAside](#) (const [GameActor](#) &gameActor) const
Check if game object standing on this tile can turn to the side, right or left or not. O(1).

- virtual bool [AddBonus](#) ([Bonus](#) *bonus)
Add bonus to tile.
- virtual bool [CollectBonus](#) ()
Collect bonus from tile.
- [TransformComponent](#) * GetTransform ()
Get Transform component of the tile. O(1).
- const [TransformComponent](#) * GetTransform () const
Get Transform component of the tile. O(1).

Public Member Functions inherited from [GameObject](#)

- virtual ~GameObject ()=default
Default destructor of the game object.

Additional Inherited Members

Static Public Member Functions inherited from [Tile](#)

- static [Vector2](#)< float > [GetOffsetFromTileCenter](#) (const [GameActor](#) &gameActor, const [Tile](#) *tile)
Get distance between game object and its tile center according to the current direction. O(1).

Protected Attributes inherited from [Tile](#)

- [GraphicsComponent](#) * graphics_
Graphics component of the tile class.
- [TransformComponent](#) * transform_
Transform component of the tile class.

7.57.1 Detailed Description

Class that represents Wall tile subclass of the [Tile](#) superclass.

7.57.2 Constructor & Destructor Documentation

7.57.2.1 WallTile()

WallTile::WallTile (
 [Texture](#) * texture,
 const [Vector2](#)< float > & position)

Construct a new Wall [Tile](#) object.

Parameters

texture	for graphics component to render the tile.
position	where tile should be rendered.

7.57.3 Member Function Documentation

7.57.3.1 GetType()

TileType WallTile::GetType () const [nodiscard], [override], [virtual]
Get the type of the tile. O(1).

Returns

type of the tile.

Implements [Tile](#).

7.57.3.2 isObstacle()

bool WallTile::isObstacle () const [nodiscard], [override], [virtual]

Check if this tile is obstacle or not. $O(1)$.

Returns

true if tile is obstacle.

Reimplemented from [Tile](#).

7.57.3.3 Render()

void WallTile::Render () const [override], [virtual]

Render the tile on the game map. $O(1)$.

Reimplemented from [Tile](#).

The documentation for this class was generated from the following files:

- src/game_objects/WallTile.hpp
- src/game_objects/WallTile.cpp

Chapter 8

File Documentation

8.1 AIComponent.hpp

```
00001 #pragma once
00002
00003 #include "../ControllerComponent.hpp"
00004 #include "../AICommands.hpp"
00005
00009 class AIComponent : public ControllerComponent {
00010
00011 public:
00012
00018     AIComponent(GameActor *controlled, GameActor *player);
00019
00023     ~AIComponent() override;
00024
00025     void Update() override;
00026
00027     void FixedUpdate() override;
00028
00029     void ResetControlled() override;
00030
00031 private:
00032
00034     Command *moveActor_;
00035
00037     TargetCommand *chasePlayer_;
00039     TargetCommand *catchPlayer_;
00040
00042     GameActor *player_;
00043
00044 };
```

8.2 Blinky.hpp

```
00001
```

8.3 Clyde.hpp

```
00001
```

8.4 Inky.hpp

```
00001
```

8.5 Pinky.hpp

```
00001
```

8.6 AICommands.hpp

```

00001 #include "Command.hpp"
00002 #include "../game_objects/Tile.hpp"
00003
00007 class TargetCommand {
00008 public:
00009     virtual ~TargetCommand() = default;
00010
00016     virtual void Execute(GameActor *controlled, GameActor *target) = 0;
00017 };
00018
00019
00023 class ChasePlayerCommand : public TargetCommand {
00024 public:
00025     void Execute(GameActor *controlled, GameActor *target) override;
00026
00027 protected:
00028
00032     void CheckTile();
00033
00038     Vector2<float> DecideDirection();
00039
00045     void CheckDirection(const Vector2<float> &direction, pair<Vector2<float>, float> &decision) const;
00046
00048     GameActor *player_;
00050     GameActor *controlled_;
00052     const TileNode *decidedNode_;
00053
00054 };
00055
00059 class CatchPlayerCommand : public TargetCommand {
00060 public:
00061     void Execute(GameActor *controlled, GameActor *target) override;
00062
00063 };

```

8.7 AnimationComponent.hpp

```

00001

```

8.8 Command.hpp

```

00001 #pragma once
00002
00003 #include "../game_objects/GameActor.hpp"
00004
00008 class Command {
00009 public:
00010     virtual ~Command() = default;
00011
00016     virtual void Execute(GameActor *actor) = 0;
00017 };
00018
00019 class TurnUpCommand : public Command {
00020 public:
00021     void Execute(GameActor *actor) override;
00022 };
00023
00024 class TurnDownCommand : public Command {
00025 public:
00026     void Execute(GameActor *actor) override;
00027 };
00028
00029 class TurnRightCommand : public Command {
00030 public:
00031     void Execute(GameActor *actor) override;
00032 };
00033
00034 class TurnLeftCommand : public Command {
00035 public:
00036     void Execute(GameActor *actor) override;
00037 };
00038
00039 class StopMoveCommand : public Command {
00040 public:
00041     void Execute(GameActor *actor) override;
00042 };
00043
00044 class MoveCommand : public Command {
00045 public:

```

```
00046     void Execute(GameActor *actor) override;
00047 };
```

8.9 ControllerComponent.hpp

```
00001 #pragma once
00002
00003 #include "../game_objects/GameActor.hpp"
00004 #include "Command.hpp"
00005
00009 class ControllerComponent {
00010
00011 public:
00012
00017     explicit ControllerComponent(GameActor *actor);
00018
00023     virtual ~ControllerComponent() = default;
00024
00028     virtual void Update() = 0;
00029
00030     virtual void FixedUpdate() = 0;
00031
00035     virtual void ResetControlled() = 0;
00036
00037 protected:
00038
00040     GameActor *controlled_;
00041
00042 };
```

8.10 GraphicsComponent.hpp

```
00001 #pragma once
00002
00003 #include "../structures/Texture.hpp"
00004
00005 class GraphicsComponent {
00006
00007 public:
00008
00013     explicit GraphicsComponent(Texture *texture);
00014
00018     virtual ~GraphicsComponent() = default;
00019
00020     void ChangeGraphicsTexture(const string &pathOrMessage);
00021
00026     virtual void Render(const Vector2<float> &position) const;
00027
00028     virtual void Render(const Vector2<float> &position, double angle) const;
00029
00034     inline const TextureSize<float> &GetTextureSize() const;
00035
00036 private:
00037
00039     Texture *texture_;
00040
00041 };
00042
00043 inline const TextureSize<float> &GraphicsComponent::GetTextureSize() const {
00044     return texture_>GetSize();
00045 }
```

8.11 PhysicsComponent.hpp

```
00001 #pragma once
00002
00003 #include "../structures/Vector2.hpp"
00004
00005 class TileNode;
00006
00007 class GameActor;
00008
00014 class PhysicsComponent {
00015
00016 public:
00017
00022     explicit PhysicsComponent(GameActor &gameActor);
00023
00030     [[nodiscard]]
```

```

00031     bool TouchesObstacle(const Vector2<float> &direction) const;
00032
00033     [[nodiscard]]
00034     bool isOnNextTile() const;
00035
00036     void Move(Vector2<float> newPosition);
00037
00038     TileNode *currentNode_;
00039
00040 private:
00041
00042     void ResolveOutOfScreenBounds(Vector2<float> &newPosition);
00043
00044     void ResolveCollisions(Vector2<float> &newPosition);
00045
00046     void ResolveObstacles(const Vector2<float> &oldPosition, Vector2<float> &newPosition);
00047
00048     GameActor &gameActor_;
00049     bool resolvedObstaclesCollision;
00050 };

```

8.12 PlayerController.hpp

```

00001 #pragma once
00002
00003 #include "ControllerComponent.hpp"
00004
00005 class PlayerController : public ControllerComponent {
00006 public:
00007
00008     explicit PlayerController(GameActor *actor);
00009
00010     ~PlayerController() override;
00011
00012     void Update() override;
00013
00014     void FixedUpdate() override;
00015
00016     void ResetControlled() override;
00017
00018 protected:
00019
00020     Command *buttonUp_;
00021     Command *buttonDown_;
00022     Command *buttonLeft_;
00023     Command *buttonRight_;
00024     Command *buttonEscape_;
00025
00026     Command *moveActor_;
00027 };

```

8.13 TransformComponent.hpp

```

00001 #pragma once
00002
00003 #include "../structures/Vector2.hpp"
00004
00005 class GameObject;
00006
00007 class TransformComponent {
00008 public:
00009
00010     TransformComponent();
00011
00012     TransformComponent(const Vector2<float> &initialPosition, const Vector2<float> &direction);
00013
00014     TransformComponent(const Vector2<float> &initialPosition);
00015
00016     void Translate(const Vector2<float> &newPosition);
00017
00018     Vector2<float> direction_;
00019
00020     Vector2<float> position_;
00021 };

```

8.14 Game.hpp

```

00001 #pragma once
00002
00003 #include <vector>
00004 #include <iostream>
00005
00006 using namespace std;
00007
00008 #include "../components/ControllerComponent.hpp"
00009 #include "../components/PlayerController.hpp"
00010 #include "../components/AI_components/AIComponent.hpp"
00011 #include "../game_objects/GameObject.hpp"
00012 #include "../game_objects/Pacman.hpp"
00013 #include "../game_objects/Ghost.hpp"
00014 #include "../game_objects/Controller.hpp"
00015 #include "GameGraphics.hpp"
00016 #include "GameTime.hpp"
00017 #include "GameEvents.hpp"
00018 #include "../UI/Canvas.hpp"
00019
00020
00025 class Game {
00026
00027 public:
00028
00032     Game() = default;
00033
00038     ~Game();
00039
00043     void GameStart();
00044
00048     void QuitGame();
00049
00051     vector<Controller *> gameControllers_;
00052
00053 private:
00054
00059     void GameInit();
00060
00064     void InitGameLocator();
00065
00069     void InitGameObjects();
00070
00075     void GameLoop();
00076
00080     void RenderGame();
00081
00085     void RenderGameObjects();
00086
00090     void UpdateGame();
00091
00092     void FixedUpdateGame();
00093
00097     void UpdateGameObjects();
00098
00099     void FixedUpdateGameObject();
00100
00104     void FreeGameObjects();
00105
00107     GameEvents events_;
00109     vector<GameActor *> gameActors_;
00110     vector<UIElement *> UIElements_;
00111 };

```

8.15 GameAudio.hpp

```

00001 #pragma once
00002
00003 class GameAudio {
00004
00005 };

```

8.16 GameEvents.hpp

```

00001 #include "SDL2/SDL.h"
00002
00006 class GameEvents {
00007
00008 public:
00009

```

```

00013     GameEvents();
00014
00018     void ProcessInput();
00019
00023     void QuitGame();
00024
00030     inline bool isQuit() const;
00031
00036     inline bool isQuitEvent() const;
00037
00038 private:
00039
00041     bool quit_;
00043     SDL_Event event_;
00044 };
00045
00046 inline
00047 bool GameEvents::isQuit() const {
00048     return quit_;
00049 }
00050
00051 inline
00052 bool GameEvents::isQuitEvent() const {
00053     return event_.type == SDL_QUIT;
00054 }

```

8.17 GameFilesystem.hpp

```

00001 #pragma once
00002
00003 #include <unordered_map>
00004
00005 #include "../structures/Texture.hpp"
00006 #include "../structures/NullTexture.hpp"
00007 #include "../structures/TextureSize.hpp"
00008 #include "../game_objects/Tile.hpp"
00009
00011 enum class TextureType {
00012     PACMAN, BLINKY_GHOST,
00013     NULL_TEXTURE, EMPTY, CROSS, TUNNEL,
00014     WALL_STRAIGHT, WALL_CORNER_OUTER, WALL_CORNER_INNER,
00015     BONUS
00016 };
00017
00019 namespace TextureSizes {
00020
00021     const TextureSize<float> ACTOR(40, 40);
00022     const TextureSize<float> TILE(22, 22);
00023     const TextureSize<float> BONUS(5, 5);
00024 }
00025
00031 class GameFilesystem {
00032
00033 public:
00034
00038     GameFilesystem();
00039
00040     ~GameFilesystem();
00041
00043     inline Texture *GetTexture(TextureType type);
00044
00045 private:
00046
00050     void InitTextures();
00051
00057     void LoadTexture(TextureType type, const string &path);
00058
00059     void LoadTexture(TextureType type, Texture *texture);
00060
00067     void LoadTexture(TextureType type, const string &path, const TextureSize<float> &size);
00068
00070     unordered_map<TextureType, Texture *> textures_;
00071
00072 };
00073
00074 inline Texture *GameFilesystem::GetTexture(TextureType type) {
00075     return textures_[type];
00076 }
00077

```

8.18 GameGraphics.hpp

```

00001 #pragma once
00002
00003 #include <SDL2/SDL.h>
00004 #include <SDL2/SDL_image.h>
00005 #include <SDL2/SDL_ttf.h>
00006 #include <iostream>
00007
00008 constexpr size_t SCREEN_HEIGHT = 1080;
00009 constexpr size_t SCREEN_WIDTH = 1080;
00010
00015 class GameGraphics {
00016
00017 public:
00018
00023     GameGraphics();
00024
00028     ~GameGraphics();
00029
00033     void ClearRender();
00034
00038     void RenderGame();
00039
00041     inline SDL_Renderer *GetRenderer();
00042
00043 private:
00044
00049     void InitGameGraphics();
00050
00055     static bool InitSDL();
00056
00061     static bool InitIMG();
00062
00067     static void SetSDLSettings();
00068
00073     bool CreateGameGraphics();
00074
00079     bool CreateGameWindow();
00080
00085     bool CreateGameRenderer();
00086
00090     void QuitGameGraphics();
00091
00095     void SetScreenBackgroundWhite();
00096
00100     void SetScreenBackgroundBlack();
00101
00103     SDL_Window *window_;
00105     SDL_Renderer *renderer_;
00106
00107 };
00108
00109 inline SDL_Renderer *GameGraphics::GetRenderer() {
00110     return renderer_;
00111 }

```

8.19 GameLocator.hpp

```

00001 #pragma once
00002
00003 #include "GameTime.hpp"
00004 #include "GameMap.hpp"
00005 #include "GameFilesystem.hpp"
00006 #include "GameAudio.hpp"
00007 #include "GameManager.hpp"
00008 #include "GameGraphics.hpp"
00009
00017 class GameLocator {
00018
00019 public:
00020
00024     static void InitLocator();
00025
00029     static void QuitLocator();
00030
00035     static inline void ProvideTime(GameTime *time);
00036
00041     static inline void ProvideMap(GameMap *map);
00042
00047     static inline void ProvideFilesystem(GameFilesystem *filesystem);
00048
00053     static inline void ProvideAudio(GameAudio *audio);
00054

```

```

00060     static inline void ProvideManager(GameManager *manager);
00061
00066     static inline void ProvideGraphics(GameGraphics *graphics);
00067
00068
00074     static inline GameTime &GetTime();
00075
00080     static inline GameMap &GetMap();
00081
00086     static inline GameFilesystem &GetFilesystem();
00087
00089     static inline GameAudio &GetAudio();
00090
00092     static inline GameManager &GetManager();
00093
00095     static inline GameGraphics &GetGraphics();
00096
00098     static inline SDL_Renderer *GetRenderer();
00099
00100 private:
00102     static GameTime *time_;
00104     static GameMap *map_;
00107     static GameFilesystem *filesystem_;
00109     static GameAudio *audio_;
00112     static GameManager *manager_;
00115     static GameGraphics *graphics_;
00116 };
00117
00118
00119 inline
00120 void GameLocator::ProvideTime(GameTime *time) {
00121     time_ = time;
00122 }
00123
00124 inline
00125 void GameLocator::ProvideAudio(GameAudio *audio) {
00126     audio_ = audio;
00127 }
00128
00129 inline
00130 void GameLocator::ProvideFilesystem(GameFilesystem *filesystem) {
00131     filesystem_ = filesystem;
00132 }
00133
00134 inline
00135 void GameLocator::ProvideGraphics(GameGraphics *graphics) {
00136     graphics_ = graphics;
00137 }
00138
00139 inline
00140 void GameLocator::ProvideManager(GameManager *manager) {
00141     manager_ = manager;
00142 }
00143
00144 inline
00145 void GameLocator::ProvideMap(GameMap *map) {
00146     map_ = map;
00147 }
00148
00149 inline
00150 GameTime &GameLocator::GetTime() {
00151     return *time_;
00152 }
00153
00154 inline
00155 GameAudio &GameLocator::GetAudio() {
00156     return *audio_;
00157 }
00158
00159 inline
00160 GameFilesystem &GameLocator::GetFilesystem() {
00161     return *filesystem_;
00162 }
00163
00164 inline
00165 GameGraphics &GameLocator::GetGraphics() {
00166     return *graphics_;
00167 }
00168
00169 inline
00170 GameManager &GameLocator::GetManager() {
00171     return *manager_;
00172 }
00173
00174 inline
00175 GameMap &GameLocator::GetMap() {
00176     return *map_;

```



```

00177 }
00178
00179 inline
00180 SDL_Renderer *GameLocator::GetRenderer() {
00181     return graphics_>GetRenderer();
00182 }

```

8.20 GameManager.hpp

```

00001 #pragma once
00002
00003 #include "../game_states/GameState.hpp"
00004 #include "../game_states/GameStateNormal.hpp"
00005 #include "../game_states/GameStateReset.hpp"
00006
00007 class Game;
00008
00012 class GameManager {
00013
00014 public:
00015
00020     explicit GameManager(Game &game);
00021
00025     void Update();
00026
00027     void FixedUpdate();
00028
00032     void ReportPlayerCollision();
00033
00037     static void ReportBonusCollection();
00038
00040     static constexpr float GAME_SPEED = 200.0f;
00041
00043     GameState *state_;
00044
00046     Game &game_;
00047
00048 };

```

8.21 GameMap.hpp

```

00001 #pragma once
00002
00003 #include <map>
00004 #include <algorithm>
00005 #include <string>
00006 #include <cmath>
00007
00008 #include "../structures/TileNodeBuilder.hpp"
00009 #include "../structures/MapParser.hpp"
00010 #include "../structures/MapReader.hpp"
00011 #include "../game_objects/NullTile.hpp"
00012 #include "../structures/TileNode.hpp"
00013
00014 constexpr int TILE_SIZE = 20;
00015
00019 class GameMap {
00020
00021 public:
00022
00027     explicit GameMap(const string &path);
00028
00032     ~GameMap();
00033
00037     void RenderMap() const;
00038
00043     void LoadMap(const string &path);
00044
00048     void ClearMap();
00049
00055     static Vector2<int> CentralizePosition(const Vector2<int> &position);
00056
00062     static Vector2<int> GetPixelsPosition(const Vector2<int> &position);
00063
00065     inline MapInfo GetMapInfo();
00066
00067 private:
00068
00072     void LoadRenderMap();
00073
00075     vector<vector<TileNode *>> tileNodesArray2D_;

```

```

00076
00078     vector<Tile *> renderTilesArray_;
00079
00081     MapInfo mapInfo_;
00082
00083 };
00084
00085 inline MapInfo GameMap::GetMapInfo() {
00086     return mapInfo_;
00087 }

```

8.22 GameTime.hpp

```

00001 #pragma once
00002
00003 #include <SDL2/SDL.h>
00004
00005 using namespace std;
00006
00010 class GameTime {
00011
00012 public:
00013
00017     GameTime();
00018
00023     void InitGameTicks();
00024
00029     void UpdateCurrentTicks();
00030
00035     void UpdateLastTicks();
00036
00038     inline uint GetDeltaTicks() const;
00039
00041     inline float GetDeltaTime() const;
00042
00043     static constexpr uint MS_PER_UPDATE = 16;
00044     static constexpr float FMS_PER_UPDATE = 0.016f;
00045
00046 private:
00047
00049     uint64_t currentTicks_;
00051     uint64_t lastTicks_;
00053     uint deltaTicks_;
00054
00055 };
00056
00057
00058 inline
00059 uint GameTime::GetDeltaTicks() const {
00060     return deltaTicks_;
00061 }
00062
00063 inline
00064 float GameTime::GetDeltaTime() const {
00065     return static_cast<float>(deltaTicks_) / 1000;
00066 }

```

8.23 Bonus.hpp

```

00001 #pragma once
00002
00003 #include "GameObject.hpp"
00004
00008 class Bonus : public GameObject {
00009
00010 public:
00011
00016     explicit Bonus(GraphicsComponent *graphics);
00017
00018     ~Bonus() override;
00019
00020     void Update() override;
00021
00022     void FixedUpdate() override;
00023
00028     void Render(const Vector2<float> &position);
00029
00030 private:
00031
00033     GraphicsComponent *graphics_;
00034
00035 };

```

8.24 Controller.hpp

```

00001 #pragma once
00002
00003 #include "GameObject.hpp"
00004 #include "../components/ControllerComponent.hpp"
00005
00010 class Controller : public GameObject {
00011
00012 public:
00013
00014     enum State {
00015         ENABLED,
00016         DISABLED
00017     };
00018
00023     explicit Controller(ControllerComponent *controller);
00024
00028     ~Controller() override;
00029
00030     void Update() override;
00031
00032     void FixedUpdate() override;
00033
00035     inline ControllerComponent *GetController();
00036
00038     inline void SetState(int value);
00039
00040 private:
00041
00043     ControllerComponent *controller_;
00044
00046     int state_;
00047
00048 };
00049
00050 inline ControllerComponent *Controller::GetController() {
00051     return controller_;
00052 }
00053
00054 inline void Controller::SetState(int value) {
00055     if (value <= DISABLED)
00056         state_ = value;
00057 }

```

8.25 CrossTile.hpp

```

00001 #pragma once
00002
00003 #include "Tile.hpp"
00004 #include "EmptyTile.hpp"
00005
00010 class CrossTile : public EmptyTile {
00011
00012 public:
00013
00019     CrossTile(Texture *texture, const Vector2<float> &position);
00020
00021     void Render() const override;
00022
00023     [[nodiscard]]
00024     bool CanTurnAside(const GameActor &gameActor) const override;
00025
00026     [[nodiscard]]
00027     TileType GetType() const override;
00028
00029 private:
00030
00037     [[nodiscard]]
00038     bool isInThreshold(const GameActor &obj) const;
00039
00041     static constexpr int IN_THRESHOLD = 5;
00043     static constexpr int OUT_THRESHOLD = 2;
00044 };

```

8.26 EmptyTile.hpp

```

00001 #pragma once
00002
00003 #include "Tile.hpp"
00004

```

```

00009 class EmptyTile : public Tile {
00010
00011 public:
00012
00018     EmptyTile(Texture *texture, const Vector2<float> &position);
00019
00020     ~EmptyTile() override;
00021
00022     void Render() const override;
00023
00024     bool AddBonus(Bonus *bonus) override;
00025
00026     bool CollectBonus() override;
00027
00028     [[nodiscard]]
00029     TileType GetType() const override;
00030
00031     [[nodiscard]]
00032     bool CanTurnAside(const GameActor &gameActor) const override;
00033
00034 protected:
00035
00037     Bonus *bonus_;
00038
00039 };

```

8.27 GameActor.hpp

```

00001 #pragma once
00002
00003 #include "GameObject.hpp"
00004 #include "../components/PhysicsComponent.hpp"
00005
00007 enum State {
00008     STANDING,
00009     MOVING
00010 };
00011
00019 class GameActor : public GameObject {
00020
00021 public:
00022
00029     GameActor(TransformComponent *transform, GraphicsComponent *graphics, GameActor &actor);
00030
00034     ~GameActor() override;
00035
00039     virtual void Render() const = 0;
00040
00044     virtual void TurnUp() = 0;
00045
00049     virtual void TurnDown() = 0;
00050
00054     virtual void TurnRight() = 0;
00055
00059     virtual void TurnLeft() = 0;
00060
00064     virtual void StopMoving() = 0;
00065
00069     virtual void Move() = 0;
00070
00075     virtual void TurnTo(const Vector2<float> &direction) = 0;
00076
00078     inline const GraphicsComponent *GetGraphics() const;
00079
00081     inline PhysicsComponent *GetPhysics();
00082
00083     inline const PhysicsComponent *GetPhysics() const;
00084
00086     inline TransformComponent *GetTransform();
00087
00088     inline const TransformComponent *GetTransform() const;
00089
00091     inline float GetVelocity() const;
00092
00094     inline void SetVelocity(float value);
00095
00097     inline int GetState() const;
00098
00100     inline void SetState(int value);
00101
00102 protected:
00103
00105     TransformComponent *transform_;
00107     GraphicsComponent *graphics_;

```

```

00109     PhysicsComponent *physics_;
00110
00111     float velocity_;
00112
00113     int state_;
00114
00115 };
00116
00117 inline TransformComponent *GameActor::GetTransform() {
00118     return transform_;
00119 }
00120
00121 inline const TransformComponent *GameActor::GetTransform() const {
00122     return transform_;
00123 }
00124
00125 inline const GraphicsComponent *GameActor::GetGraphics() const {
00126     return graphics_;
00127 }
00128
00129 inline const PhysicsComponent *GameActor::GetPhysics() const {
00130     return physics_;
00131 }
00132
00133 inline PhysicsComponent *GameActor::GetPhysics() {
00134     return physics_;
00135 }
00136
00137 inline float GameActor::GetVelocity() const {
00138     return velocity_;
00139 }
00140
00141 inline void GameActor::SetVelocity(float value) {
00142     velocity_ = value;
00143 }
00144
00145 inline int GameActor::GetState() const {
00146     return state_;
00147 }
00148
00149 inline void GameActor::SetState(int value) {
00150     if (value <= MOVING)
00151         state_ = value;
00152 }
00153
00154 }

```

8.28 GameObject.hpp

```

00001 #pragma once
00002
00003 #include "../components/GraphicsComponent.hpp"
00004 #include "../components/TransformComponent.hpp"
00005
00006 class GameObject {
00007 public:
00008     virtual ~GameObject() = default;
00009
00010     virtual void Update() = 0;
00011
00012     virtual void FixedUpdate() = 0;
00013 };

```

8.29 Ghost.hpp

```

00001 #pragma once
00002
00003 #include "GameActor.hpp"
00004
00005 class Ghost : public GameActor {
00006 public:
00007     Ghost(TransformComponent *transform, GraphicsComponent *graphics);
00008
00009     void TurnUp() override;
00010
00011     void TurnDown() override;
00012
00013     void TurnRight() override;
00014 }

```

```

00026     void TurnLeft() override;
00027
00028     void StopMoving() override;
00029
00030     void Move() override;
00031
00032     void TurnTo(const Vector2<float> &direction) override;
00033
00034     void Update() override;
00035
00036     void FixedUpdate() override;
00037
00038     void Render() const override;
00039
00040 private:
00041
00042     void CheckTunnel();
00043
00044 };

```

8.30 NullTile.hpp

```

00001 #pragma once
00002
00003 #include "Tile.hpp"
00004
00009 class NullTile : public Tile {
00010
00011 public:
00012
00017     NullTile(const Vector2<float> &position);
00018
00019     void Render() const override;
00020
00021     [[nodiscard]]
00022     TileType GetType() const override;
00023
00024     [[nodiscard]]
00025     bool isObstacle() const override;
00026
00027 };

```

8.31 Pacman.hpp

```

00001 #pragma once
00002
00003 #include "GameActor.hpp"
00004
00009 class Pacman : public GameActor {
00010
00011 public:
00012
00018     Pacman(TransformComponent *transform, GraphicsComponent *graphics);
00019
00020     void TurnUp() override;
00021
00022     void TurnDown() override;
00023
00024     void TurnRight() override;
00025
00026     void TurnLeft() override;
00027
00028     void StopMoving() override;
00029
00030     void Move() override;
00031
00032     void TurnTo(const Vector2<float> &direction) override;
00033
00034     void Update() override;
00035
00036     void FixedUpdate() override;
00037
00038     void Render() const override;
00039
00045     void TryChangeDirection(const Vector2<float> &newDirection);
00046
00050     void CollectBonus();
00051
00052 };

```

8.32 PortalTile.hpp

```

00001 #pragma once
00002
00003 #include "TunnelTile.hpp"
00004
00005 class PortalTile : public TunnelTile {
00006 public:
00007
00014     PortalTile(Texture *texture, const Vector2<float> &position, const Vector2<float> &direction);
00015
00016     TileType GetType() const override;
00017
00018     bool isPortal() const override;
00019
00020     void Teleport(GameActor &gameActor, TileNode &portal) override;
00021 };

```

8.33 Tile.hpp

```

00001 #pragma once
00002
00003 #include "../components/GraphicsComponent.hpp"
00004 #include "../components/TransformComponent.hpp"
00005 #include "GameActor.hpp"
00006 #include "Bonus.hpp"
00007
00009 enum class TileType {
00010     NULL_TILE, EMPTY, WALL, CROSS, TUNNEL, PORTAL
00011 };
00012 };
00013
00015 enum class TileOption {
00016     NONE, BONUS, BIG_BONUS, CHERRY,
00017     INNER_WALL_CORNER, OUTER_WALL_CORNER
00018 };
00019
00024 class Tile : public GameObject {
00025 public:
00026
00033     Tile(Texture *texture, const Vector2<float> &position);
00034
00035     Tile(Texture *texture, TransformComponent *transform);
00036
00040     ~Tile() override;
00041
00046     [[nodiscard]]
00047     virtual TileType GetType() const = 0;
00048
00052     virtual void Render() const;
00053
00054     void Update() override;
00055
00056     void FixedUpdate() override;
00057
00062     [[nodiscard]]
00063     virtual bool isObstacle() const;
00064
00069     [[nodiscard]]
00070     virtual bool isTunnel() const;
00071
00076     [[nodiscard]]
00077     virtual bool isPortal() const;
00078
00084     virtual void Teleport(GameActor &gameActor, TileNode &portalNode);
00085
00091     [[nodiscard]]
00092     virtual bool CanTurnAside(const GameActor &gameActor) const;
00093
00099     virtual bool AddBonus(Bonus *bonus);
00100
00105     virtual bool CollectBonus();
00106
00111     [[nodiscard]]
00112     static Vector2<float> GetOffsetFromTileCenter(const GameActor &gameActor, const Tile *tile);
00113
00115     [[nodiscard]]
00116     inline TransformComponent *GetTransform();
00117
00119     [[nodiscard]]
00120     inline const TransformComponent *GetTransform() const;
00121
00122 protected:

```

```

00123
00125     GraphicsComponent *graphics_;
00127     TransformComponent *transform_;
00128 };
00129
00130 inline TransformComponent *Tile::GetTransform() {
00131     return transform_;
00132 }
00133
00134 inline const TransformComponent *Tile::GetTransform() const {
00135     return transform_;
00136 }

```

8.34 TunnelTile.hpp

```

00001 #pragma once
00002
00003 #include "EmptyTile.hpp"
00004
00005 class TunnelTile : public EmptyTile {
00006 public:
00012     TunnelTile(Texture *texture, const Vector2<float> &position);
00013
00014     TileType GetType() const override;
00015
00016     bool isTunnel() const override;
00017
00018 };
00019

```

8.35 WallTile.hpp

```

00001 #pragma once
00002
00003 #include "Tile.hpp"
00004
00005 class WallTile : public Tile {
00010
00011 public:
00012
00018     WallTile(Texture *texture, const Vector2<float> &position);
00019
00020     WallTile(Texture *texture, TransformComponent *transform);
00021
00022     void Render() const override;
00023
00024     [[nodiscard]]
00025     bool isObstacle() const override;
00026
00027     [[nodiscard]]
00028     TileType GetType() const override;
00029
00030 };

```

8.36 GameState.hpp

```

00001 #pragma once
00002
00003 class GameManager;
00004
00005 class GameStateOver;
00006
00007 class GameStateNormal;
00008
00009 class GameStateReset;
00010
00014 class GameState {
00015
00016 public:
00017
00018
00022     virtual ~GameState() = default;
00023
00027     virtual void Update() = 0;
00028
00029     virtual void FixedUpdate() = 0;
00030
00034     static void ProvideGameManager(GameManager &manager);
00035

```



```

00039     virtual void PlayerCollision() = 0;
00040
00044     virtual void InitGameState() = 0;
00045
00047     static GameStateNormal normal_;
00049     static GameStateReset reset_;
00051     static GameStateOver over_;
00052
00053 protected:
00054
00056     static GameManager *manager_;
00057 };

```

8.37 GameStateNormal.hpp

```

00001 #pragma once
00002
00003 #include "GameState.hpp"
00004
00008 class GameStateNormal : public GameState {
00009
00010 public:
00011
00015     GameStateNormal() = default;
00016
00017     void Update() override;
00018
00019     void FixedUpdate() override;
00020
00021     void PlayerCollision() override;
00022
00023     void InitGameState() override;
00024
00029     static void SetBonusAmount(int amount);
00030
00034     static void DecrementBonus();
00035
00036     static void AddScore(int score);
00037
00038     static int GetGameScore();
00039
00040     static int GetPlayerLives();
00041
00042 private:
00043
00047     void DisableControllers();
00048
00050     static int playerLives_;
00051
00053     static int bonuses_;
00054
00055     static int score_;
00056
00058     static constexpr int MAX_PLAYER_LIVES = 3;
00059
00060 };

```

8.38 GameStateOver.hpp

```

00001 #pragma once
00002
00003 #include "GameStateReset.hpp"
00004
00008 class GameStateOver : public GameStateReset {
00009
00010 public:
00011
00012     void Update() override;
00013
00014     void FixedUpdate() override;
00015
00016     void InitGameState() override;
00017
00018 private:
00019
00021     static constexpr float GAME_OVER_DELAY = 4.0f;
00022
00023 };

```

8.39 GameStateReset.hpp

```

00001 #pragma once
00002
00003 #include "GameState.hpp"
00004
00005 class GameStateReset : public GameState {
00006
00007 public:
00008     GameStateReset();
00009
00010     void Update() override;
00011
00012     void FixedUpdate () override;
00013
00014     void PlayerCollision() override;
00015
00016     void InitGameState() override;
00017
00018 protected:
00019
00020     float delay_;
00021
00022 private:
00023
00024     static void ResetControllers();
00025
00026     static constexpr float RESET_DELAY = 3.0f;
00027
00028 };

```

8.40 MapParser.hpp

```

00001 #pragma once
00002
00003 #include <vector>
00004 #include <string>
00005 #include <iostream>
00006
00007 #include "Vector2.hpp"
00008 #include "../game_objects/Tile.hpp"
00009 #include "MapReader.hpp"
00010
00011 using namespace std;
00012
00013 struct TileInfo {
00014
00015     TileType type_ = TileType::NULL_TILE;
00016     TileOption option_ = TileOption::NONE;
00017     Vector2<float> position_ = Vector2<float>::Zero();
00018     Vector2<float> direction_ = Vector2<float>::Zero();
00019
00020 };
00021
00022 class MapParser {
00023
00024 public:
00025
00026     MapParser() = default;
00027
00028     explicit MapParser(const vector<vector<TileData>> &tilesData, const MapHeader &header);
00029
00030     bool ParseMap(const vector<vector<TileData>> &tilesData, const MapHeader &header);
00031
00032     inline const vector<vector<TileInfo>> &GetTilesInfo() const;
00033
00034 private:
00035
00036     TileInfo ParseTile(TileData tileData, const Vector2<int> &tilePos);
00037
00038     static TileType ParseTileType(char tileType);
00039
00040     static TileOption ParseOptionType(char tileOption);
00041
00042     static Vector2<float> ParseTileDirection(char tileDirection);
00043
00044     Vector2<float> ParseTilePosition(const Vector2<int> &tilePosition);
00045
00046     vector<vector<TileInfo>> tilesInfo;
00047
00048     MapHeader header_;
00049 };

```

```

00101 inline const vector<vector<TileInfo> &MapParser::GetTilesInfo() const {
00102     return tilesInfo;
00103 }

```

8.41 MapReader.hpp

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <fstream>
00005 #include <string>
00006 #include <vector>
00007
00008 #include "Vector2.hpp"
00009
00010 using namespace std;
00011
00012 struct MapHeader {
00013     pair<size_t, size_t> mapSize_ = {0, 0};
00014     pair<Vector2<int>, Vector2<int> portalsPair_;
00015     pair<Vector2<int>, Vector2<int> spawnPoints_;
00016     bool noPortals_;
00017 };
00018
00019 struct TileData {
00020     char type_;
00021     char option_;
00022     char direction_;
00023 };
00024
00025 class MapReader {
00026 public:
00027     MapReader() = default;
00028     explicit MapReader(const string &path);
00029     ~MapReader();
00030     bool Read(const string &path);
00031     inline const vector<vector<TileData> &GetTilesData() const;
00032     inline const MapHeader GetMapHeader() const;
00033 private:
00034     bool OpenBrace();
00035     bool CloseBrace();
00036     bool CheckDataType(const string &dataType);
00037     bool ReadMaps();
00038     bool ReadMap(const string &type);
00039     bool InitMap();
00040     bool ReadTiles();
00041     bool ReadOptions();
00042     bool ReadDirections();
00043     bool OpenMapFile(const string &path);
00044     bool ReadMapHeader();
00045     bool ReadSize();
00046     bool ReadSpawns();
00047     bool ReadPosition(Vector2<int> &position);
00048     bool ReadPortals();
00049     MapHeader header_;
00050     ifstream mapFileInput_;
00051     vector<vector<TileData> > tilesData;

```

```

00187
00188 };
00189
00190 inline const vector<vector<TileData>> &MapReader::GetTilesData() const {
00191     return tilesData;
00192 }
00193
00194 inline const MapHeader MapReader::GetMapHeader() const {
00195     return header_;
00196 }
00197
00198

```

8.42 NullTexture.hpp

```

00001 #pragma once
00002
00003 #include "Texture.hpp"
00004
00005 class NullTexture : public Texture {
00006
00007 public:
00008
00009     bool LoadTexture(const std::string &path) override;
00010
00011     bool LoadTexture(const std::string &path, const TextureSize<float> &size) override;
00012
00013     void RenderTexture(const Vector2<float> &position) const override;
00014
00015     bool SetNativeTextureSize() override;
00016
00017 };

```

8.43 Text.hpp

```

00001 #pragma once
00002
00003 #include "Texture.hpp"
00004 #include <SDL2/SDL_ttf.h>
00005
00006
00007
00008 namespace TextColors {
00009     constexpr SDL_Color WHITE = {0xFF, 0xFF, 0xFF, 0xFF};
00010     constexpr SDL_Color BLACK = {0x00, 0x00, 0x00, 0xFF};
00011 }
00012
00013 class Text : public Texture {
00014 public:
00015
00016     Text();
00017
00018     ~Text() override;
00019
00020     Text(const Text &src) = delete;
00021
00022     bool LoadFont(const string &path, size_t fontSize = DefaultFontSize);
00023
00024     bool LoadText(const string &msg, SDL_Color color);
00025
00026     bool LoadText(const string &msg, SDL_Color color, const TextureSize<float> &size);
00027
00028     bool LoadTexture(const string &msg) override;
00029
00030     bool LoadTexture(const string &msg, const TextureSize<float> &size) override;
00031
00032     void RenderTexture(const Vector2<float> &position) const override;
00033
00034     void RenderTexture(const Vector2<float> &position, double angle) const override;
00035
00036     static constexpr size_t DefaultFontSize = 16;
00037
00038 private:
00039
00040     SDL_FRect CreateRenderRect(const Vector2<float> &position) const;
00041
00042     void DestroyFont();
00043
00044     TTF_Font * font_ ;
00045     size_t fontSize_;
00046     SDL_Color color_;

```

```

00047     string text_;
00048
00049 };

```

8.44 Texture.hpp

```

00001 #pragma once
00002
00003 #include <SDL2/SDL.h>
00004 #include <SDL2/SDL_image.h>
00005 #include <iostream>
00006 #include <string>
00007
00008 using namespace std;
00009
00010 #include "Vector2.hpp"
00011 #include "TextureSize.hpp"
00012
00016 class Texture {
00017 public:
00018     Texture();
00023     Texture(const Texture &src) = delete;
00024
00028     virtual ~Texture() noexcept;
00029
00033     Texture(Texture &&src) noexcept;
00034
00038     virtual bool LoadTexture(const string &path);
00047
00055     virtual bool LoadTexture(const string &path, const TextureSize<float> &size);
00056
00061     virtual void RenderTexture(const Vector2<float> &position) const;
00062
00063     virtual void RenderTexture(const Vector2<float> &position, double angle) const;
00064
00071     virtual bool SetNativeTextureSize();
00072
00077     inline void SetTextureSize(const TextureSize<float> &size);
00078
00083     [[nodiscard]]
00084     inline const TextureSize<float> &GetSize() const;
00085
00086 protected:
00087
00095     [[nodiscard]]
00096     SDL_FRect CreateRenderRectCentered(const Vector2<float> &position) const;
00097
00101     void DestroyTexture();
00102
00104     SDL_Texture *texture_;
00106     TextureSize<float> size_;
00107 };
00108
00109 inline void Texture::SetTextureSize(const TextureSize<float> &size) {
00110     size_ = size;
00111 }
00112
00113 inline const TextureSize<float> &Texture::GetSize() const {
00114     return size_;
00115 }

```

8.45 TextureSize.hpp

```

00001 #pragma once
00002
00007 template<typename T_>
00008 struct TextureSize {
00009
00015     TextureSize(T_ w = 0, T_ h = 0);
00016
00021     TextureSize(const TextureSize &src);
00022
00026     TextureSize &operator=(const TextureSize &rhs);
00027
00031     static TextureSize Zero();
00032
00033     // Arithmetics operators.
00034

```

```

00035     template<typename U_>
00036     friend TextureSize<U_> operator/(const TextureSize<U_> &lhs, const U_ &rhs);
00037
00038     template<typename U_, typename L_>
00039     friend TextureSize<U_> operator/(const TextureSize<U_> &lhs, const L_ &rhs);
00040
00042     T_ width_, height_;
00043 };
00044
00045 #include "TextureSize.inl"

```

8.46 TextureSize.inl

```

00001 #pragma once
00002
00003 #include "TextureSize.hpp"
00004
00005 template<typename T_>
00006 TextureSize<T_>::TextureSize(T_ w, T_ h)
00007     : width_(w)
00008     , height_(h) {}
00009
00010 template<typename T_>
00011 TextureSize<T_>::TextureSize(const TextureSize<T_> &src)
00012     : width_(src.width_)
00013     , height_(src.height_) {}
00014
00015 template<typename T_>
00016 TextureSize<T_> &TextureSize<T_>::operator=(const TextureSize<T_> &rhs) {
00017     if (this == &rhs)
00018         return *this;
00019     width_ = rhs.width_;
00020     height_ = rhs.height_;
00021     return *this;
00022 }
00023
00024 template<typename T_>
00025 TextureSize<T_> TextureSize<T_>::Zero() {
00026     return {0, 0};
00027 }
00028
00029 template<typename T_>
00030 TextureSize<T_> operator/(const TextureSize<T_> &lhs, const T_ &rhs) {
00031     return {lhs.width_ / rhs, lhs.height_ / rhs};
00032 }
00033
00034 template<typename T_, typename U_>
00035 TextureSize<T_> operator/(const TextureSize<T_> &lhs, const U_ &rhs) {
00036     return {lhs.width_ / static_cast<T_>(rhs), lhs.height_ / static_cast<T_>(rhs)};
00037 }
00038 }
00039
00040
00041
00042

```

8.47 TileNode.hpp

```

00001 #pragma once
00002
00003 #include "Vector2.hpp"
00004 #include "../game_objects/Tile.hpp"
00005
00009 class TileNode {
00010
00011 public:
00012
00016     explicit TileNode(Tile *wrapped);
00017
00021     ~TileNode();
00022
00027     inline TileType GetType() const;
00028
00032     inline void Render() const;
00033
00039     [[nodiscard]]
00040     inline bool isObstacle() const;
00041
00047     inline bool isTunnel() const;
00048
00054     inline bool isPortal() const;

```

```

00055
00060     inline void Teleport(GameActor &gameActor);
00061
00068     [[nodiscard]]
00069     inline bool CanTurnAside(const GameActor &gameActor) const;
00070
00077     inline bool AddBonus(Bonus *bonus);
00078
00084     inline bool CollectBonus();
00085
00087     inline TransformComponent *GetTransform();
00088
00090     inline const TransformComponent *GetTransform() const;
00091
00093     inline Tile *GetTile();
00094
00096     inline const Tile *GetTile() const;
00097
00103     [[nodiscard]]
00104     TileNode *NextNode(const Vector2<float> &direction);
00105
00111     static Vector2<float> GetOffsetFromNodeCenter(const GameActor &gameActor, const TileNode *node);
00112
00114     TileNode *upperNode;
00116     TileNode *bottomNode;
00118     TileNode *rightNode;
00120     TileNode *leftNode;
00121
00122 private:
00123
00125     Tile *wrapped_;
00126 };
00127
00128 inline TileType TileNode::GetType() const {
00129     return wrapped_->GetType();
00130 }
00131
00132 inline void TileNode::Render() const {
00133     wrapped_->Render();
00134 }
00135
00136 inline bool TileNode::isObstacle() const {
00137     return wrapped_->isObstacle();
00138 }
00139
00140 inline bool TileNode::CanTurnAside(const GameActor &gameActor) const {
00141     return wrapped_->CanTurnAside(gameActor);
00142 }
00143
00144 inline TransformComponent *TileNode::GetTransform() {
00145     return wrapped_->GetTransform();
00146 }
00147
00148 inline const TransformComponent *TileNode::GetTransform() const {
00149     return wrapped_->GetTransform();
00150 }
00151
00152 inline Tile *TileNode::GetTile() {
00153     return wrapped_;
00154 }
00155
00156 inline const Tile *TileNode::GetTile() const {
00157     return wrapped_;
00158 }
00159
00160 inline bool TileNode::AddBonus(Bonus *bonus) {
00161     return wrapped_->AddBonus(bonus);
00162 }
00163
00164 inline bool TileNode::CollectBonus() {
00165     return wrapped_->CollectBonus();
00166 }
00167
00168 bool TileNode::isTunnel() const {
00169     return wrapped_->isTunnel();
00170 }
00171
00172 bool TileNode::isPortal() const {
00173     return wrapped_->isPortal();
00174 }
00175
00176 void TileNode::Teleport(GameActor &gameActor) {
00177     wrapped_->Teleport(gameActor, *this);
00178 }

```

8.48 TileNodeBuilder.hpp

```

00001 #pragma once
00002
00003 #include "../game_objects/EmptyTile.hpp"
00004 #include "../game_objects/CrossTile.hpp"
00005 #include "../game_objects/WallTile.hpp"
00006 #include "../game_objects/TunnelTile.hpp"
00007 #include "../game_objects/PortalTile.hpp"
00008 #include "MapParser.hpp"
00009
00010 class GameFilesystem;
00011
00012 struct MapInfo {
00013     int bonuses_ = 0;
00014     TileNode *playerSpawn_ = nullptr;
00015     TileNode *ghostsSpawn_ = nullptr;
00016 };
00017
00018 class TileNodeBuilder {
00019 public:
00020     TileNodeBuilder();
00021
00022     void LoadTileNodes(const vector<vector<TileInfo>> &tilesInfo, const MapHeader &header);
00023
00024     void ClearTileNodes();
00025
00026     inline const vector<vector<TileNode *>> &GetTileNodesMap() const;
00027
00028     inline MapInfo GetMapInfo() const;
00029 private:
00030     bool CheckSpawnPoints();
00031
00032     bool CheckPortals();
00033
00034     void BuildTileNodes(const vector<vector<TileInfo>> &tilesInfo);
00035
00036     TileNode *BuildTileNode(const TileInfo &tileInfo);
00037
00038     void ConnectNodes();
00039
00040     void ConnectPortals();
00041
00042     void ConnectPortal(TileNode *entrance, TileNode *exit);
00043
00044     TileNode *BuildEmptyTileNode(const TileInfo &tileInfo);
00045
00046     TileNode *BuildPortalTileNode(const TileInfo &tileInfo);
00047
00048     TileNode *BuildCrossTileNode(const TileInfo &tileInfo);
00049
00050     TileNode *BuildWallTileNode(const TileInfo &tileInfo);
00051
00052     TileNode *BuildTunnelTileNode(const TileInfo &tileInfo);
00053
00054     TileNode *BuildNullTileNode(const TileInfo &tileInfo);
00055
00056     void AddSpawns();
00057
00058     void AddOption(TileNode *tileNode, TileOption option);
00059
00060     vector<vector<TileNode *>> tileNodes_;
00061     MapHeader header_;
00062     MapInfo mapInfo_;
00063     GameFilesystem &filesystem_;
00064 };
00065
00066 inline const vector<vector<TileNode *>> &TileNodeBuilder::GetTileNodesMap() const {
00067     return tileNodes_;
00068 }
00069
00070 inline MapInfo TileNodeBuilder::GetMapInfo() const {
00071     return mapInfo_;
00072 }

```

8.49 Vector2.hpp

```

00001 #pragma once
00002

```



```

00003 #include <iostream>
00004 #include <cmath>
00005
00006 template<typename T__>
00007 struct Vector2 {
00013     Vector2(T__ x = 0, T__ y = 0);
00014
00021     template<typename U__>
00022     Vector2(U__ x = 0, U__ y = 0);
00023
00027     Vector2(const Vector2 &src);
00028
00034     template<typename U__>
00035     Vector2(const Vector2<U__> &src);
00036
00040     Vector2 &operator=(const Vector2 &src);
00041
00047     template<typename U__>
00048     Vector2 &operator=(const Vector2<U__> &src);
00049
00051     static Vector2 Zero();
00052
00054     static Vector2 Up();
00055
00057     static Vector2 Down();
00058
00060     static Vector2 Right();
00061
00063     static Vector2 Left();
00064
00066     static Vector2 One();
00067
00072     static Vector2 Abs(const Vector2 &src);
00073
00078     static T__ Distance(const Vector2 &a, const Vector2 &b);
00079
00084     Vector2 ToLeft() const;
00085
00090     Vector2 ToRight() const;
00091
00096     Vector2 ToBack() const;
00097
00098     float ToDegrees() const;
00099
00104     template<typename U__>
00105     Vector2<U__> ConvertTo() const;
00106
00107     // Arithmetics operators.
00108
00109     template<typename U__>
00110     friend Vector2<U__> operator+(const Vector2<U__> &lhs, const Vector2<U__> &rhs);
00111
00112     template<typename U__>
00113     friend Vector2<U__> operator+(const Vector2<U__> &lhs, const U__ &rhs);
00114
00115     template<typename U__, typename L__>
00116     friend Vector2<U__> operator+(const Vector2<U__> &lhs, const L__ &rhs);
00117
00118     template<typename U__>
00119     friend Vector2<U__> operator-(const Vector2<U__> &src);
00120
00121     template<typename U__>
00122     friend Vector2<U__> operator-(const Vector2<U__> &lhs, const Vector2<U__> &rhs);
00123
00124     template<typename U__>
00125     friend Vector2<U__> operator-(const Vector2<U__> &lhs, const U__ &rhs);
00126
00127     template<typename U__, typename L__>
00128     friend Vector2<U__> operator-(const Vector2<U__> &lhs, const L__ &rhs);
00129
00130     template<typename U__>
00131     friend Vector2<U__> operator*(const Vector2<U__> &lhs, const U__ &rhs);
00132
00133     template<typename U__, typename L__>
00134     friend Vector2<U__> operator*(const Vector2<U__> &lhs, const L__ &rhs);
00135
00136     template<typename U__>
00137     friend Vector2<U__> operator*(const Vector2<U__> &lhs, const Vector2<U__> &rhs);
00138
00139     template<typename U__>
00140     friend Vector2<U__> operator/(const Vector2<U__> &lhs, const U__ &rhs);
00141
00142     template<typename U__, typename L__>
00143     friend Vector2<U__> operator/(const Vector2<U__> &lhs, const L__ &rhs);
00144
00145     template<typename U__>
00146     friend Vector2<U__> operator%(const Vector2<U__> &lhs, int rhs);

```

```

00147
00148 // Comparison operators.
00149
00150 template<typename U_>
00151 friend bool operator==(const Vector2<U_> &lhs, const Vector2<U_> &rhs);
00152
00153 template<typename U_>
00154 friend bool operator!=(const Vector2<U_> &lhs, const Vector2<U_> &rhs);
00155
00156 template<typename U_>
00157 friend bool operator<=(const Vector2<U_> &lhs, const Vector2<U_> &rhs);
00158
00159 template<typename U_>
00160 friend bool operator>=(const Vector2<U_> &lhs, const Vector2<U_> &rhs);
00161
00162 template<typename U_>
00163 friend bool operator<(const Vector2<U_> &lhs, const Vector2<U_> &rhs);
00164
00165 template<typename U_>
00166 friend bool operator>(const Vector2<U_> &lhs, const Vector2<U_> &rhs);
00167
00168 // Output operator.
00169
00170 template<typename U_>
00171 friend std::ostream &operator<<(std::ostream &os, const Vector2<U_> &src);
00172
00173 T_ x_, y_;
00174 };
00175
00176
00177 #include "Vector2.inl"

```

8.50 Vector2.inl

```

00001 #pragma once
00002
00003 #include <tuple>
00004
00005 #include "Vector2.hpp"
00006
00007 using std::tie;
00008
00009 template<typename T_>
00010 Vector2<T_>::Vector2(T_ x, T_ y)
00011     : x_(x)
00012     , y_(y) {}
00013
00014 template<typename T_>
00015 template<typename U_>
00016 Vector2<T_>::Vector2(U_ x, U_ y)
00017     : x_(static_cast<T_>(x))
00018     , y_(static_cast<T_>(y)) {}
00019
00020 template<typename T_>
00021 Vector2<T_>::Vector2(const Vector2<T_> &src)
00022     : x_(src.x_)
00023     , y_(src.y_) {}
00024
00025 template<typename T_>
00026 template<typename U_>
00027 Vector2<T_>::Vector2(const Vector2<U_> &src)
00028     : x_(static_cast<T_>(src.x_))
00029     , y_(static_cast<T_>(src.y_)) {}
00030
00031
00032 template<typename T_>
00033 Vector2<T_> &Vector2<T_>::operator=(const Vector2<T_> &src) {
00034
00035     if (this == &src)
00036         return *this;
00037
00038     x_ = src.x_;
00039     y_ = src.y_;
00040     return *this;
00041 }
00042
00043 template<typename T_>
00044 template<typename U_>
00045 Vector2<T_> &Vector2<T_>::operator=(const Vector2<U_> &src) {
00046
00047     x_ = static_cast<T_>(src.x_);
00048     y_ = static_cast<T_>(src.y_);
00049     return *this;
00050 }
00051

```

```

00052 template<typename T_>
00053 template<typename U_>
00054 Vector2<U_> Vector2<T_>::ConvertTo() const {
00055     return {static_cast<U_>(x_), static_cast<U_>(y_)};
00056 }
00057
00058 template<typename T_>
00059 Vector2<T_> Vector2<T_>::Zero() {
00060     return {0, 0};
00061 }
00062
00063 template<typename T_>
00064 Vector2<T_> Vector2<T_>::Up() {
00065     return {0, -1};
00066 }
00067
00068 template<typename T_>
00069 Vector2<T_> Vector2<T_>::Down() {
00070     return {0, 1};
00071 }
00072
00073 template<typename T_>
00074 Vector2<T_> Vector2<T_>::Right() {
00075     return {1, 0};
00076 }
00077
00078 template<typename T_>
00079 Vector2<T_> Vector2<T_>::Left() {
00080     return {-1, 0};
00081 }
00082
00083 template<typename T_>
00084 Vector2<T_> Vector2<T_>::One() {
00085     return {1, 1};
00086 }
00087
00088 template<typename T_>
00089 Vector2<T_> operator+(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00090     return {lhs.x_ + rhs.x_, lhs.y_ + rhs.y_};
00091 }
00092
00093 template<typename U_>
00094 Vector2<U_> operator+(const Vector2<U_> &lhs, const U_ &rhs) {
00095     return {lhs.x_ + rhs, lhs.y_ + rhs};
00096 }
00097
00098 template<typename U_, typename L_>
00099 Vector2<U_> operator+(const Vector2<U_> &lhs, const L_ &rhs) {
00100     return {lhs.x_ + static_cast<U_>(rhs),
00101             lhs.y_ + static_cast<U_>(rhs)};
00102 }
00103
00104 template<typename T_>
00105 Vector2<T_> operator-(const Vector2<T_> &src) {
00106     return {-src.x_, -src.y_};
00107 }
00108
00109 template<typename T_>
00110 Vector2<T_> operator-(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00111     return lhs + (-rhs);
00112 }
00113
00114 template<typename U_>
00115 Vector2<U_> operator-(const Vector2<U_> &lhs, const U_ &rhs) {
00116     return {lhs.x_ - rhs, lhs.y_ - rhs};
00117 }
00118
00119 template<typename T_>
00120 Vector2<T_> operator*(const Vector2<T_> &lhs, const T_ &rhs) {
00121     return {lhs.x_ * rhs, lhs.y_ * rhs};
00122 }
00123
00124 template<typename T_>
00125 Vector2<T_> operator*(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00126     return {lhs.x_ * rhs.x_, lhs.y_ * rhs.y_};
00127 }
00128
00129 template<typename T_, typename U_>
00130 Vector2<T_> operator*(const Vector2<T_> &lhs, const U_ &rhs) {
00131     return {lhs.x_ * static_cast<T_>(rhs),
00132             lhs.y_ * static_cast<T_>(rhs)};
00133 }
00134
00135 template<typename U_, typename L_>
00136 Vector2<U_> operator-(const Vector2<U_> &lhs, const L_ &rhs) {
00137     return {lhs.x_ - static_cast<U_>(rhs),
00138             lhs.y_ - static_cast<U_>(rhs)};

```

```

00139 }
00140
00141 template<typename T_>
00142 Vector2<T_> operator/(const Vector2<T_> &lhs, const T_ &rhs) {
00143     return {lhs.x_ / rhs, lhs.y_ / rhs};
00144 }
00145
00146 template<typename T_, typename U_>
00147 Vector2<T_> operator/(const Vector2<T_> &lhs, const U_ &rhs) {
00148     return {lhs.x_ / static_cast<T_>(rhs),
00149             lhs.y_ / static_cast<T_>(rhs)};
00150 }
00151
00152 template<typename T_>
00153 bool operator==(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00154     return tie(lhs.x_, lhs.y_) == tie(rhs.x_, rhs.y_);
00155 }
00156
00157 template<typename T_>
00158 bool operator!=(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00159     return tie(lhs.x_, lhs.y_) != tie(rhs.x_, rhs.y_);
00160 }
00161
00162 template<typename T_>
00163 bool operator<(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00164     return tie(lhs.x_, lhs.y_) < tie(rhs.x_, rhs.y_);
00165 }
00166
00167 template<typename T_>
00168 bool operator<=(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00169     return tie(lhs.x_, lhs.y_) <= tie(rhs.x_, rhs.y_);
00170 }
00171
00172 template<typename T_>
00173 bool operator>=(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00174     return tie(lhs.x_, lhs.y_) >= tie(rhs.x_, rhs.y_);
00175 }
00176
00177 template<typename T_>
00178 bool operator>(const Vector2<T_> &lhs, const Vector2<T_> &rhs) {
00179     return tie(lhs.x_, lhs.y_) > tie(rhs.x_, rhs.y_);
00180 }
00181
00182 template<typename T_>
00183 std::ostream &operator<<(std::ostream &os, const Vector2<T_> &src) {
00184     os << "{ " << src.x_ << " " << src.y_ << " }";
00185     return os;
00186 }
00187
00188 template<typename U_>
00189 Vector2<U_> operator%(const Vector2<U_> &lhs, int rhs) {
00190     return {lhs.x_ % rhs, lhs.y_ % rhs};
00191 }
00192
00193 template<typename T_>
00194 Vector2<T_> Vector2<T_>::Abs(const Vector2<T_> &src) {
00195     return {src.x_ < 0 ? -src.x_ : src.x_, src.y_ < 0 ? -src.y_ : src.y_};
00196 }
00197
00198 template<typename T_>
00199 Vector2<T_> Vector2<T_>::ToLeft() const {
00200     return {y_, -x_};
00201 }
00202
00203 template<typename T_>
00204 Vector2<T_> Vector2<T_>::ToRight() const {
00205     return {-y_, x_};
00206 }
00207
00208 template<typename T_>
00209 T_ Vector2<T_>::Distance(const Vector2<T_> &a, const Vector2<T_> &b) {
00210     Vector2<T_> c(a - b);
00211     return hypot(c.x_, c.y_);
00212 }
00213
00214 template<typename T_>
00215 Vector2<T_> Vector2<T_>::ToBack() const {
00216     return {-x_, -y_};
00217 }
00218
00219 template<typename T_>
00220 float Vector2<T_>::ToDegrees() const {
00221     return atan2(y_, x_) * 180 / M_PI;
00222 }

```

8.51 Canvas.hpp

```

00001 #pragma once
00002
00003 #include "UIElement.hpp"
00004 #include <vector>
00005 #include "../structures/Text.hpp"
00006
00007 using namespace std;
00008
00009 class Canvas : public UIElement {
00010 public:
00011
00012     Canvas(TransformComponent *transform, GraphicsComponent *graphics);
00013
00014     ~Canvas() override;
00015
00016     void Render() const override;
00017
00018     void Render(const Vector2<float> &position) const override;
00019
00020     void RenderScore() const;
00021
00022     void DecreaseLives();
00023
00024     void Update() override;
00025
00026 private:
00027
00028     vector<UIElement *> livesBar_;
00029     Texture *livesTexture;
00030     int playerLives;
00031     pair<UIElement *, int> scoreBar_;
00032     Text *scoreText_;
00033
00034     Vector2<float> center_;
00035     TextureSize<float> canvasSize_;
00036 };

```

8.52 UIElement.hpp

```

00001 #pragma once
00002
00003 #include "../game_objects/GameObject.hpp"
00004
00005 class UIElement : public GameObject {
00006
00007 public:
00008
00009     UIElement(TransformComponent *transform, GraphicsComponent *graphics);
00010
00011     ~UIElement() override;
00012
00013     void Update() override;
00014
00015     void FixedUpdate() override;
00016
00017     virtual void Render(const Vector2<float> &position) const;
00018
00019     virtual void Render() const;
00020
00021     inline Vector2<float> GetPosition() const;
00022
00023     void ChangeGraphicsTexture(const string &pathOrMessage);
00024
00025 protected:
00026
00027     TransformComponent *transform_;
00028
00029     GraphicsComponent *graphics_;
00030
00031 };
00032
00033
00034 inline Vector2<float> UIElement::GetPosition() const {
00035     return transform_->position_;
00036 }

```


Index

- Abs
 - Vector2< T_ >, [116](#)
- AddBonus
 - EmptyTile, [36](#)
 - Tile, [96](#)
 - TileNode, [100](#)
- AIComponent, [17](#)
 - AIComponent, [18](#)
 - FixedUpdate, [18](#)
 - ResetControlled, [18](#)
 - Update, [18](#)
- Bonus, [19](#)
 - Bonus, [20](#)
 - FixedUpdate, [20](#)
 - Render, [20](#)
 - Update, [20](#)
- CanTurnAside
 - CrossTile, [33](#)
 - EmptyTile, [36](#)
 - Tile, [96](#)
 - TileNode, [100](#)
- Canvas, [21](#)
 - Render, [22](#)
 - Update, [22](#)
- CatchPlayerCommand, [23](#)
 - Execute, [23](#)
- CentralizePosition
 - GameMap, [48](#)
- ChasePlayerCommand, [23](#)
 - CheckDirection, [24](#)
 - DecideDirection, [25](#)
 - Execute, [25](#)
- CheckDirection
 - ChasePlayerCommand, [24](#)
- CollectBonus
 - EmptyTile, [36](#)
 - Tile, [96](#)
 - TileNode, [101](#)
- Command, [25](#)
 - Execute, [26](#)
- Controller, [26](#)
 - Controller, [28](#)
 - FixedUpdate, [28](#)
 - Update, [28](#)
- ControllerComponent, [28](#)
 - ControllerComponent, [29](#)
 - ResetControlled, [30](#)
 - Update, [30](#)
- ConvertTo
 - Vector2< T_ >, [116](#)
- CreateRenderRectCentered
 - Texture, [90](#)
- CrossTile, [30](#)
 - CanTurnAside, [33](#)
 - CrossTile, [33](#)
 - GetType, [33](#)
 - Render, [33](#)
- DecideDirection
 - ChasePlayerCommand, [25](#)
- Distance
 - Vector2< T_ >, [116](#)
- Down
 - Vector2< T_ >, [116](#)
- EmptyTile, [33](#)
 - AddBonus, [36](#)
 - CanTurnAside, [36](#)
 - CollectBonus, [36](#)
 - EmptyTile, [36](#)
 - GetType, [37](#)
 - Render, [37](#)
- Execute
 - CatchPlayerCommand, [23](#)
 - ChasePlayerCommand, [25](#)
 - Command, [26](#)
 - MoveCommand, [67](#)
 - StopMoveCommand, [85](#)
 - TargetCommand, [86](#)
 - TurnDownCommand, [109](#)
 - TurnLeftCommand, [110](#)
 - TurnRightCommand, [110](#)
 - TurnUpCommand, [111](#)
- FixedUpdate
 - AIComponent, [18](#)
 - Bonus, [20](#)
 - Controller, [28](#)
 - GameStateNormal, [53](#)
 - GameStateOver, [55](#)
 - GameStateReset, [57](#)
 - Ghost, [61](#)
 - Pacman, [75](#)
 - PlayerController, [80](#)
 - Tile, [96](#)
 - UIElement, [113](#)
- Game, [37](#)

- GameActor, 38
 - GameActor, 40
 - Move, 40
 - Render, 40
 - StopMoving, 40
 - TurnDown, 40
 - TurnLeft, 40
 - TurnRight, 40
 - TurnTo, 40
 - TurnUp, 41
- GameAudio, 41
- GameEvents, 41
 - isQuit, 41
 - isQuitEvent, 41
- GameFilesystem, 42
- GameGraphics, 42
- GameLocator, 43
 - GetFilesystem, 44
 - GetMap, 44
 - GetTime, 44
 - ProvideAudio, 44
 - ProvideFilesystem, 44
 - ProvideGraphics, 44
 - ProvideManager, 44
 - ProvideMap, 45
 - ProvideTime, 45
- GameManager, 45
 - GameManager, 47
- GameMap, 47
 - CentralizePosition, 48
 - GameMap, 48
 - GetPixelsPosition, 48
 - LoadMap, 48
- GameObject, 49
 - Update, 49
- GameState, 49
 - InitGameState, 51
 - PlayerCollision, 51
 - Update, 51
- GameStateNormal, 51
 - FixedUpdate, 53
 - InitGameState, 53
 - PlayerCollision, 53
 - SetBonusAmount, 53
 - Update, 53
- GameStateOver, 54
 - FixedUpdate, 55
 - InitGameState, 55
 - Update, 55
- GameStateReset, 56
 - FixedUpdate, 57
 - InitGameState, 57
 - PlayerCollision, 57
 - Update, 57
- GameTime, 58
- GetFilesystem
 - GameLocator, 44
- GetMap
 - GameLocator, 44
- GetOffsetFromNodeCenter
 - TileNode, 101
- GetOffsetFromTileCenter
 - Tile, 96
- GetPixelsPosition
 - GameMap, 48
- GetSize
 - Texture, 90
- GetTextureSize
 - GraphicsComponent, 63
- GetTime
 - GameLocator, 44
- GetType
 - CrossTile, 33
 - EmptyTile, 37
 - NullTile, 72
 - PortalTile, 84
 - Tile, 97
 - TileNode, 101
 - TunnelTile, 108
 - WallTile, 120
- Ghost, 58
 - FixedUpdate, 61
 - Ghost, 61
 - Move, 61
 - Render, 61
 - StopMoving, 61
 - TurnDown, 61
 - TurnLeft, 61
 - TurnRight, 61
 - TurnTo, 62
 - TurnUp, 62
 - Update, 62
- GraphicsComponent, 62
 - GetTextureSize, 63
 - GraphicsComponent, 62
 - Render, 63
- InitGameState
 - GameState, 51
 - GameStateNormal, 53
 - GameStateOver, 55
 - GameStateReset, 57
- isObstacle
 - NullTile, 72
 - Tile, 97
 - TileNode, 101
 - WallTile, 120
- isOnNextTile
 - PhysicsComponent, 77
- isPortal
 - PortalTile, 84
 - Tile, 97
 - TileNode, 101
- isQuit
 - GameEvents, 41
- isQuitEvent
 - GameEvents, 41

- isTunnel
 - Tile, [97](#)
 - TileNode, [102](#)
 - TunnelTile, [108](#)
- Left
 - Vector2< T__ >, [116](#)
- LoadMap
 - GameMap, [48](#)
- LoadTexture
 - Text, [88](#)
 - Texture, [91](#)
- LoadTileNodes
 - TileNodeBuilder, [103](#)
- MapHeader, [63](#)
 - spawnPoints__, [63](#)
- MapInfo, [64](#)
- MapParser, [64](#)
 - MapParser, [65](#)
 - ParseMap, [65](#)
- MapReader, [65](#)
 - MapReader, [66](#)
 - Read, [66](#)
- Move
 - GameActor, [40](#)
 - Ghost, [61](#)
 - Pacman, [75](#)
 - PhysicsComponent, [78](#)
- MoveCommand, [66](#)
 - Execute, [67](#)
- NextNode
 - TileNode, [102](#)
- NullTexture, [67](#)
 - RenderTexture, [69](#)
 - SetNativeTextureSize, [69](#)
- NullTile, [69](#)
 - GetType, [72](#)
 - isObstacle, [72](#)
 - NullTile, [72](#)
 - Render, [72](#)
- One
 - Vector2< T__ >, [116](#)
- operator=
 - Vector2< T__ >, [116](#)
- Pacman, [1](#), [72](#)
 - FixedUpdate, [75](#)
 - Move, [75](#)
 - Pacman, [75](#)
 - Render, [75](#)
 - StopMoving, [75](#)
 - TryChangeDirection, [75](#)
 - TurnDown, [75](#)
 - TurnLeft, [76](#)
 - TurnRight, [76](#)
 - TurnTo, [76](#)
 - TurnUp, [76](#)
 - Update, [76](#)
- ParseMap
 - MapParser, [65](#)
- PhysicsComponent, [76](#)
 - isOnNextTile, [77](#)
 - Move, [78](#)
 - PhysicsComponent, [77](#)
 - TouchesObstacle, [78](#)
- PlayerCollision
 - GameState, [51](#)
 - GameStateNormal, [53](#)
 - GameStateReset, [57](#)
- PlayerController, [78](#)
 - FixedUpdate, [80](#)
 - PlayerController, [80](#)
 - ResetControlled, [80](#)
 - Update, [80](#)
- PortalTile, [81](#)
 - GetType, [84](#)
 - isPortal, [84](#)
 - PortalTile, [84](#)
 - Teleport, [84](#)
- ProvideAudio
 - GameLocator, [44](#)
- ProvideFilesystem
 - GameLocator, [44](#)
- ProvideGraphics
 - GameLocator, [44](#)
- ProvideManager
 - GameLocator, [44](#)
- ProvideMap
 - GameLocator, [45](#)
- ProvideTime
 - GameLocator, [45](#)
- Read
 - MapReader, [66](#)
- Render
 - Bonus, [20](#)
 - Canvas, [22](#)
 - CrossTile, [33](#)
 - EmptyTile, [37](#)
 - GameActor, [40](#)
 - Ghost, [61](#)
 - GraphicsComponent, [63](#)
 - NullTile, [72](#)
 - Pacman, [75](#)
 - Tile, [97](#)
 - WallTile, [121](#)
- RenderTexture
 - NullTexture, [69](#)
 - Text, [88](#), [89](#)
 - Texture, [91](#)
- ResetControlled
 - AIComponent, [18](#)
 - ControllerComponent, [30](#)
 - PlayerController, [80](#)
- Right

- Vector2< T_ >, 117
- SetBonusAmount
 - GameStateNormal, 53
- SetNativeTextureSize
 - NullTexture, 69
 - Texture, 91
- SetTextureSize
 - Texture, 92
- spawnPoints_
 - MapHeader, 63
- src/components/AI_components/AIComponent.hpp, 123
- src/components/AI_components/Blinky.hpp, 123
- src/components/AI_components/Clyde.hpp, 123
- src/components/AI_components/Inky.hpp, 123
- src/components/AI_components/Pinky.hpp, 123
- src/components/AICommands.hpp, 124
- src/components/AnimationComponent.hpp, 124
- src/components/Command.hpp, 124
- src/components/ControllerComponent.hpp, 125
- src/components/GraphicsComponent.hpp, 125
- src/components/PhysicsComponent.hpp, 125
- src/components/PlayerController.hpp, 126
- src/components/TransformComponent.hpp, 126
- src/game_engine/Game.hpp, 127
- src/game_engine/GameAudio.hpp, 127
- src/game_engine/GameEvents.hpp, 127
- src/game_engine/GameFilesystem.hpp, 128
- src/game_engine/GameGraphics.hpp, 129
- src/game_engine/GameLocator.hpp, 129
- src/game_engine/GameManager.hpp, 131
- src/game_engine/GameMap.hpp, 131
- src/game_engine/GameTime.hpp, 132
- src/game_objects/Bonus.hpp, 132
- src/game_objects/Controller.hpp, 133
- src/game_objects/CrossTile.hpp, 133
- src/game_objects/EmptyTile.hpp, 133
- src/game_objects/GameActor.hpp, 134
- src/game_objects/GameObject.hpp, 135
- src/game_objects/Ghost.hpp, 135
- src/game_objects/NullTile.hpp, 136
- src/game_objects/Pacman.hpp, 136
- src/game_objects/PortalTile.hpp, 137
- src/game_objects/Tile.hpp, 137
- src/game_objects/TunnelTile.hpp, 138
- src/game_objects/WallTile.hpp, 138
- src/game_states/GameState.hpp, 138
- src/game_states/GameStateNormal.hpp, 139
- src/game_states/GameStateOver.hpp, 139
- src/game_states/GameStateReset.hpp, 140
- src/structures/MapParser.hpp, 140
- src/structures/MapReader.hpp, 141
- src/structures/NullTexture.hpp, 142
- src/structures/Text.hpp, 142
- src/structures/Texture.hpp, 143
- src/structures/TextureSize.hpp, 143
- src/structures/TextureSize.inl, 144
- src/structures/TileNode.hpp, 144
- src/structures/TileNodeBuilder.hpp, 146
- src/structures/Vector2.hpp, 146
- src/structures/Vector2.inl, 148
- src/UI/Canvas.hpp, 151
- src/UI/UIElement.hpp, 151
- StopMoveCommand, 85
 - Execute, 85
- StopMoving
 - GameActor, 40
 - Ghost, 61
 - Pacman, 75
- TargetCommand, 85
 - Execute, 86
- Teleport
 - PortalTile, 84
 - Tile, 97
 - TileNode, 102
- Text, 86
 - LoadTexture, 88
 - RenderTexture, 88, 89
- Texture, 89
 - CreateRenderRectCentered, 90
 - GetSize, 90
 - LoadTexture, 91
 - RenderTexture, 91
 - SetNativeTextureSize, 91
 - SetTextureSize, 92
- TextureSize
 - TextureSize< T_ >, 93
- TextureSize< T_ >, 92
 - TextureSize, 93
 - Zero, 93
- TextureSizes, 15
- Tile, 93
 - AddBonus, 96
 - CanTurnAside, 96
 - CollectBonus, 96
 - FixedUpdate, 96
 - GetOffsetFromTileCenter, 96
 - GetType, 97
 - isObstacle, 97
 - isPortal, 97
 - isTunnel, 97
 - Render, 97
 - Teleport, 97
 - Tile, 95
 - Update, 98
- TileData, 98
- TileInfo, 98
- TileNode, 99
 - AddBonus, 100
 - CanTurnAside, 100
 - CollectBonus, 101
 - GetOffsetFromNodeCenter, 101
 - GetType, 101
 - isObstacle, 101
 - isPortal, 101
 - isTunnel, 102

- NextNode, [102](#)
- Teleport, [102](#)
- TileNodeBuilder, [102](#)
 - LoadTileNodes, [103](#)
- ToBack
 - Vector2< T_ >, [117](#)
- ToLeft
 - Vector2< T_ >, [117](#)
- ToRight
 - Vector2< T_ >, [117](#)
- ToucesObstacle
 - PhysicsComponent, [78](#)
- TransformComponent, [103](#)
 - TransformComponent, [104](#)
- TryChangeDirection
 - Pacman, [75](#)
- TunnelTile, [105](#)
 - GetType, [108](#)
 - isTunnel, [108](#)
 - TunnelTile, [107](#)
- TurnDown
 - GameActor, [40](#)
 - Ghost, [61](#)
 - Pacman, [75](#)
- TurnDownCommand, [108](#)
 - Execute, [109](#)
- TurnLeft
 - GameActor, [40](#)
 - Ghost, [61](#)
 - Pacman, [76](#)
- TurnLeftCommand, [109](#)
 - Execute, [110](#)
- TurnRight
 - GameActor, [40](#)
 - Ghost, [61](#)
 - Pacman, [76](#)
- TurnRightCommand, [110](#)
 - Execute, [110](#)
- TurnTo
 - GameActor, [40](#)
 - Ghost, [62](#)
 - Pacman, [76](#)
- TurnUp
 - GameActor, [41](#)
 - Ghost, [62](#)
 - Pacman, [76](#)
- TurnUpCommand, [111](#)
 - Execute, [111](#)
- UIElement, [112](#)
 - FixedUpdate, [113](#)
 - Update, [113](#)
- Up
 - Vector2< T_ >, [117](#)
- Update
 - AIComponent, [18](#)
 - Bonus, [20](#)
 - Canvas, [22](#)
 - Controller, [28](#)
 - ControllerComponent, [30](#)
 - GameObject, [49](#)
 - GameState, [51](#)
 - GameStateNormal, [53](#)
 - GameStateOver, [55](#)
 - GameStateReset, [57](#)
 - Ghost, [62](#)
 - Pacman, [76](#)
 - PlayerController, [80](#)
 - Tile, [98](#)
 - UIElement, [113](#)
- Vector2
 - Vector2< T_ >, [115](#)
- Vector2< T_ >, [113](#)
 - Abs, [116](#)
 - ConvertTo, [116](#)
 - Distance, [116](#)
 - Down, [116](#)
 - Left, [116](#)
 - One, [116](#)
 - operator=, [116](#)
 - Right, [117](#)
 - ToBack, [117](#)
 - ToLeft, [117](#)
 - ToRight, [117](#)
 - Up, [117](#)
 - Vector2, [115](#)
 - Zero, [117](#)
- WallTile, [118](#)
 - GetType, [120](#)
 - isObstacle, [120](#)
 - Render, [121](#)
 - WallTile, [120](#)
- Zero
 - TextureSize< T_ >, [93](#)
 - Vector2< T_ >, [117](#)